

Algorithmik – WS 26/27

Gelesen von Prof. Dr. Daniel Gaida

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Technology
Arts Sciences
TH Köln

Lernraum IV: Entwurf von Algorithmen

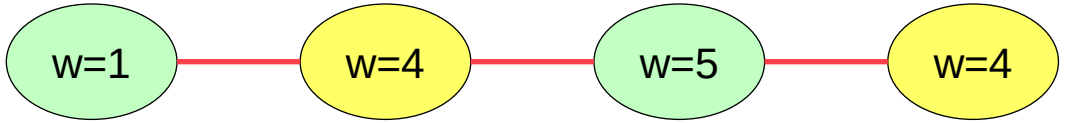
- Sortieralgorithmen als Beispiele für den Entwurf von Algorithmen
- **Dynamische Programmierung**
  - Memoization
  - Stabile Menge maximalen Gewichts in Pfadgraphen
  - Effiziente Berechnung der Fibonacci-Zahlen, 0-1-Rucksackproblem

Lernraum IV: Entwurf von Algorithmen

- Sortieralgorithmen als Beispiele für den Entwurf von Algorithmen
- **Dynamische Programmierung**
  - Memoization
  - Stabile Menge maximalen Gewichts in Pfadgraphen
  - Effiziente Berechnung der Fibonacci-Zahlen, 0-1-Rucksackproblem
- ~~P- vs. NP-Vollständigkeit~~
- ~~Caching, B-Baum~~

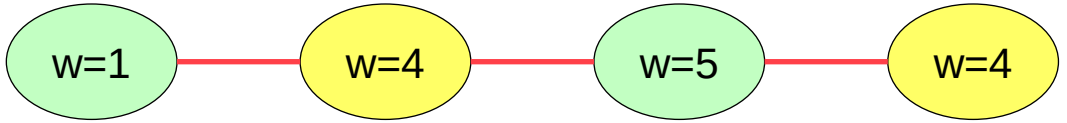
Stabile Menge maximalen Gewichts in Pfadgraphen

- Ein Pfadgraph ist ein Graph mit nur einem Pfad.
-
  -
-
  -
-
  -



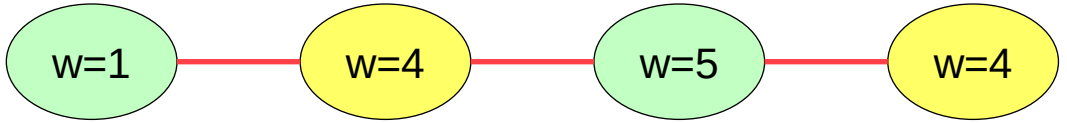
Stabile Menge maximalen Gewichts in Pfadgraphen

- Ein Pfadgraph ist ein Graph mit nur einem Pfad.
- Eine unabhängige Menge (oder stabile Menge) ist eine Teilmenge von Knoten eines Graphen, die zueinander nicht adjazent (benachbart) sind.
  -
-
  -
  -
-
  -
  -



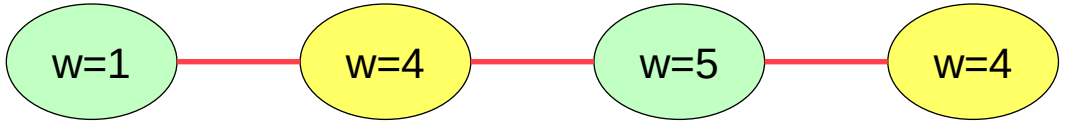
Stabile Menge maximalen Gewichts in Pfadgraphen

- Ein Pfadgraph ist ein Graph mit nur einem Pfad.
- Eine unabhängige Menge (oder stabile Menge) ist eine Teilmenge von Knoten eines Graphen, die zueinander nicht adjazent (benachbart) sind.
  - Hier: Die Menge der grünen und die Menge der gelben Knoten sind zwei stabile Mengen
-
  -
-
  -
  -



Stabile Menge maximalen Gewichts in Pfadgraphen

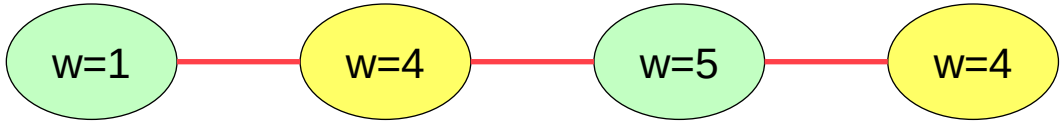
- Ein Pfadgraph ist ein Graph mit nur einem Pfad.
- Eine unabhängige Menge (oder stabile Menge) ist eine Teilmenge von Knoten eines Graphen, die zueinander nicht adjazent (benachbart) sind.
  - Hier: Die Menge der grünen und die Menge der gelben Knoten sind zwei stabile Mengen
- Eine **stabile Menge maximalen Gewichts** (SMMG) ist eine stabile Menge, in der die Summe der Knotengewichte maximal ist.
  -



-
  -
  -

Stabile Menge maximalen Gewichts in Pfadgraphen

- Ein Pfadgraph ist ein Graph mit nur einem Pfad.
- Eine unabhängige Menge (oder stabile Menge) ist eine Teilmenge von Knoten eines Graphen, die zueinander nicht adjazent (benachbart) sind.
  - Hier: Die Menge der grünen und die Menge der gelben Knoten sind zwei stabile Mengen
- Eine **stabile Menge maximalen Gewichts** (SMMG) ist eine stabile Menge, in der die Summe der Knotengewichte maximal ist.
  - Hier die beiden gelben Knoten mit Summe der Knotengewichte: 8

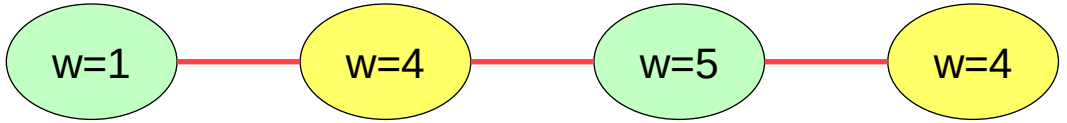


-
  -
  -

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Greedy-Algorithmen, Minimum Spanning Trees und dynamische Programmierung, Stanford University, Tim Roughgarden, Coursera

Stabile Menge maximalen Gewichts in Pfadgraphen

- Ein Pfadgraph ist ein Graph mit nur einem Pfad.
- Eine unabhängige Menge (oder stabile Menge) ist eine Teilmenge von Knoten eines Graphen, die zueinander nicht adjazent (benachbart) sind.
  - Hier: Die Menge der grünen und die Menge der gelben Knoten sind zwei stabile Mengen
- Eine **stabile Menge maximalen Gewichts** (SMMG) ist eine stabile Menge, in der die Summe der Knotengewichte maximal ist.
  - Hier die beiden gelben Knoten mit Summe der Knotengewichte: 8

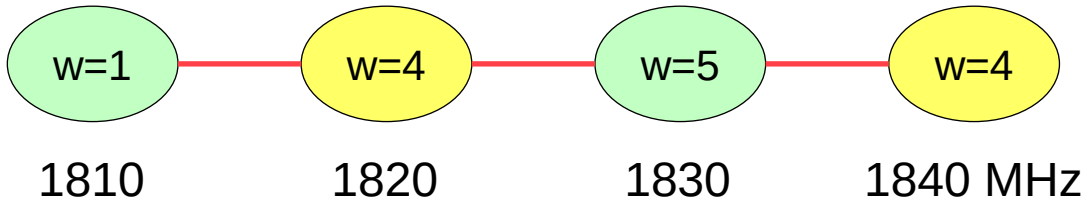


- Anwendungen:
  - Frequenzzuweisung in drahtlosen Netzen
  - Ressourcenallokation/Scheduling: Aufträge mit überlappender Laufzeit (Kante) können nicht beide auf derselben Ressource laufen. Wähle die Aufträge mit maximalem Gesamtnutzen (Knotengewicht).

Stabile Menge maximalen Gewichts in Pfadgraphen

Frequenzzuweisung in drahtlosen Netzen

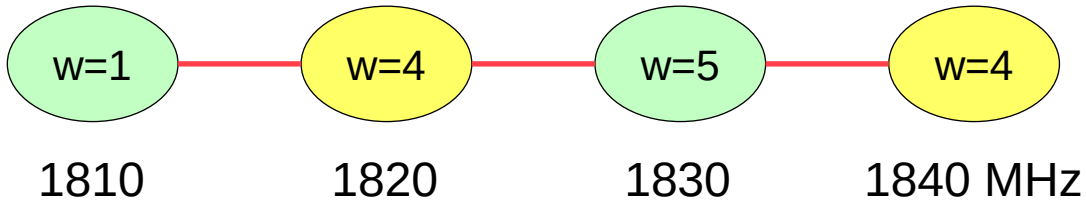
- Mehrere Nutzer möchten gleichzeitig über ein Mobilfunknetz kommunizieren
  - Ziel: Wähle die Menge gleichzeitig aktiver Kanäle mit maximaler Gesamtkapazität.
- Knoten: Kommunikationskanal (oder Frequenz)
- Kante: Zwei Kanäle stören sich, wenn sie gleichzeitig genutzt werden, z. B. weil sich ihre Frequenzbänder überlappen.
- Knotengewicht: Qualität/Kapazität des Kommunikationskanals (z. B. Datenrate in Mbit/s)



Stabile Menge maximalen Gewichts in Pfadgraphen

Frequenzzuweisung in drahtlosen Netzen

- Mehrere Nutzer möchten gleichzeitig über ein Mobilfunknetz kommunizieren
  - Ziel: Wähle die Menge gleichzeitig aktiver Kanäle mit maximaler Gesamtkapazität.
- Knoten: Kommunikationskanal (oder Frequenz)
- Kante: Zwei Kanäle stören sich, wenn sie gleichzeitig genutzt werden, z. B. weil sich ihre Frequenzbänder überlappen.
- Knotengewicht: Qualität/Kapazität des Kommunikationskanals (z. B. Datenrate in Mbit/s)

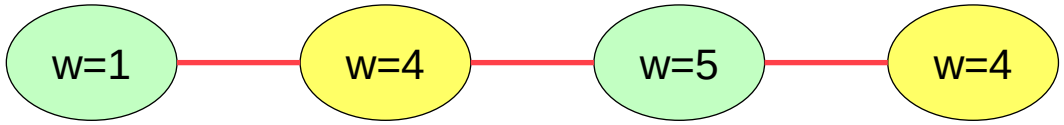


- Stabile Menge = Kanäle, die gleichzeitig ohne Störung laufen
- Stabile Menge maximalen Gewichts = die Auswahl mit dem größten Gesamtdurchsatz.

Stabile Menge maximalen Gewichts in Pfadgraphen

Iterativer Brute-Force Ansatz:

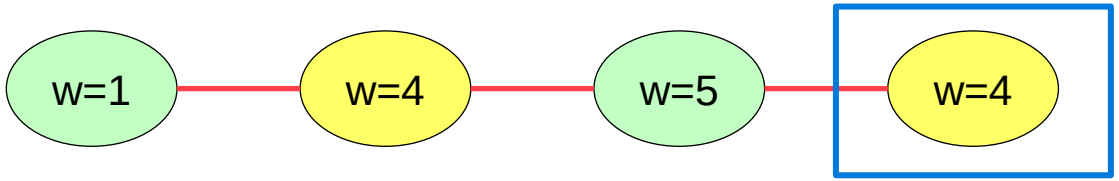
- Gehe alle Lösungsmöglichkeiten durch:
  -
  -
  -
  -



Stabile Menge maximalen Gewichts in Pfadgraphen

Iterativer Brute-Force Ansatz:

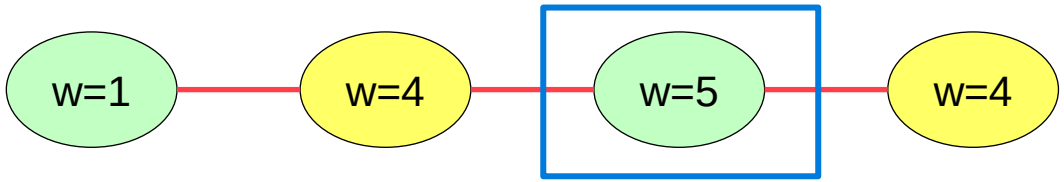
- Gehe alle Lösungsmöglichkeiten durch:
  - Iteration 1: 0001, Summe = 4
  -
  -
  -



Stabile Menge maximalen Gewichts in Pfadgraphen

Iterativer Brute-Force Ansatz:

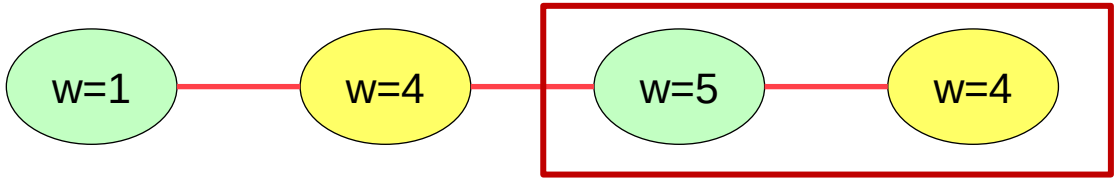
- Gehe alle Lösungsmöglichkeiten durch:
  - Iteration 1: 0001, Summe = 4
  - Iteration 2: 0010, Summe = 5
  -
  -



Stabile Menge maximalen Gewichts in Pfadgraphen

Iterativer Brute-Force Ansatz:

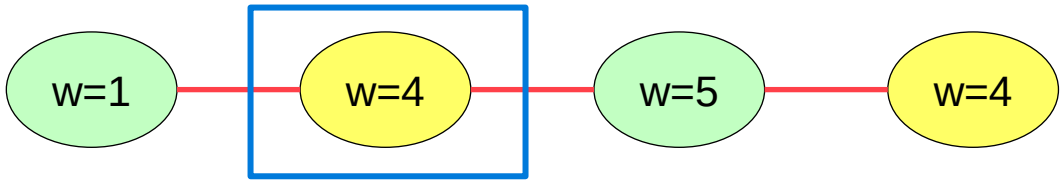
- Gehe alle Lösungsmöglichkeiten durch:
  - Iteration 1: 0001, Summe = 4
  - Iteration 2: 0010, Summe = 5
  - Iteration 3: 0011, ungültig
  -



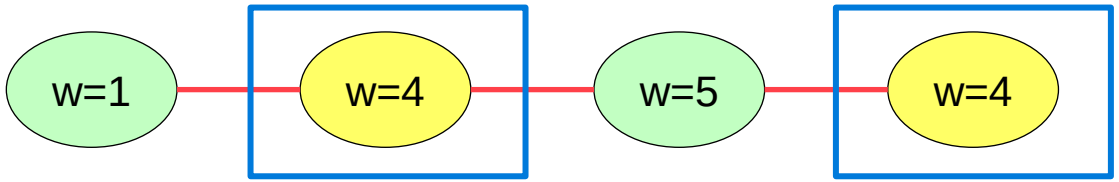
Stabile Menge maximalen Gewichts in Pfadgraphen

Iterativer Brute-Force Ansatz:

- Gehe alle Lösungsmöglichkeiten durch:
  - Iteration 1: 0001, Summe = 4
  - Iteration 2: 0010, Summe = 5
  - Iteration 3: 0011, ungültig
  - Iteration 4: 0100, Summe = 4



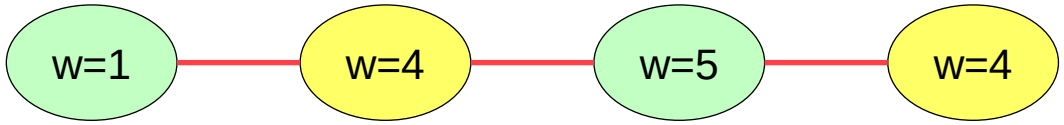
Stabile Menge maximalen Gewichts in Pfadgraphen



Iterativer Brute-Force Ansatz:

- Gehe alle Lösungsmöglichkeiten durch:
  - Iteration 1: 0001, Summe = 4
  - Iteration 2: 0010, Summe = 5
  - Iteration 3: 0011, ungültig
  - Iteration 4: 0100, Summe = 4
  - Iteration 5: 0101, Summe = 8
  -
  -
-
-

Stabile Menge maximalen Gewichts in Pfadgraphen



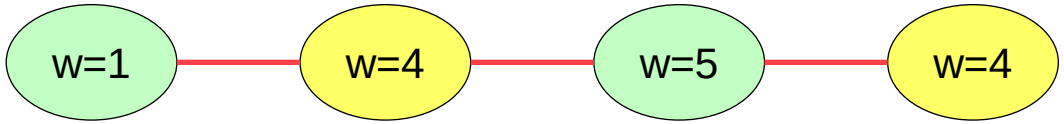
Iterativer Brute-Force Ansatz:

- Gehe alle Lösungsmöglichkeiten durch:
  - Iteration 1: 0001, Summe = 4
  - Iteration 2: 0010, Summe = 5
  - Iteration 3: 0011, ungültig
  - Iteration 4: 0100, Summe = 4
  - Iteration 5: 0101, Summe = 8
  - …
  - Iteration 15: 1111, ungültig

- Laufzeit: $O(2^n)$
- Korrekte Lösung wird gefunden

Stabile Menge maximalen Gewichts in Pfadgraphen

Rekursiver Brute-Force Ansatz



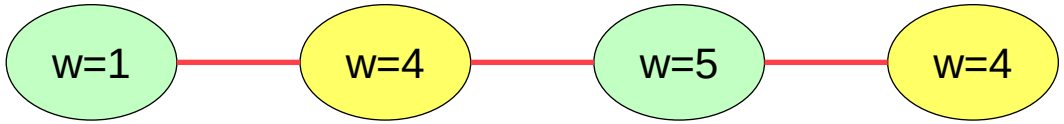
Rekursionsbaum:

SMMG:
0

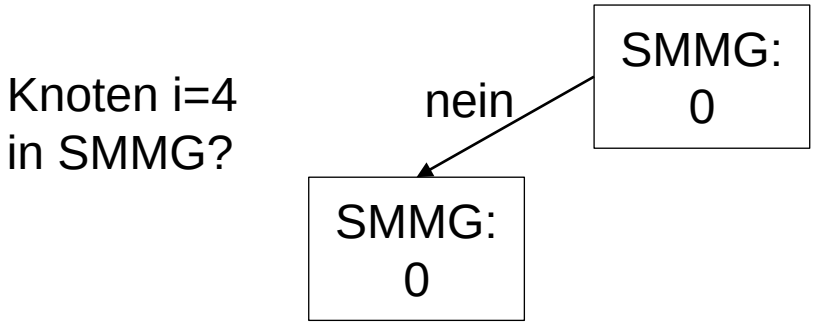
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Stabile Menge maximalen Gewichts in Pfadgraphen

Rekursiver Brute-Force Ansatz



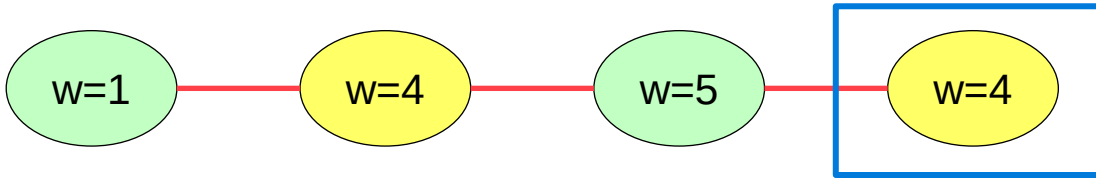
Rekursionsbaum:



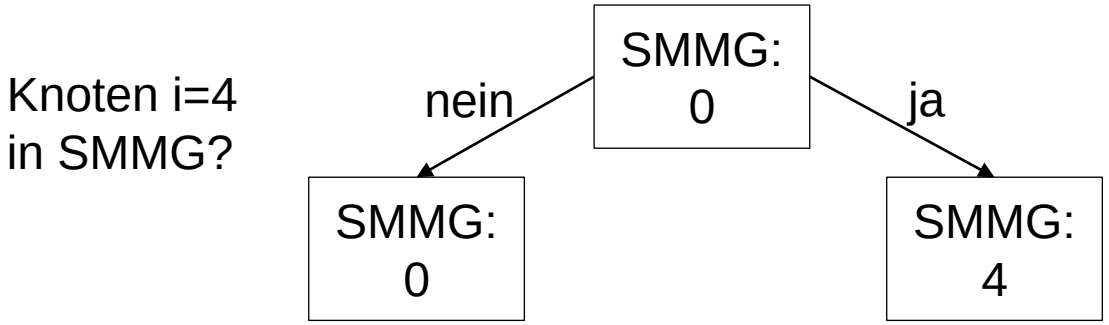
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology
Arts Sciences
TH Köln

Stabile Menge maximalen Gewichts in Pfadgraphen

Rekursiver Brute-Force Ansatz

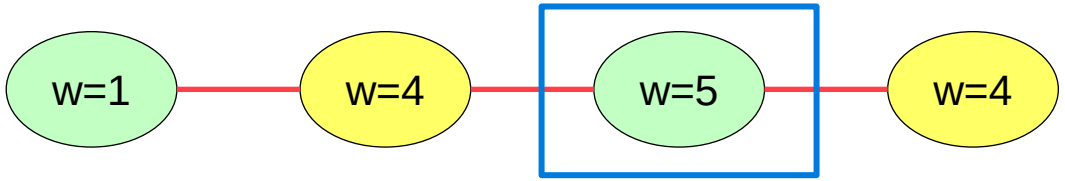


Rekursionsbaum:

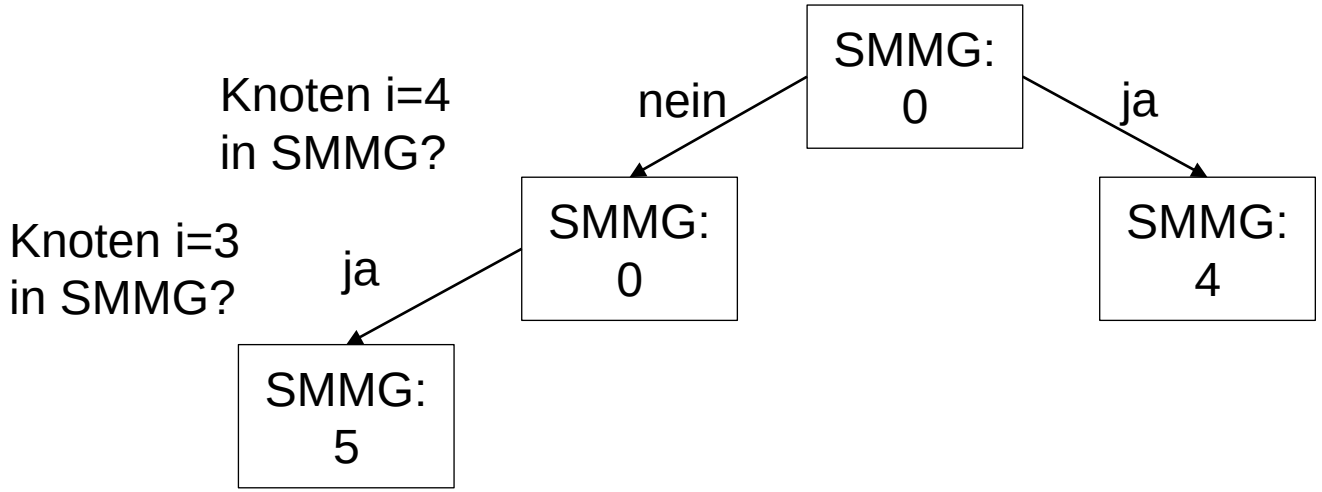


Stabile Menge maximalen Gewichts in Pfadgraphen

Rekursiver Brute-Force Ansatz



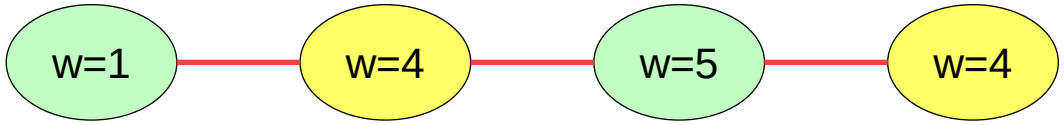
Rekursionsbaum:



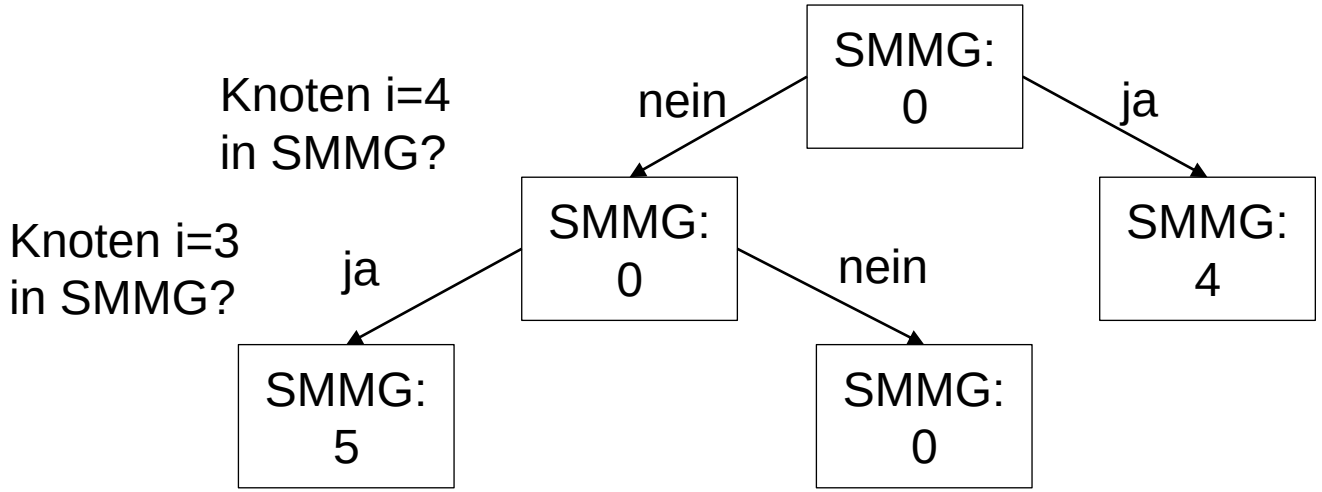
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology
Arts Sciences
TH Köln

Stabile Menge maximalen Gewichts in Pfadgraphen

Rekursiver Brute-Force Ansatz



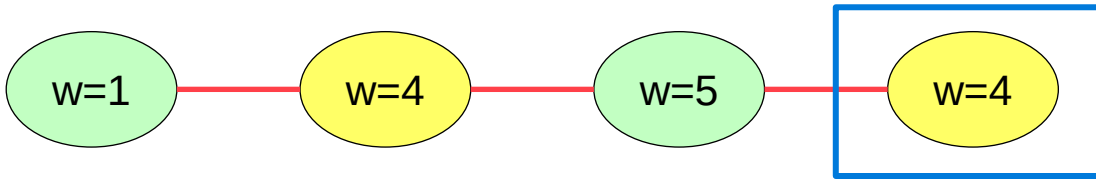
Rekursionsbaum:



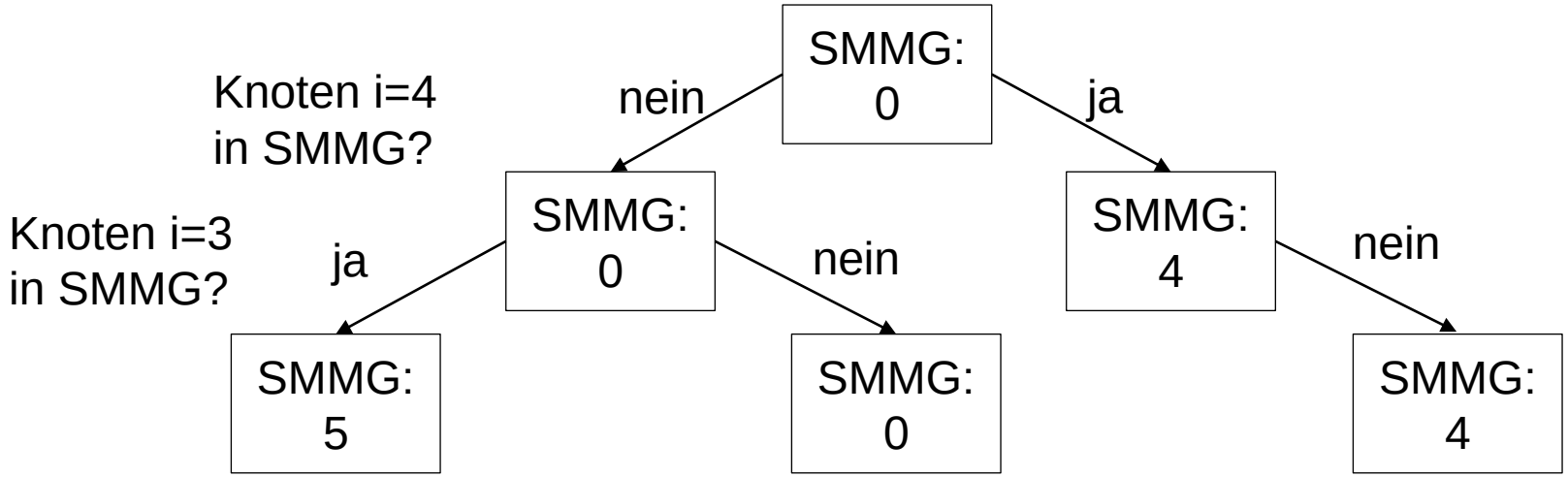
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology
Arts Sciences
TH Köln

Stabile Menge maximalen Gewichts in Pfadgraphen

Rekursiver Brute-Force Ansatz



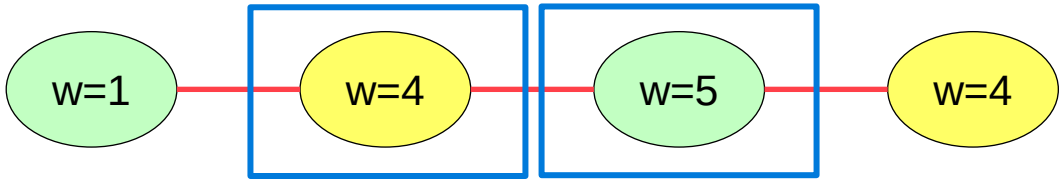
Rekursionsbaum:



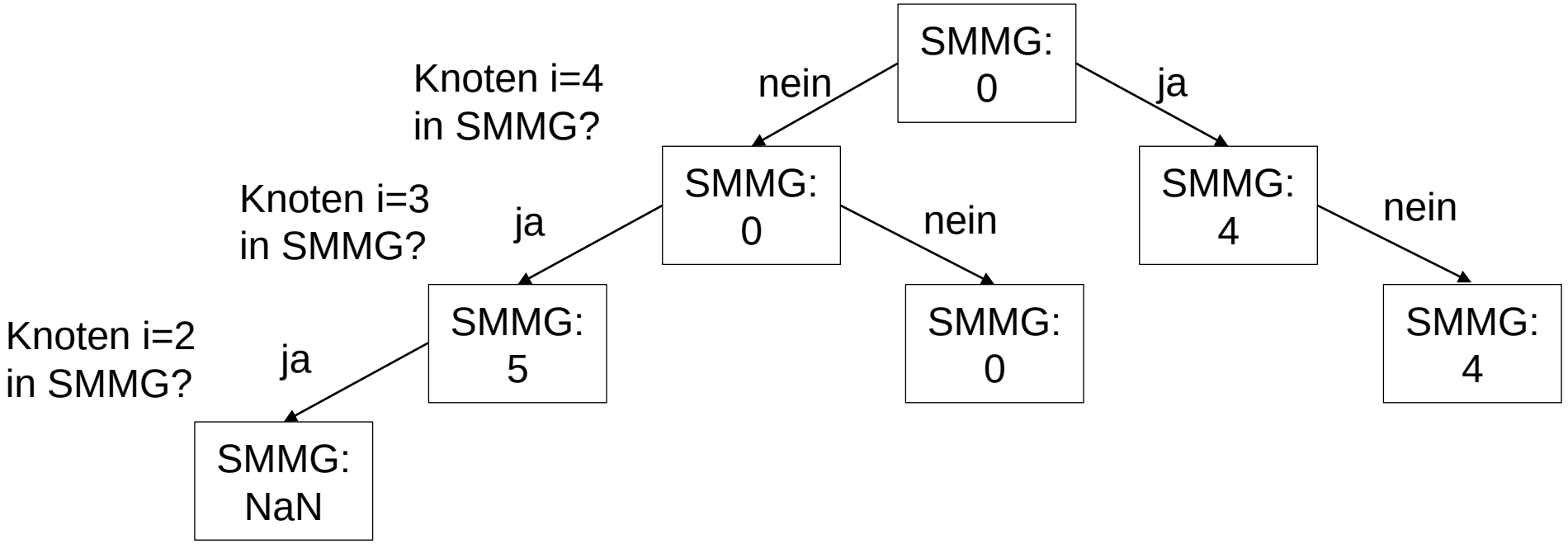
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology
Arts Sciences
TH Köln

Stabile Menge maximalen Gewichts in Pfadgraphen

Rekursiver Brute-Force Ansatz



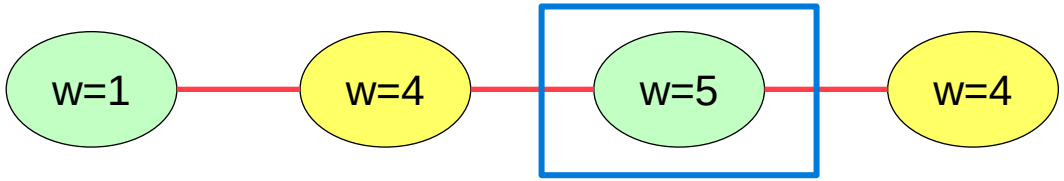
Rekursionsbaum:



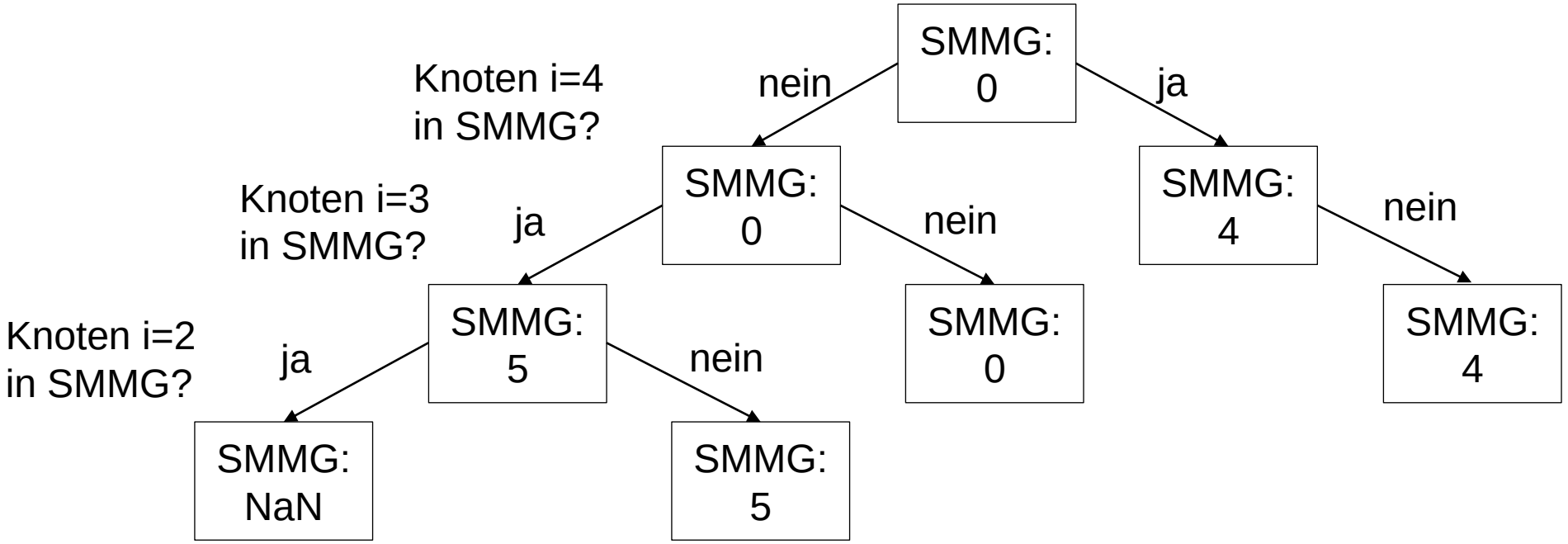
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Stabile Menge maximalen Gewichts in Pfadgraphen

Rekursiver Brute-Force Ansatz



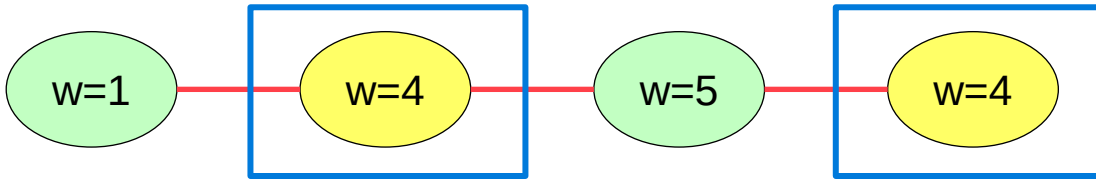
Rekursionsbaum:



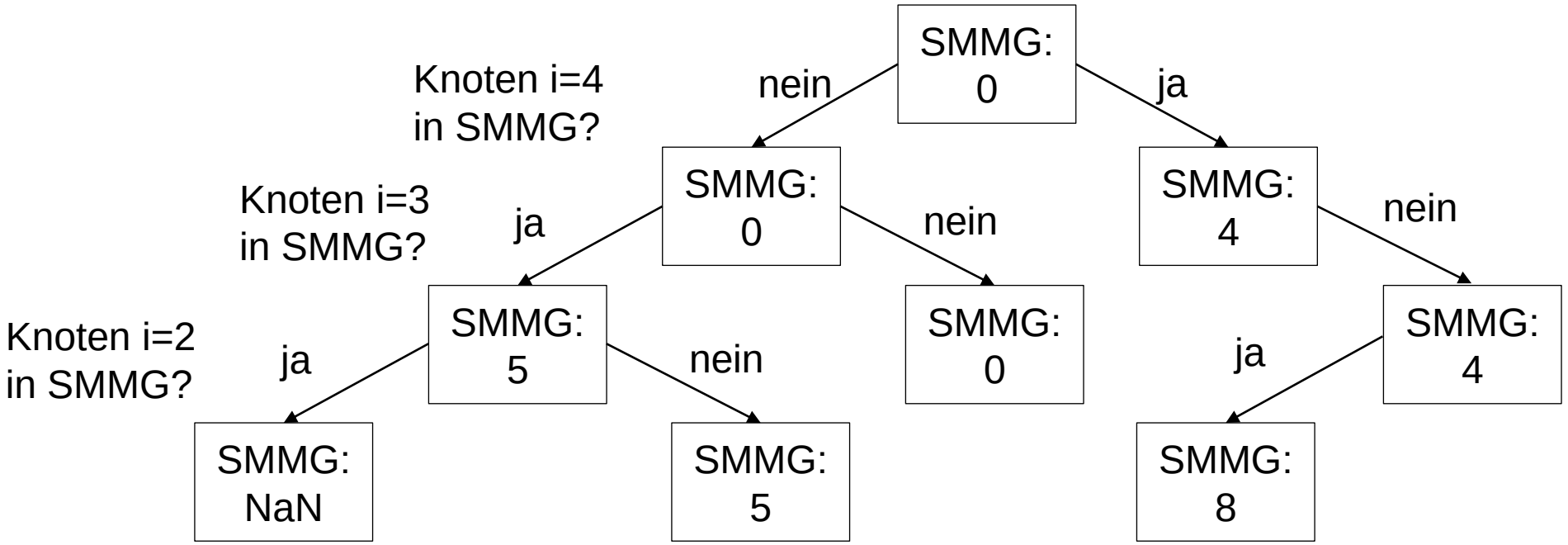
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Stabile Menge maximalen Gewichts in Pfadgraphen

Rekursiver Brute-Force Ansatz



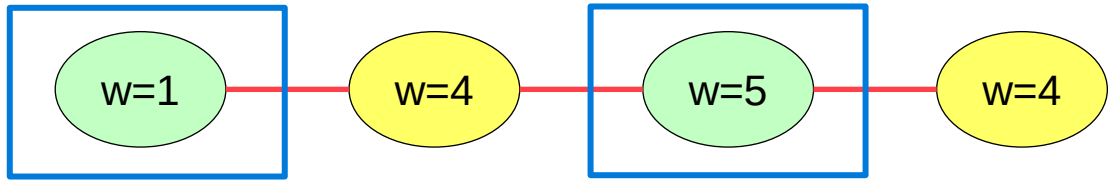
Rekursionsbaum:



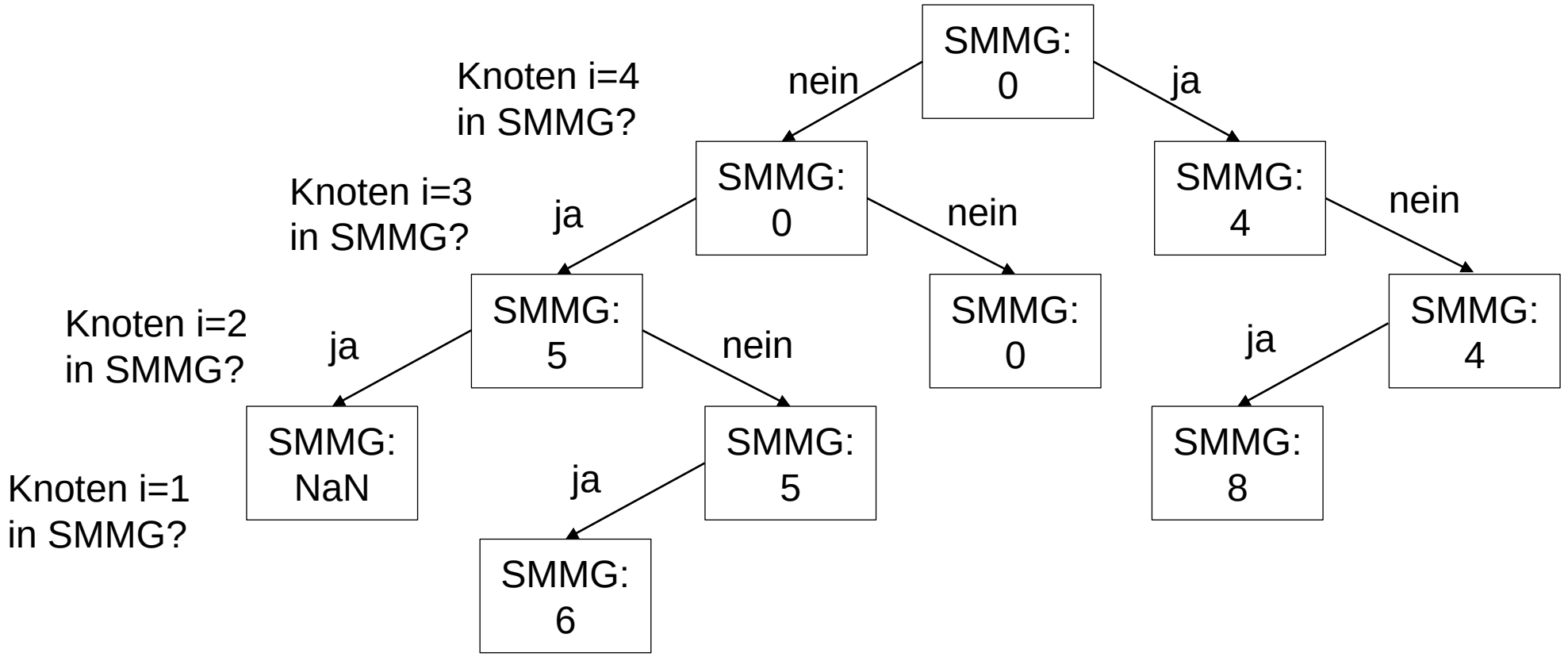
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology
Arts Sciences
TH Köln

Stabile Menge maximalen Gewichts in Pfadgraphen

Rekursiver Brute-Force Ansatz



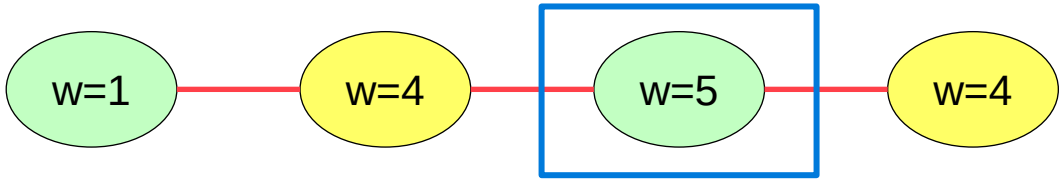
Rekursionsbaum:



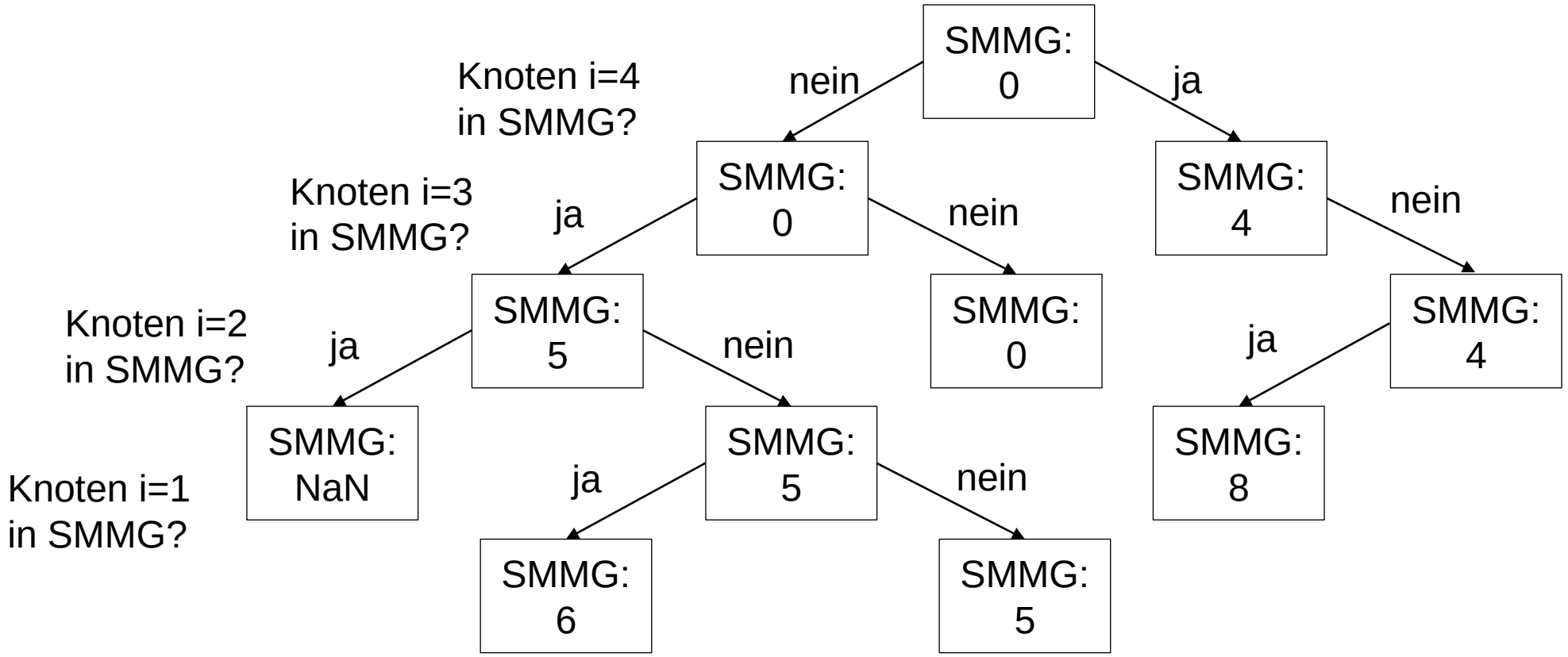
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Stabile Menge maximalen Gewichts in Pfadgraphen

Rekursiver Brute-Force Ansatz



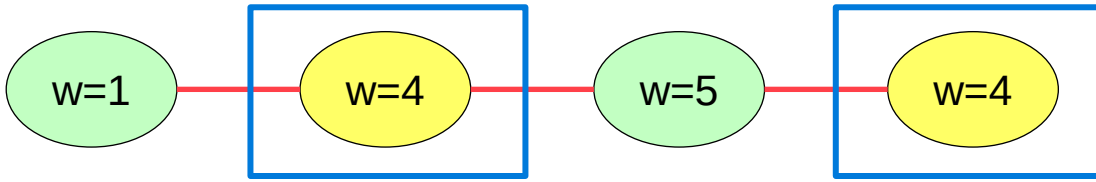
Rekursionsbaum:



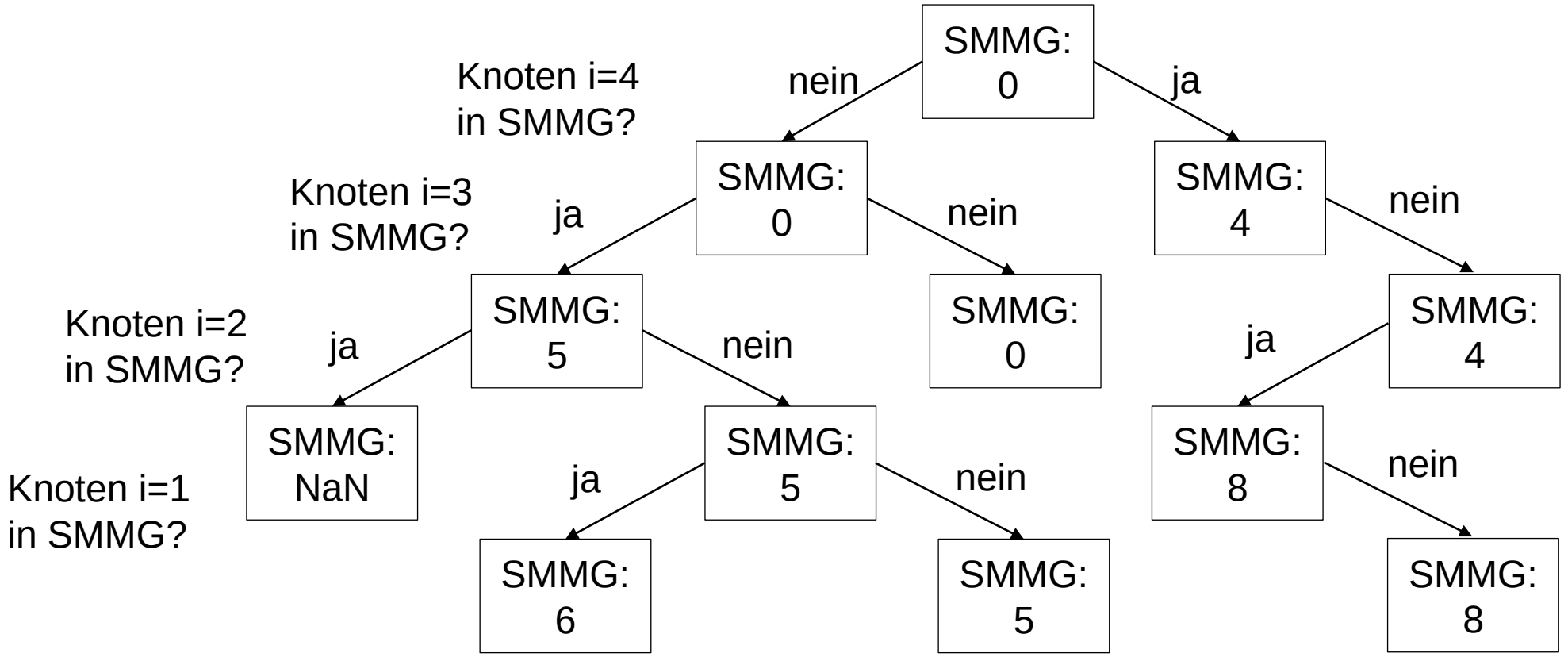
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Stabile Menge maximalen Gewichts in Pfadgraphen

Rekursiver Brute-Force Ansatz



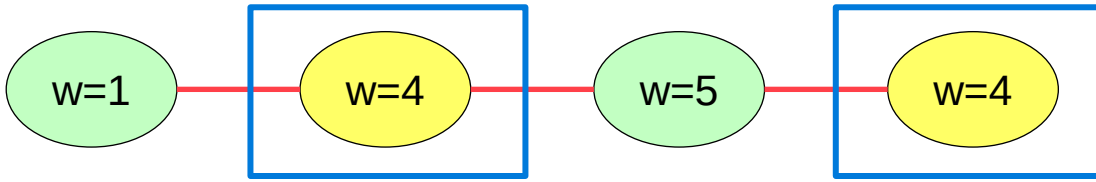
Rekursionsbaum:



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

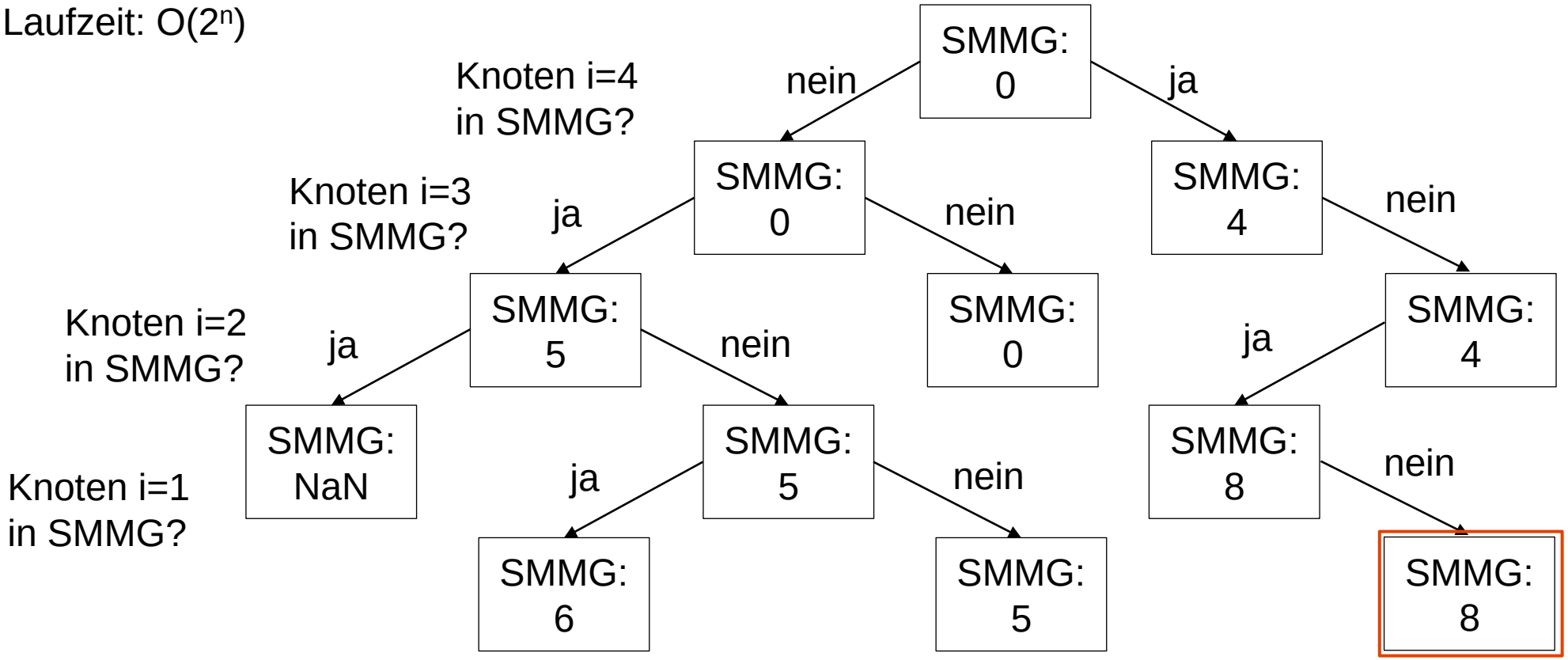
Stabile Menge maximalen Gewichts in Pfadgraphen

Rekursiver Brute-Force Ansatz



Rekursionsbaum:

Laufzeit: $O(2^n)$



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Greedy Algorithmus – Ein weiteres Entwurfsmuster

Tun in jeder Iteration das, was ihnen zu dieser Iteration am besten erscheint.

- „Sofortige Befriedigung“ - gierig
- „in jeder Iteration wird die lokal optimale Wahl getroffen“

Greedy Algorithmus – Ein weiteres Entwurfsmuster

Tun in jeder Iteration das, was ihnen zu dieser Iteration am besten erscheint.

- „Sofortige Befriedigung“ - gierig
- „in jeder Iteration wird die lokal optimale Wahl getroffen“

Dijkstra ist „gierig“ (greedy), denn sobald ein Knoten als „bearbeitet“ markiert ist, wird er nie wieder besucht.

Aus diesem Grund funktioniert Dijkstra nicht mit negativen Kantengewichten.

Greedy Algorithmus – Ein weiteres Entwurfsmuster

Tun in jeder Iteration das, was ihnen zu dieser Iteration am besten erscheint.

- „Sofortige Befriedigung“ - gierig
- „in jeder Iteration wird die lokal optimale Wahl getroffen“

Dijkstra ist „gierig“ (greedy), denn sobald ein Knoten als „bearbeitet“ markiert ist, wird er nie wieder besucht.

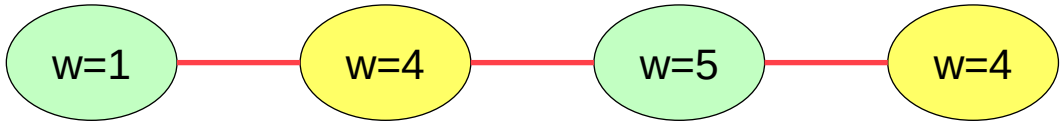
Aus diesem Grund funktioniert Dijkstra nicht mit negativen Kantengewichten.

Andere Beispiele für Greedy Algorithmen sind:

- Algorithmus von Kruskal und Algorithmus von Prim für minimale Spannbäume
- Gradientenabstieg (Mathematik oder KI)

Stabile Menge maximalen Gewichts in Pfadgraphen

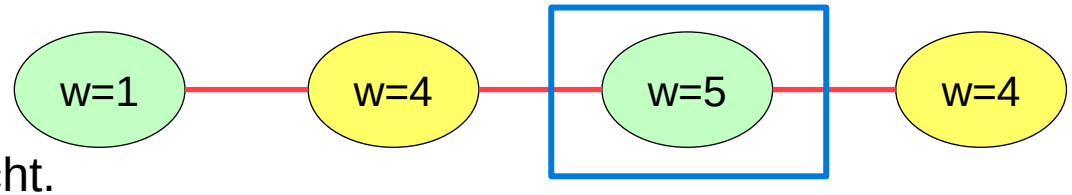
Greedy Algorithmus:



- Sortiere alle Knoten absteigend nach Knotengewicht.
  Gehe alle Knoten durch und füge einen Knoten hinzu, wenn wir diesen hinzufügen dürfen.
- Warum? Weil wir die stabile Menge **maximalen** Gewichts bestimmen möchten
  -
  -
  -
  -
-
-

Stabile Menge maximalen Gewichts in Pfadgraphen

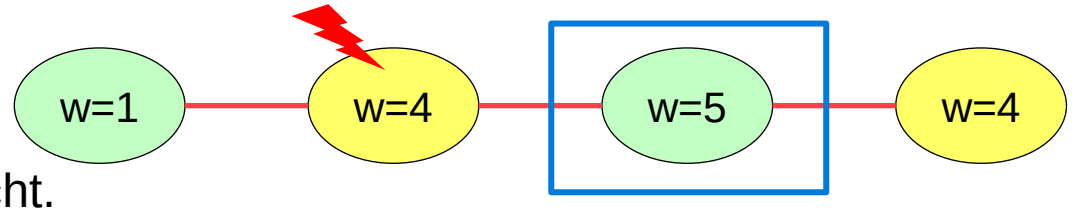
Greedy Algorithmus:



- Sortiere alle Knoten absteigend nach Knotengewicht.
  Gehe alle Knoten durch und füge einen Knoten hinzu, wenn wir diesen hinzufügen dürfen.
- Warum? Weil wir die stabile Menge **maximalen** Gewichts bestimmen möchten
  - Iteration 1: Knoten mit w=5 hinzufügen, Summe = 5
  -
  -
  -
-
-

Stabile Menge maximalen Gewichts in Pfadgraphen

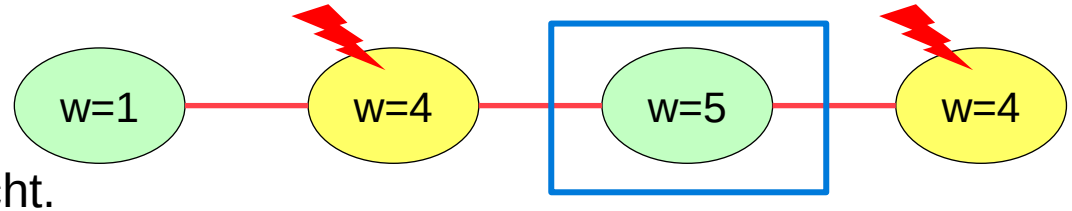
Greedy Algorithmus:



- Sortiere alle Knoten absteigend nach Knotengewicht.
  Gehe alle Knoten durch und füge einen Knoten hinzu, wenn wir diesen hinzufügen dürfen.
- Warum? Weil wir die stabile Menge **maximalen** Gewichts bestimmen möchten
  - Iteration 1: Knoten mit w=5 hinzufügen, Summe = 5
  - Iteration 2: Knoten mit w=4 überspringen
  -
  -
-
-

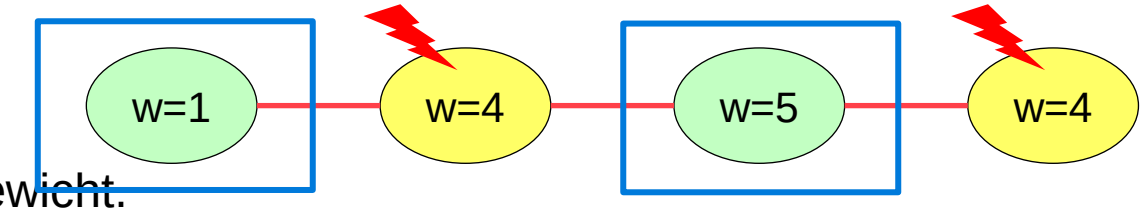
Stabile Menge maximalen Gewichts in Pfadgraphen

Greedy Algorithmus:



- Sortiere alle Knoten absteigend nach Knotengewicht.
  Gehe alle Knoten durch und füge einen Knoten hinzu, wenn wir diesen hinzufügen dürfen.
- Warum? Weil wir die stabile Menge **maximalen** Gewichts bestimmen möchten
  - Iteration 1: Knoten mit w=5 hinzufügen, Summe = 5
  - Iteration 2: Knoten mit w=4 überspringen
  - Iteration 3: Knoten mit w=4 überspringen
  -
-
-

Stabile Menge maximalen Gewichts in Pfadgraphen



Greedy Algorithmus:

- Sortiere alle Knoten absteigend nach Knotengewicht.
  Gehe alle Knoten durch und füge einen Knoten hinzu, wenn wir diesen hinzufügen dürfen.
- Warum? Weil wir die stabile Menge **maximalen** Gewichts bestimmen möchten
  - Iteration 1: Knoten mit w=5 hinzufügen, Summe = 5
  - Iteration 2: Knoten mit w=4 überspringen
  - Iteration 3: Knoten mit w=4 überspringen
  - Iteration 4: Knoten mit w=1 hinzufügen, Summe = 6

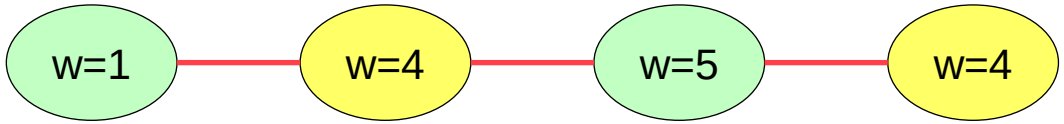
- Ergebnis: Summe der Knotengewichte: 6

- Greedy Algorithmus findet nicht die optimale Lösung, Laufzeit wäre aber O(n)

Stabile Menge maximalen Gewichts in Pfadgraphen

Teile und Beherrsche:

- Bestimme rekursiv die stabile Menge max. Gew.
  für die erste Hälfte des Graphen,
  tue das gleiche für die zweite Hälfte
  und kombiniere beide Lösungen
  -
  -
  -
-

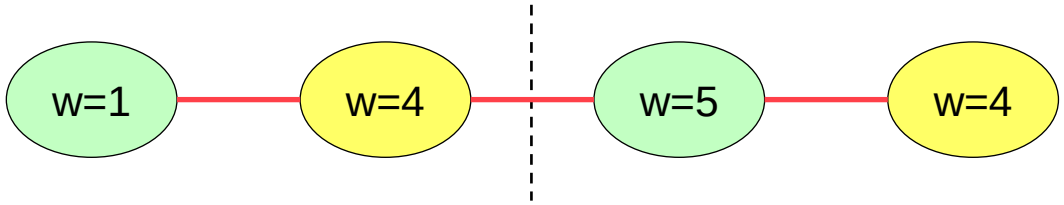


Quelle: Greedy-Algorithmen, Minimum Spanning Trees und dynamische Programmierung, Stanford University, Tim Roughgarden, Coursera

Stabile Menge maximalen Gewichts in Pfadgraphen

Teile und Beherrsche:

- Bestimme rekursiv die stabile Menge max. Gew.
  für die erste Hälfte des Graphen,
  tue das gleiche für die zweite Hälfte
  und kombiniere beide Lösungen
  -
  -
  -
-

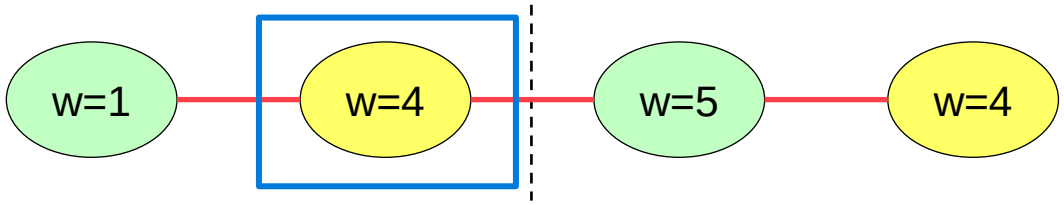



```
SMMG(nodes):
    if len(nodes) <= 2:
        return argmax(nodes)
    leftside = SMMG(nodes[:mid])
    rightside = SMMG(nodes[mid+1:])
    return leftside + rightside
```

Stabile Menge maximalen Gewichts in Pfadgraphen

Teile und Beherrsche:

- Bestimme rekursiv die stabile Menge max. Gew.
  für die erste Hälfte des Graphen,
  tue das gleiche für die zweite Hälfte
  und kombiniere beide Lösungen
  - Erste Hälfte: Stabile Menge max. Gew.: Knoten mit w=4
  -
  -
-



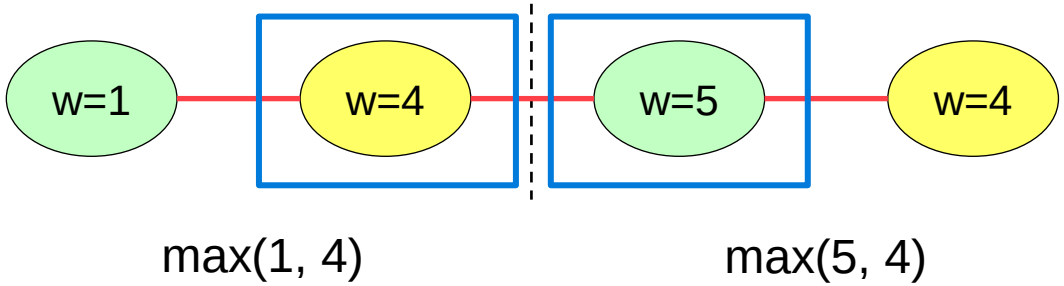
max(1, 4)

```
SMMG(nodes):
    if len(nodes) <= 2:
        return argmax(nodes)
    leftside = SMMG(nodes[:mid])
    rightside = SMMG(nodes[mid+1:])
    return leftside + rightside
```

Stabile Menge maximalen Gewichts in Pfadgraphen

Teile und Beherrsche:

- Bestimme rekursiv die stabile Menge max. Gew.
  für die erste Hälfte des Graphen,
  tue das gleiche für die zweite Hälfte
  und kombiniere beide Lösungen
  - Erste Hälfte: Stabile Menge max. Gew.: Knoten mit w=4
  - Zweite Hälfte: Stabile Menge max. Gew.: Knoten mit w=5
  -
-



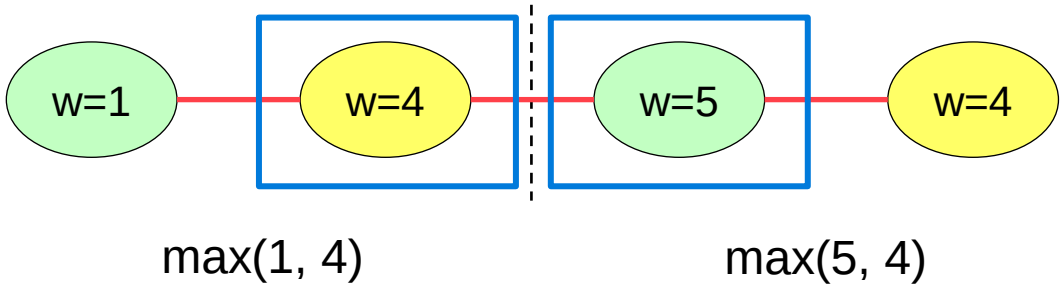

```
SMMG(nodes):
    if len(nodes) <= 2:
        return argmax(nodes)
    leftside = SMMG(nodes[:mid])
    rightside = SMMG(nodes[mid+1:])
    return leftside + rightside
```

Stabile Menge maximalen Gewichts in Pfadgraphen

Teile und Beherrsche:

- Bestimme rekursiv die stabile Menge max. Gew.
  für die erste Hälfte des Graphen,
  tue das gleiche für die zweite Hälfte
  und kombiniere beide Lösungen
  - Erste Hälfte: Stabile Menge max. Gew.: Knoten mit w=4
  - Zweite Hälfte: Stabile Menge max. Gew.: Knoten mit w=5
  - Kombination: ungültig, da beide Knoten benachbart sind

- Ergebnis: keine Lösung




```
SMMG(nodes):
    if len(nodes) <= 2:
        return argmax(nodes)
    leftside = SMMG(nodes[:mid])
    rightside = SMMG(nodes[mid+1:])
    return leftside + rightside
```

Stabile Menge maximalen Gewichts in Pfadgraphen

Entwurfsmuster	Laufzeit	Optimalität der Lösung
Iteratives Brute-Force	$O(2^n)$	Ja
Rekursives Brute-Force	$O(2^n)$	Ja
Greedy	O(n)	Nein, suboptimale Lösung
Divide and Conquer	O(log n)	Nein, keine Lösung

Stabile Menge maximalen Gewichts in Pfadgraphen

Entwurfsmuster	Laufzeit	Optimalität der Lösung
Iteratives Brute-Force	$O(2^n)$	Ja
Rekursives Brute-Force	$O(2^n)$	Ja
Greedy	O(n)	Nein, suboptimale Lösung
Divide and Conquer	O(log n)	Nein, keine Lösung

Wir kommen nachher auf das Problem zurück und lösen es mit der Technik der Dynamischen Programmierung

Was ist Dynamische Programmierung?

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Eine algorithmische Technik zur effizienten Lösung bestimmter Problemstellungen durch:

Aufteilung des Problems in Teilprobleme und systematische Speicherung der Ergebnisse der Teilprobleme

- Aufbauend auf „Divide and Conquer“
-
-
-

Was ist Dynamische Programmierung?

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Eine algorithmische Technik zur effizienten Lösung bestimmter Problemstellungen durch:

Aufteilung des Problems in Teilprobleme und systematische Speicherung der Ergebnisse der Teilprobleme

- Aufbauend auf „Divide and Conquer“
- Zuerst werden Lösungen der kleinsten Teilprobleme berechnet und dann geeignet zu einer Lösung eines nächstgrößeren Teilproblems zusammengesetzt.
-
-



Was ist Dynamische Programmierung?

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Eine algorithmische Technik zur effizienten Lösung bestimmter Problemstellungen durch:

Aufteilung des Problems in Teilprobleme und systematische Speicherung der Ergebnisse der Teilprobleme

- Aufbauend auf „Divide and Conquer“

- Zuerst werden Lösungen der kleinsten Teilprobleme berechnet und dann geeignet zu einer Lösung eines nächstgrößeren Teilproblems zusammengesetzt.
- Dieses Verfahren fortsetzen, bis das ursprüngliche Problem gelöst wurde.
-



Was ist Dynamische Programmierung?

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Eine algorithmische Technik zur effizienten Lösung bestimmter Problemstellungen durch:

Aufteilung des Problems in Teilprobleme und systematische Speicherung der Ergebnisse der Teilprobleme

- Aufbauend auf „Divide and Conquer“

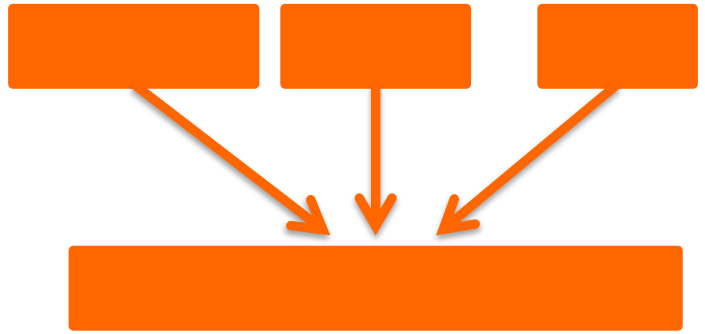
- Zuerst werden Lösungen der kleinsten Teilprobleme berechnet und dann geeignet zu einer Lösung eines nächstgrößeren Teilproblems zusammengesetzt.
- Dieses Verfahren fortsetzen, bis das ursprüngliche Problem gelöst wurde.
- **Memoization**: Berechnete Teilergebnisse werden gespeichert. Bei nachfolgenden Berechnungen gleichartiger Teilprobleme wird auf diese Zwischenlösungen zurückgegriffen, anstatt sie jedes Mal neu zu berechnen.



Vergleich dynamische Programmierung mit Divide & Conquer

Divide & Conquer

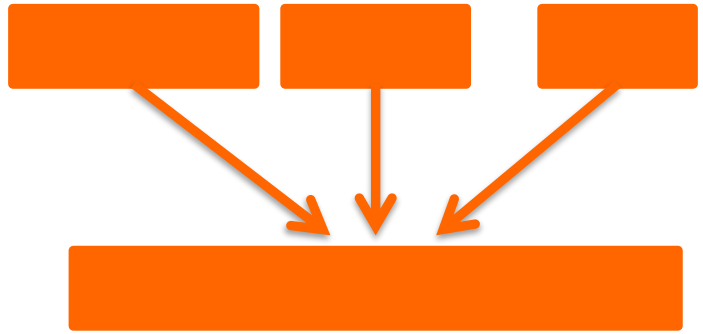
- unabhängige Teilprobleme
- separat lösen
- Lösungen zusammenführen -> Originalproblem gelöst



Vergleich dynamische Programmierung mit Divide & Conquer

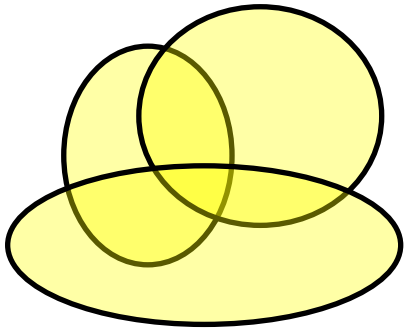
Divide & Conquer

- unabhängige Teilprobleme
- separat lösen
- Lösungen zusammenführen -> Originalproblem gelöst

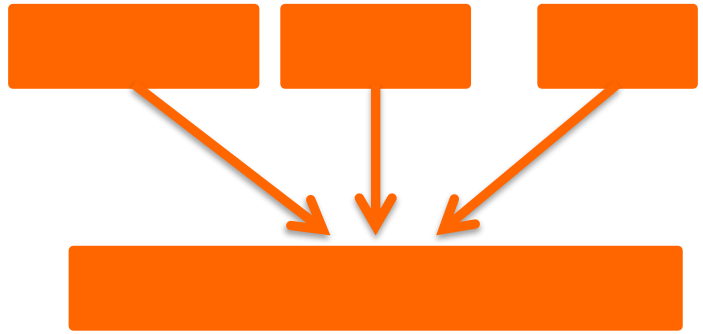


Dynamische Programmierung

- überlappende Teilprobleme
- kleine Teilprobleme lösen
- Lösungen wiederverwenden



Vergleich dynamische Programmierung mit Divide & Conquer

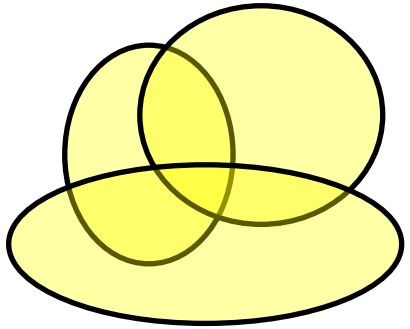


Divide & Conquer

- unabhängige Teilprobleme
- separat lösen
- Lösungen zusammenführen -> Originalproblem gelöst

Dynamische Programmierung

- überlappende Teilprobleme
- kleine Teilprobleme lösen
- Lösungen wiederverwenden



Wenn Teilprobleme nur einmal auftreten, dann
Divide and Conquer wählen, sonst dyn. Programmierung
in Betracht ziehen

Vorgehensweise bei dynamischer Programmierung

Herausforderungen:

- Identifizierung, ob Problem in Teilprobleme zerlegt werden kann und wie die Teilprobleme lauten
-
-
-
  -

Vorgehensweise bei dynamischer Programmierung

Herausforderungen:

- Identifizierung, ob Problem in Teilprobleme zerlegt werden kann und wie die Teilprobleme lauten

Vorgehensweise:

- Entwurf einer rekursiven „Brute-Force“-Lösung (keine Technik der dyn. Programmierung)
- Identifizierung der Teilprobleme und speichern der Teilergebnisse für künftige rekursive Aufrufe (Memoization)
-

Vorgehensweise bei dynamischer Programmierung

Herausforderungen:

- Identifizierung, ob Problem in Teilprobleme zerlegt werden kann und wie die Teilprobleme lauten

Vorgehensweise:

- Entwurf einer rekursiven „Brute-Force“-Lösung (keine Technik der dyn. Programmierung)
- Identifizierung der Teilprobleme und speichern der Teilergebnisse für künftige rekursive Aufrufe (Memoization)
- Alternative: Bottom-Up-Ansatz (iterativ)
  - Lösen der Teilprobleme in aufsteigender Reihenfolge (von „klein“ nach „groß“)

Beispiel: Effiziente Berechnung der Fibonacci-Zahlen

Gegeben eine Zahl n, Ausgabe der n-ten Fibonacci-Zahl (0 ist die 0-te Zahl).

Beispiel:

- n = 9; Fibonacci-Zahlen: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34
- Ausgabe: 34

Beispiel: Effiziente Berechnung der Fibonacci-Zahlen

Gegeben eine Zahl n, Ausgabe der n-ten Fibonacci-Zahl (0 ist die 0-te Zahl).

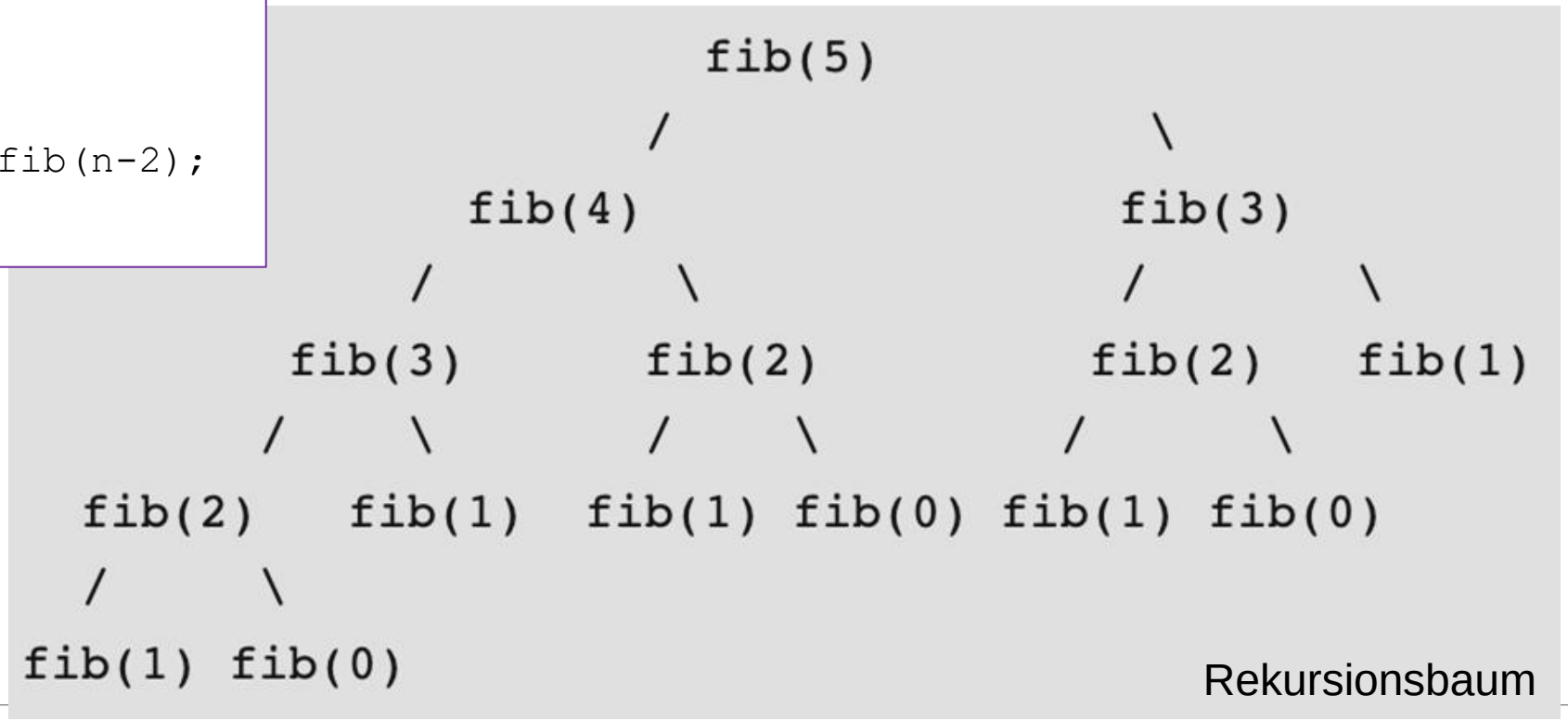
Beispiel:

- n = 9; Fibonacci-Zahlen: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34
- Ausgabe: 34

Wie können wir dieses Problem auf die bestmögliche Weise lösen?

Fibonacci-Zahlen: Rekursiver Brute-Force Ansatz

```
public static int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n-1) + fib(n-2);
}
```



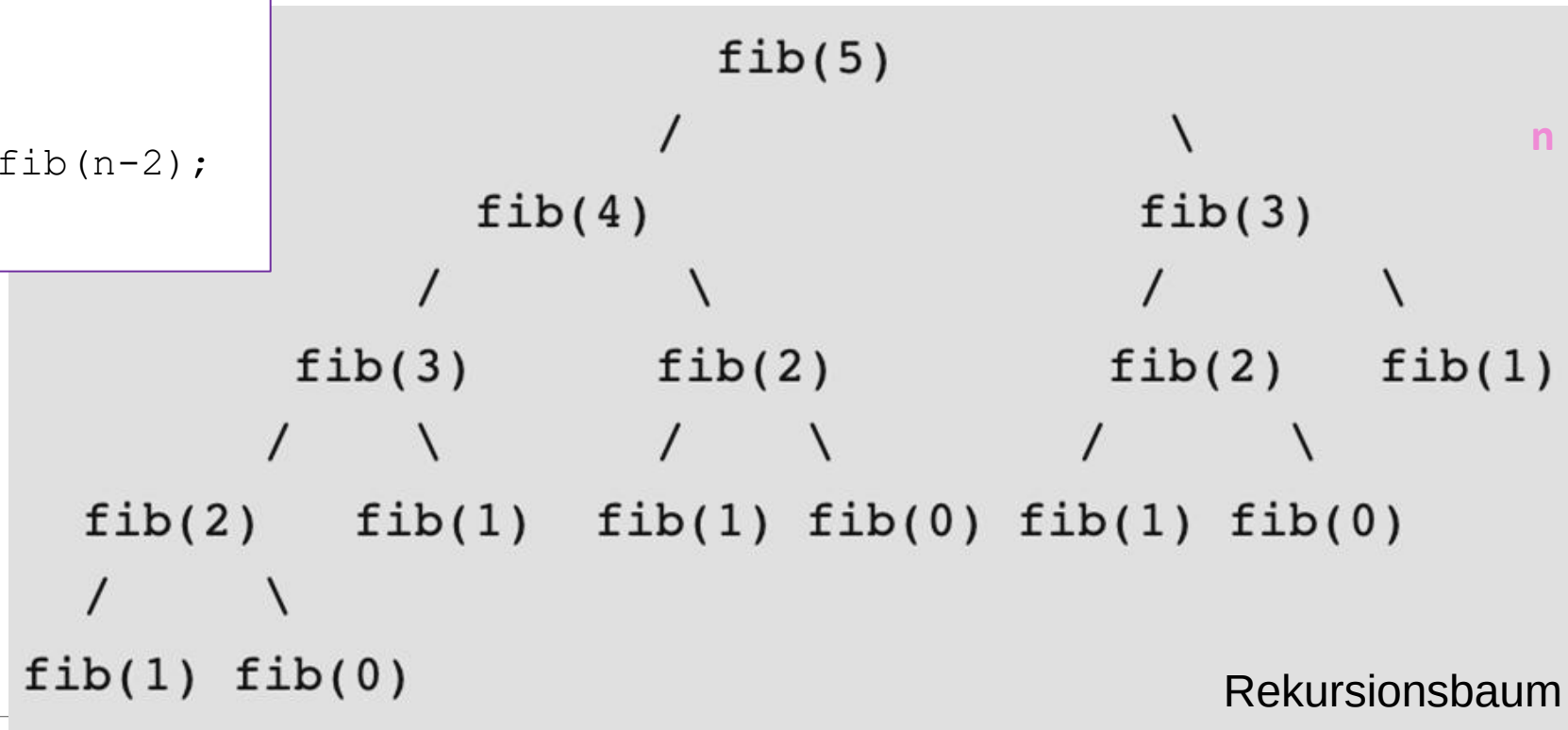
Rekursionsbaum

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Fibonacci-Zahlen: Rekursiver Brute-Force Ansatz

```
public static int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n-1) + fib(n-2);
}
```

Was ist das Problem bei dieser Implementierung?



Rekursionsbaum

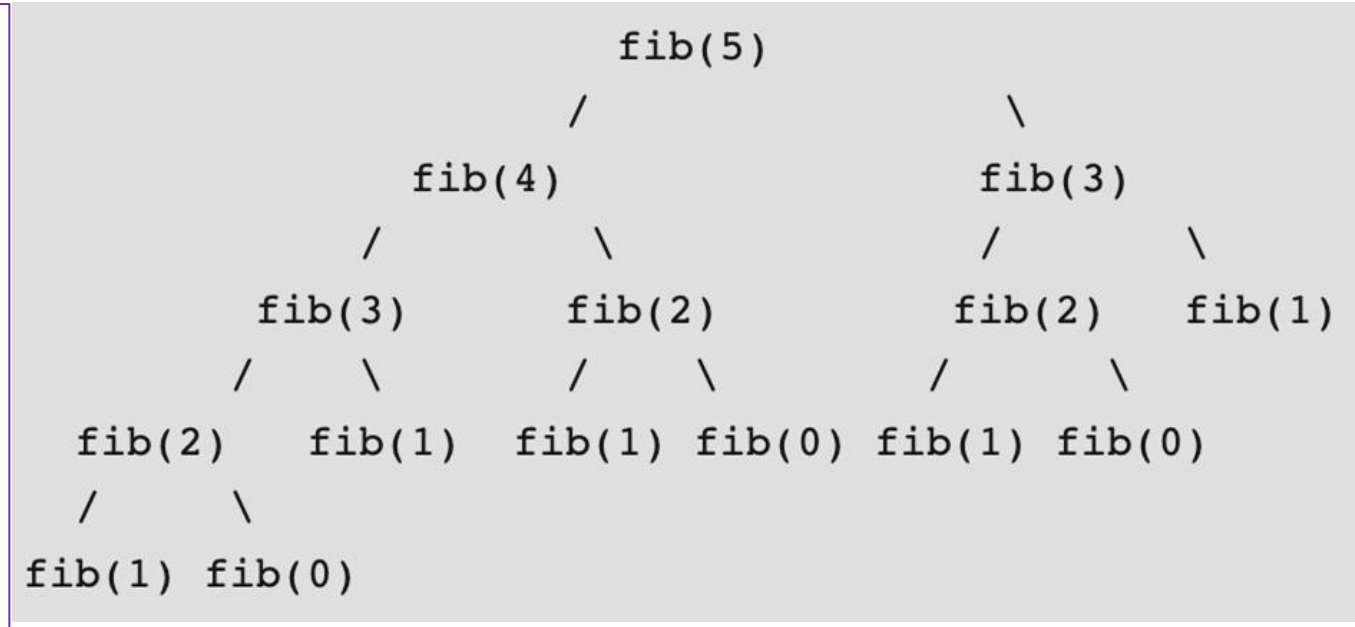
Laufzeit: $O(2^n)$, da in k-ter Ebene des Baumes 2^{k-1} Aufrufe gemacht werden
Speicherplatzbedarf: O(n)

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Fibonacci-Zahlen: Dyn. Programmieren Ansatz mit Memoization

```
public int fib(int n, int[] memo) {
  if (memo[n] != null) {
    return memo[n];
  } else if (n <= 1) {
    memo[n] = n
  } else {
    memo[n] = fib(n-1,memo) + fib(n-2,memo);
  }
  return memo[n];
}
```

An Position memo[n] wird fib(n) gespeichert



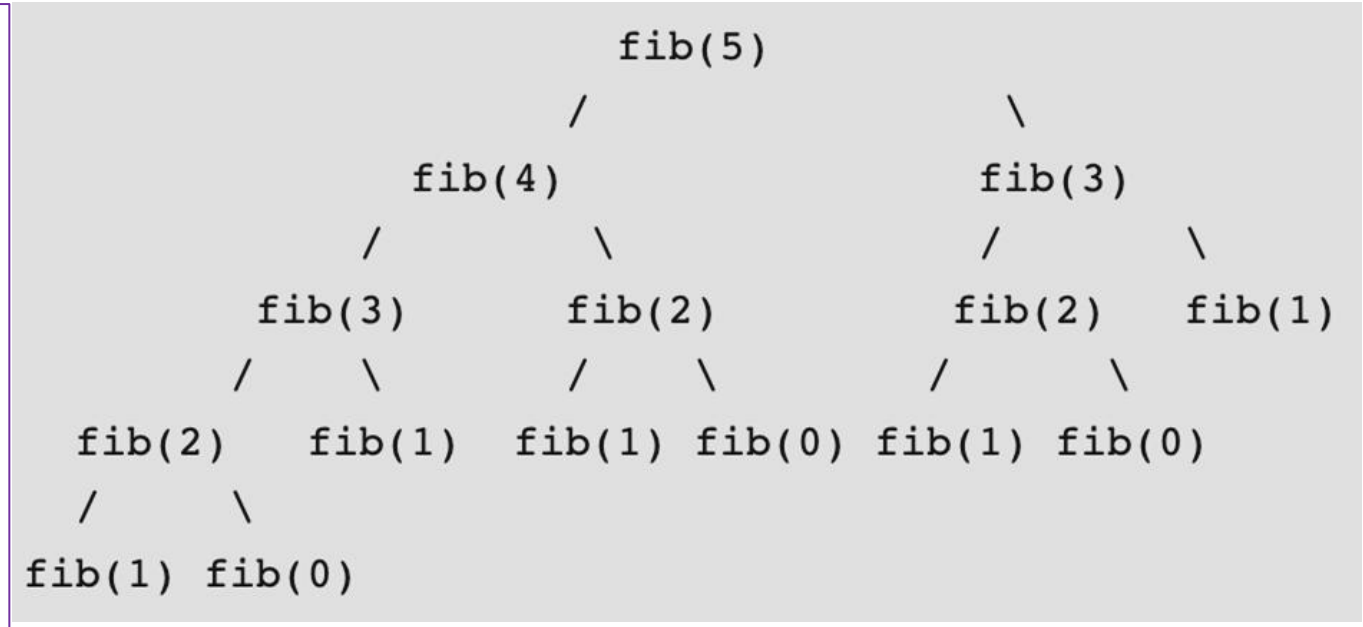
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Fibonacci-Zahlen: Dyn. Programmieren Ansatz mit Memoization

memo = [null , null , null , null , null]

```
public int fib(int n, int[] memo) {
  if (memo[n] != null) {
    return memo[n];
  } else if (n <= 1) {
    memo[n] = n
  } else {
    memo[n] = fib(n-1,memo) + fib(n-2,memo);
  }
  return memo[n];
}
```

An Position memo[n] wird fib(n) gespeichert



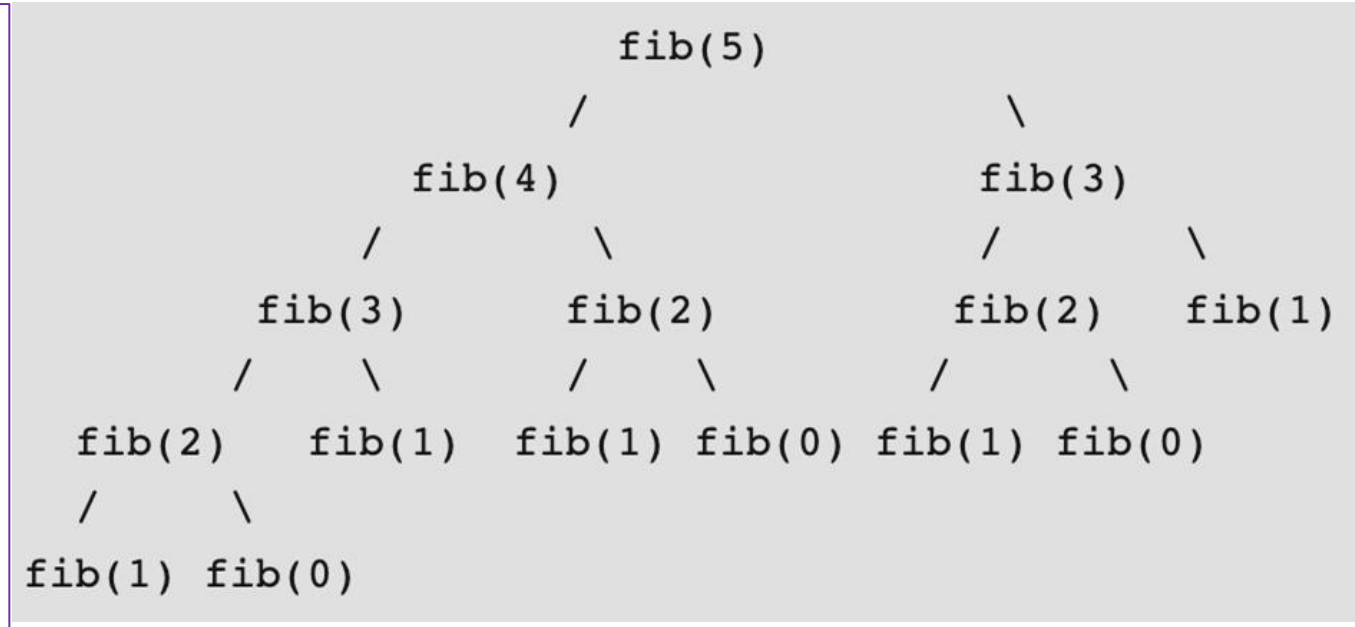
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Fibonacci-Zahlen: Dyn. Programmieren Ansatz mit Memoization

memo = [null , 1 , null , null , null]

```
public int fib(int n, int[] memo) {
  if (memo[n] != null) {
    return memo[n];
  } else if (n <= 1) {
    memo[n] = n
  } else {
    memo[n] = fib(n-1,memo) + fib(n-2,memo);
  }
  return memo[n];
}
```

An Position memo[n] wird fib(n) gespeichert

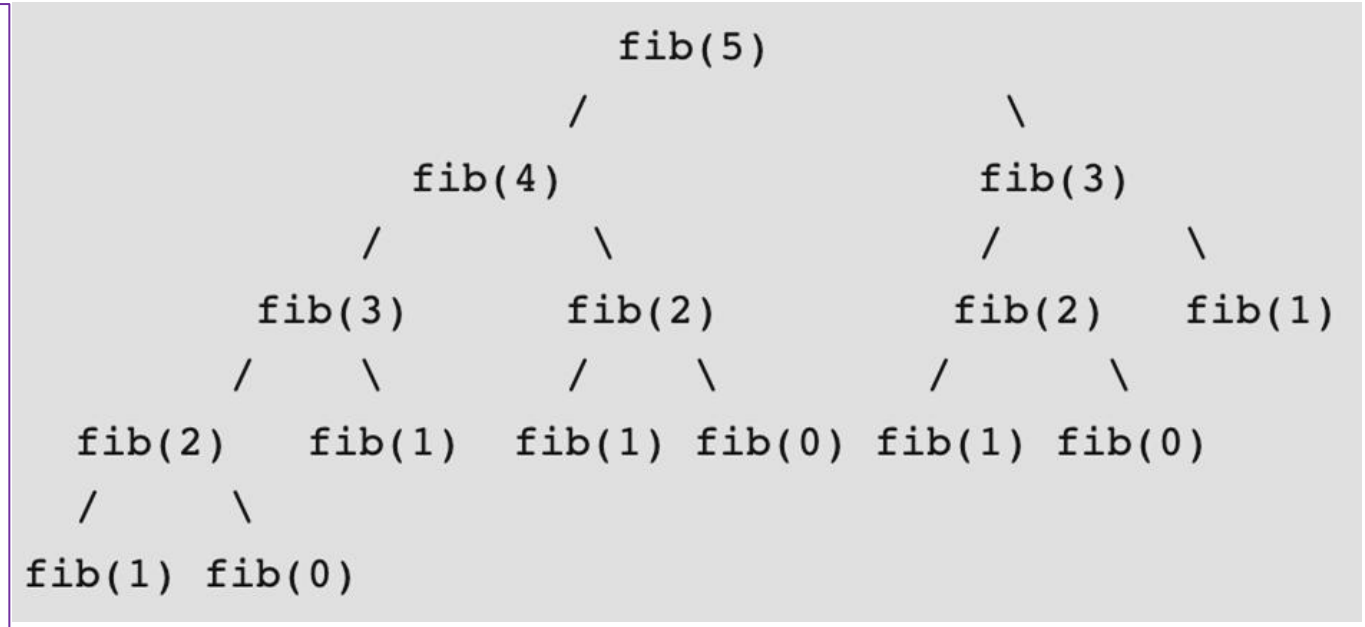


Fibonacci-Zahlen: Dyn. Programmieren Ansatz mit Memoization

memo = [0 , 1 , null , null , null]

```
public int fib(int n, int[] memo) {
  if (memo[n] != null) {
    return memo[n];
  } else if (n <= 1) {
    memo[n] = n
  } else {
    memo[n] = fib(n-1,memo) + fib(n-2,memo);
  }
  return memo[n];
}
```

An Position memo[n] wird fib(n) gespeichert

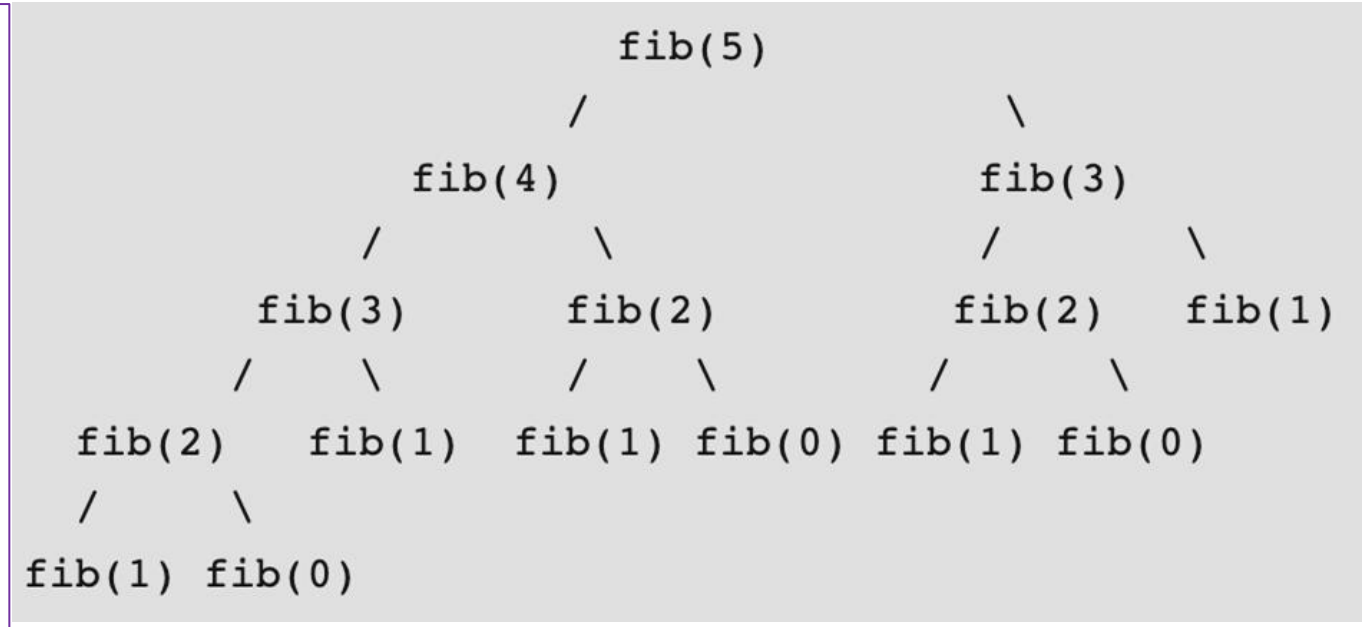


Fibonacci-Zahlen: Dyn. Programmieren Ansatz mit Memoization

memo = [0 , 1 , 1 , null , null]

```
public int fib(int n, int[] memo) {
  if (memo[n] != null) {
    return memo[n];
  } else if (n <= 1) {
    memo[n] = n
  } else {
    memo[n] = fib(n-1,memo) + fib(n-2,memo);
  }
  return memo[n];
}
```

An Position memo[n] wird fib(n) gespeichert

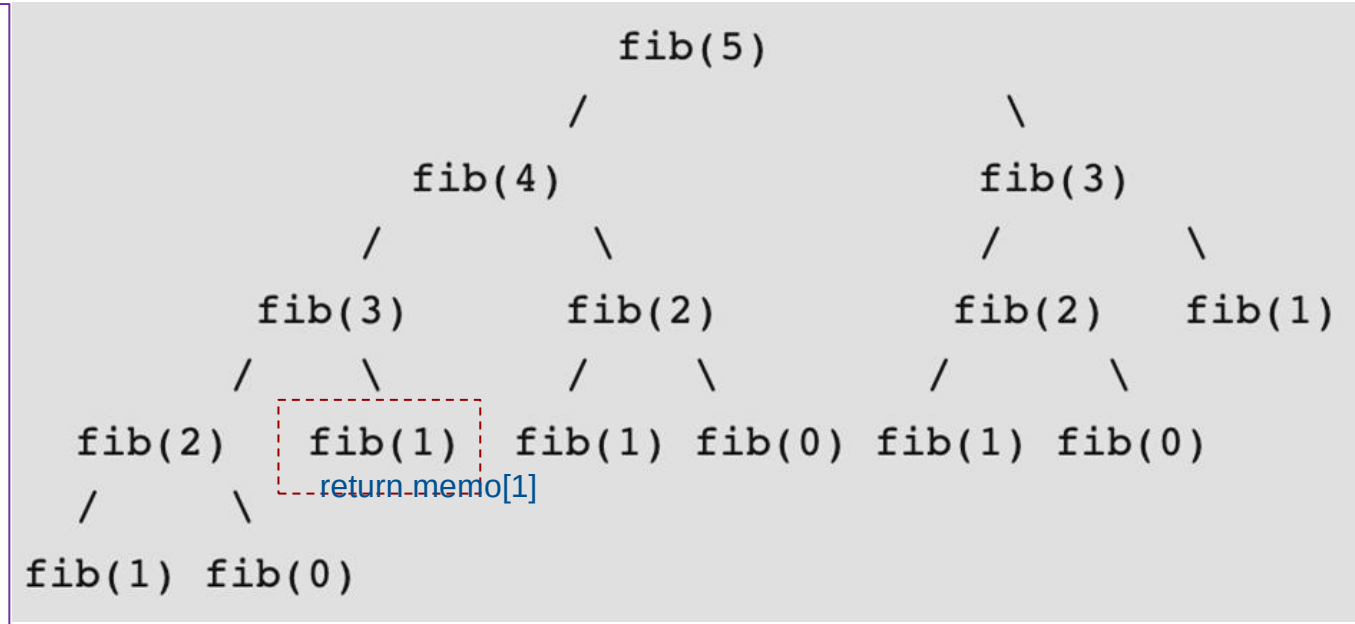


Fibonacci-Zahlen: Dyn. Programmieren Ansatz mit Memoization

memo = [0 , 1 , 1 , null , null]

```
public int fib(int n, int[] memo) {
  if (memo[n] != null) {
    return memo[n];
  } else if (n <= 1) {
    memo[n] = n
  } else {
    memo[n] = fib(n-1,memo) + fib(n-2,memo);
  }
  return memo[n];
}
```

An Position memo[n] wird fib(n) gespeichert



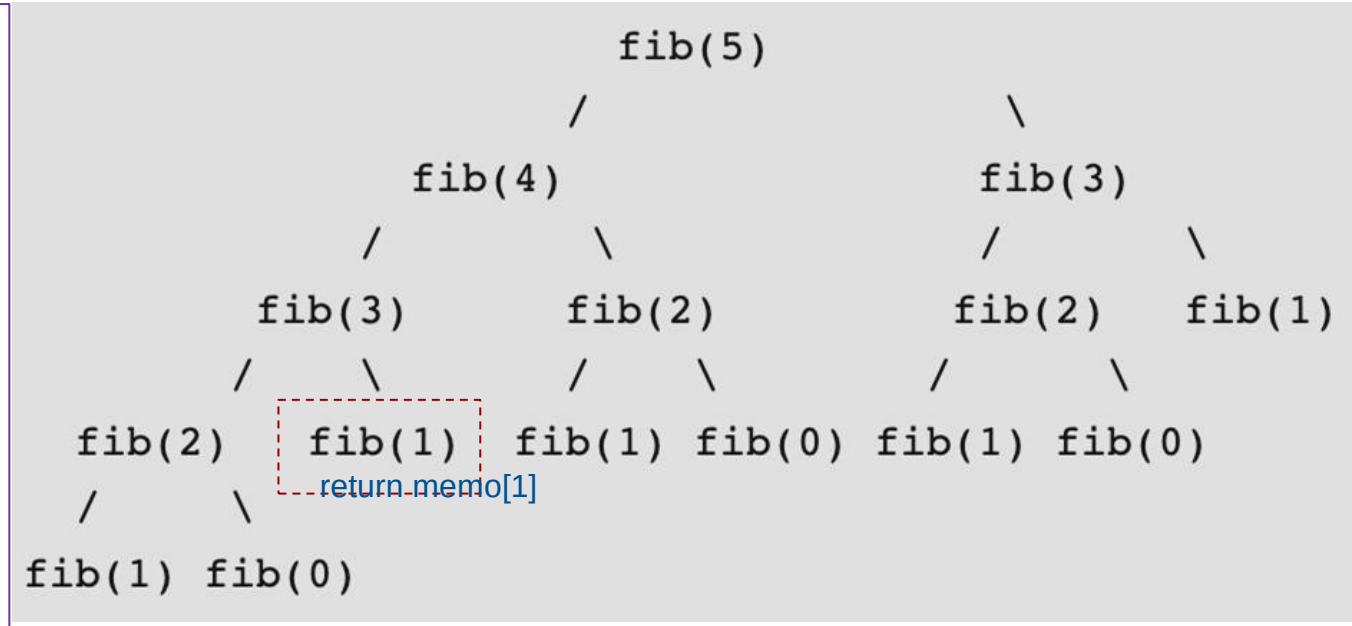
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Fibonacci-Zahlen: Dyn. Programmieren Ansatz mit Memoization

memo = [0 , 1 , 1 , 2 , null]

```
public int fib(int n, int[] memo) {
  if (memo[n] != null) {
    return memo[n];
  } else if (n <= 1) {
    memo[n] = n
  } else {
    memo[n] = fib(n-1,memo) + fib(n-2,memo);
  }
  return memo[n];
}
```

An Position memo[n] wird fib(n) gespeichert

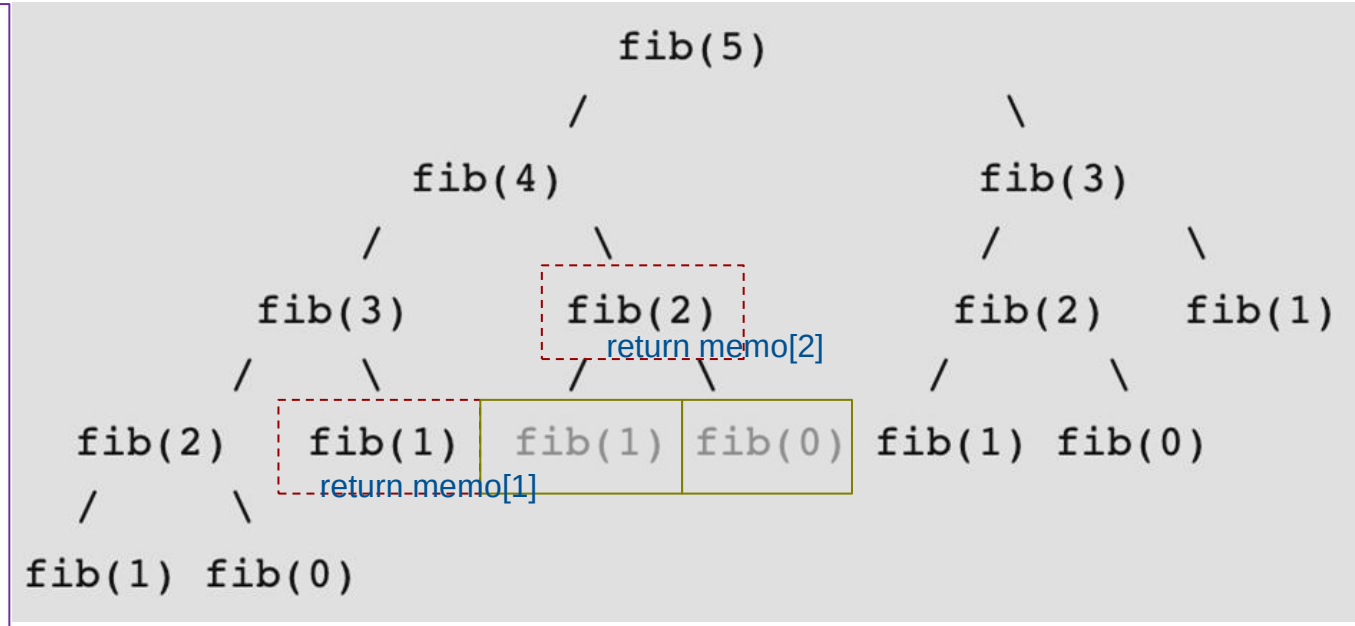


Fibonacci-Zahlen: Dyn. Programmieren Ansatz mit Memoization

memo = [0 , 1 , 1 , 2 , null]

```
public int fib(int n, int[] memo) {
  if (memo[n] != null) {
    return memo[n];
  } else if (n <= 1) {
    memo[n] = n
  } else {
    memo[n] = fib(n-1,memo) + fib(n-2,memo);
  }
  return memo[n];
}
```

An Position memo[n] wird fib(n) gespeichert



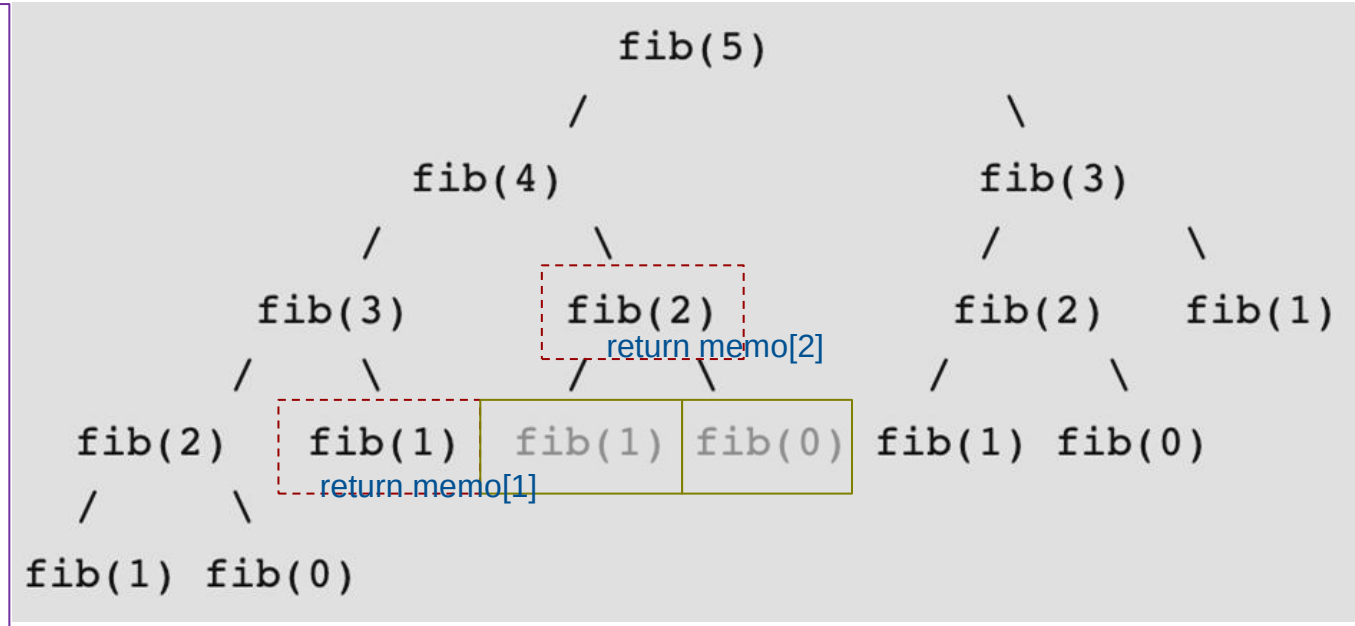
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Fibonacci-Zahlen: Dyn. Programmieren Ansatz mit Memoization

memo = [0 , 1 , 1 , 2 , 3]

```
public int fib(int n, int[] memo) {
  if (memo[n] != null) {
    return memo[n];
  } else if (n <= 1) {
    memo[n] = n
  } else {
    memo[n] = fib(n-1,memo) + fib(n-2,memo);
  }
  return memo[n];
}
```

An Position memo[n] wird fib(n) gespeichert

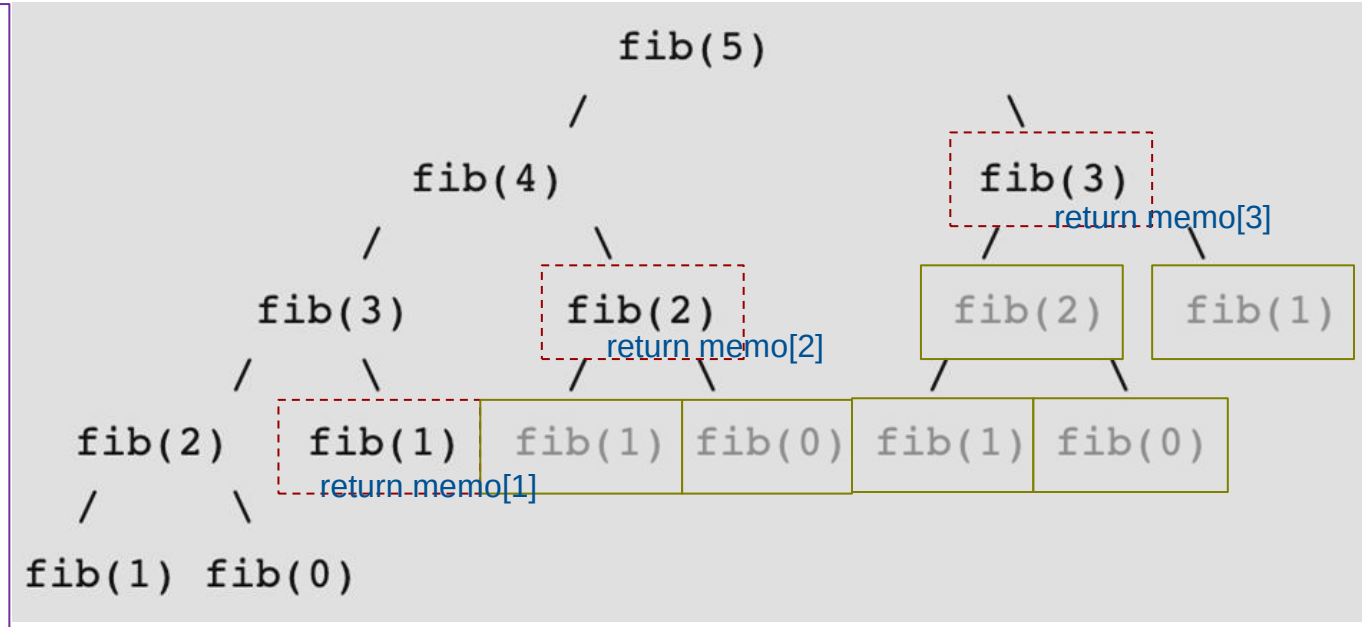


Fibonacci-Zahlen: Dyn. Programmieren Ansatz mit Memoization

memo = [0 , 1 , 1 , 2 , 3]

```
public int fib(int n, int[] memo) {
  if (memo[n] != null) {
    return memo[n];
  } else if (n <= 1) {
    memo[n] = n
  } else {
    memo[n] = fib(n-1,memo) + fib(n-2,memo);
  }
  return memo[n];
}
```

An Position memo[n] wird fib(n) gespeichert

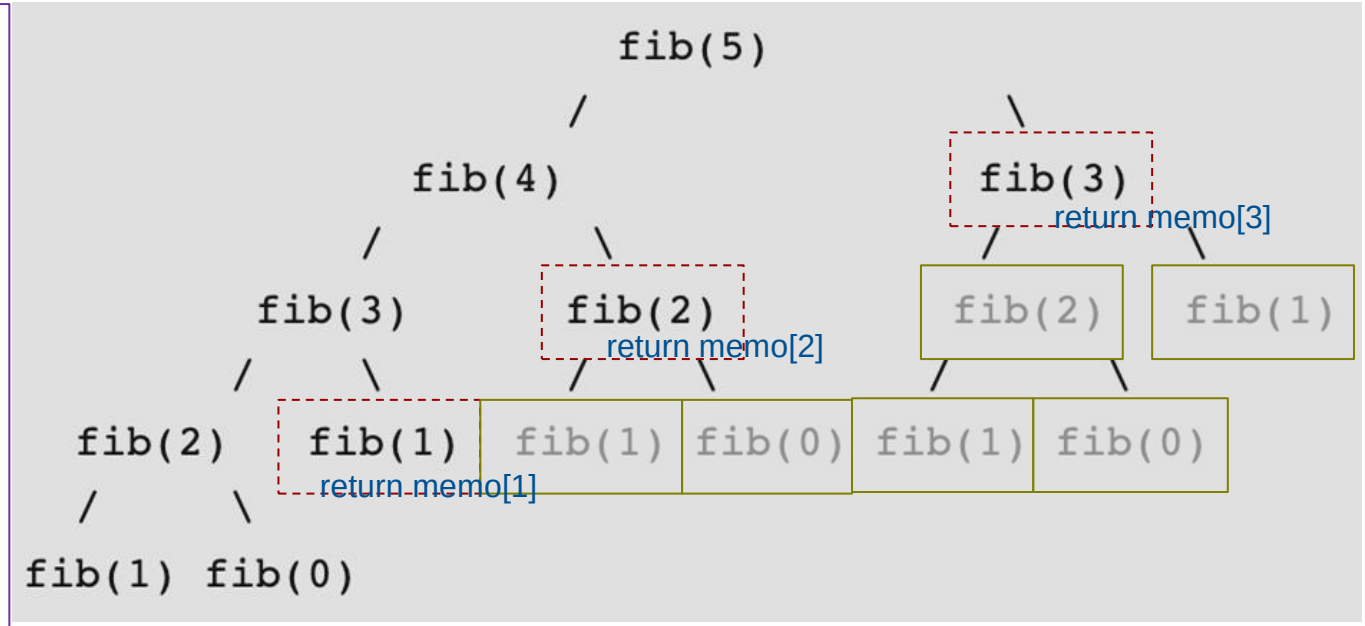


Fibonacci-Zahlen: Dyn. Programmieren Ansatz mit Memoization

memo = [0 , 1 , 1 , 2 , 3]

```
public int fib(int n, int[] memo) {
  if (memo[n] != null) {
    return memo[n];
  } else if (n <= 1) {
    memo[n] = n
  } else {
    memo[n] = fib(n-1,memo) + fib(n-2,memo);
  }
  return memo[n];
}
```

An Position memo[n] wird fib(n) gespeichert



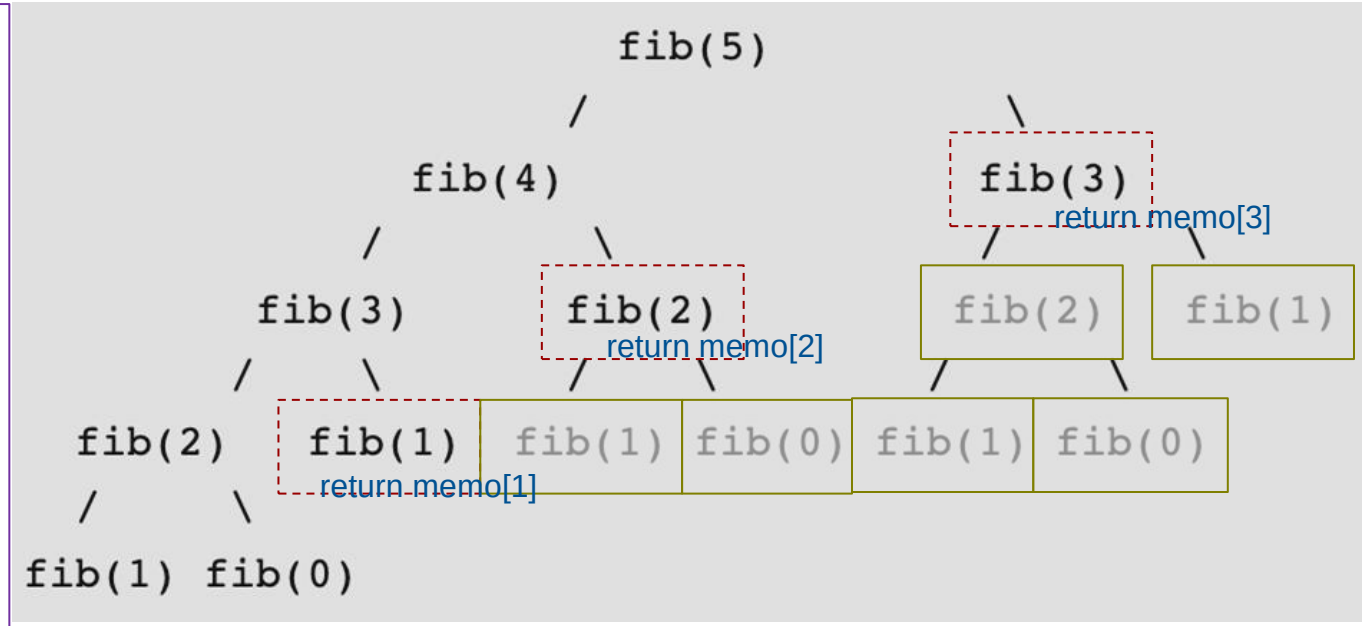
n rekursive Aufrufe, da für jedes i=0,…,n, fib() nur einmal (fast) rekursiv aufgerufen wird. Deshalb Laufzeit: O(n) anstelle von $O(2^n)$.

Fibonacci-Zahlen: Dyn. Programmieren Ansatz mit Memoization

memo = [0 , 1 , 1 , 2 , 3]

```
public int fib(int n, int[] memo) {
  if (memo[n] != null) {
    return memo[n];
  } else if (n <= 1) {
    memo[n] = n
  } else {
    memo[n] = fib(n-1,memo) + fib(n-2,memo);
  }
  return memo[n];
}
```

An Position memo[n] wird fib(n) gespeichert



n rekursive Aufrufe, da für jedes i=0,…,n, fib() nur einmal (fast) rekursiv aufgerufen wird. Deshalb Laufzeit: O(n) anstelle von $O(2^n)$.

Speicherplatzbedarf: O(n); man tauscht bessere Laufzeit gegen höheren Speicherplatzbedarf

Fibonacci-Zahlen: Dyn. Programmieren

Bottom-Up-Ansatz

Detailed walkthrough of this solution

```
public int fib(int n) {
    int f[] = new int[n+1];
    f[0] = 0;
    f[1] = 1;
    for (int i = 2; i <= n; i++) {
        f[i] = f[i-1] + f[i-2];
    }
    return f[n];
}
```

-
-

f = [null , null , null , null , null]

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Fibonacci-Zahlen: Dyn. Programmieren

Bottom-Up-Ansatz

Detailed walkthrough of this solution

```
public int fib(int n) {
    int f[] = new int[n+1];
    f[0] = 0;
    f[1] = 1;
    for (int i = 2; i <= n; i++) {
        f[i] = f[i-1] + f[i-2];
    }
    return f[n];
}
```

-
-

f = [0 , 1 , null , null , null]

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Fibonacci-Zahlen: Dyn. Programmieren

Bottom-Up-Ansatz

Detailed walkthrough of this solution

```
public int fib(int n) {
    int f[] = new int[n+1];
    f[0] = 0;
    f[1] = 1;
    for (int i = 2; i <= n; i++) {
        f[i] = f[i-1] + f[i-2];
    }
    return f[n];
}
```

i=2:

- f[2] = f[1] + f[0]
-

f = [0 , 1 , null , null , null]

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Fibonacci-Zahlen: Dyn. Programmieren

Bottom-Up-Ansatz

Detailed walkthrough of this solution

```
public int fib(int n) {
    int f[] = new int[n+1];
    f[0] = 0;
    f[1] = 1;
    for (int i = 2; i <= n; i++) {
        f[i] = f[i-1] + f[i-2];
    }
    return f[n];
}
```

i=2:

- f[2] = f[1] + f[0]
-

f = [0 , 1 , 1 , null , null]

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Fibonacci-Zahlen: Dyn. Programmieren

Bottom-Up-Ansatz

Detailed walkthrough of this solution

```
public int fib(int n) {
    int f[] = new int[n+1];
    f[0] = 0;
    f[1] = 1;
    for (int i = 2; i <= n; i++) {
        f[i] = f[i-1] + f[i-2];
    }
    return f[n];
}
```

i=2:

- f[2] = f[1] + f[0]

i=3:

- f[3] = f[2] + f[1]

f = [0 , 1 , 1 , 2 , null]

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Fibonacci-Zahlen: Dyn. Programmieren

Bottom-Up-Ansatz

Detailed walkthrough of this solution

```
public int fib(int n) {
    int f[] = new int[n+1];
    f[0] = 0;
    f[1] = 1;
    for (int i = 2; i <= n; i++) {
        f[i] = f[i-1] + f[i-2];
    }
    return f[n];
}
```

Laufzeit: O(n)

Speicherplatzbedarf: O(n)

i=2:

- f[2] = f[1] + f[0]

i=3:

- f[3] = f[2] + f[1]

f = [0 , 1 , 1 , 2 , null]

Fibonacci-Zahlen: Dyn. Programmieren

Bottom-Up-Ansatz ohne zusätzliche Datenstruktur (Array)

```
static int fib(int n) {
  int a = 0, b = 1, c;
  if (n == 0)
    return a;
  for (int i = 2; i <= n; i++) {
    c = a + b;
    a = b;
    b = c;
  }
  return b;
}
```



Fibonacci-Zahlen: Dyn. Programmieren

Bottom-Up-Ansatz ohne zusätzliche Datenstruktur (Array)

```
static int fib(int n) {
  int a = 0, b = 1, c;
  if (n == 0)
    return a;
  for (int i = 2; i <= n; i++) {
    c = a + b;
    a = b;
    b = c;
  }
  return b;
}
```

f[i-2]	f[i-1]	f[i]	f[i+1]	f[i+2]	f[i+3]
a	b	c			
	a	b	c		
		a	b	c	
			a	b	c

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Fibonacci-Zahlen: Dyn. Programmieren

Bottom-Up-Ansatz ohne zusätzliche Datenstruktur (Array)

```
static int fib(int n) {
  int a = 0, b = 1, c;
  if (n == 0)
    return a;
  for (int i = 2; i <= n; i++) {
    c = a + b;
    a = b;
    b = c;
  }
  return b;
}
```

f[i-2]	f[i-1]	f[i]	f[i+1]	f[i+2]	f[i+3]
a	b	c			
	a	b	c		
		a	b	c	
			a	b	c

Fibonacci-Zahlen: Dyn. Programmieren

Bottom-Up-Ansatz ohne zusätzliche Datenstruktur (Array)

```
static int fib(int n) {
  int a = 0, b = 1, c;
  if (n == 0)
    return a;
  for (int i = 2; i <= n; i++) {
    c = a + b;
    a = b;
    b = c;
  }
  return b;
}
```

f[i-2]	f[i-1]	f[i]	f[i+1]	f[i+2]	f[i+3]
	a	b	c		
		a	b	c	
			a	b	c

Fibonacci-Zahlen: Dyn. Programmieren

Bottom-Up-Ansatz ohne zusätzliche Datenstruktur (Array)

```
static int fib(int n) {
  int a = 0, b = 1, c;
  if (n == 0)
    return a;
  for (int i = 2; i <= n; i++) {
    c = a + b;
    a = b;
    b = c;
  }
  return b;
}
```

f[i-2]	f[i-1]	f[i]	f[i+1]	f[i+2]	f[i+3]
		a	b	c	
			a	b	c

Fibonacci-Zahlen: Dyn. Programmieren

Bottom-Up-Ansatz ohne zusätzliche Datenstruktur (Array)

```
static int fib(int n) {
  int a = 0, b = 1, c;
  if (n == 0)
    return a;
  for (int i = 2; i <= n; i++) {
    c = a + b;
    a = b;
    b = c;
  }
  return b;
}
```

f[i-2]	f[i-1]	f[i]	f[i+1]	f[i+2]	f[i+3]
			a	b	c

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Fibonacci-Zahlen: Dyn. Programmieren

Bottom-Up-Ansatz ohne zusätzliche Datenstruktur (Array)

```
static int fib(int n) {
  int a = 0, b = 1, c;
  if (n == 0)
    return a;
  for (int i = 2; i <= n; i++) {
    c = a + b;
    a = b;
    b = c;
  }
  return b;
}
```

Test für n = 9

Fib = 0, 1, 1, 2, 3, 5, 8, 13, 21, 34

Sich ergebende Werte der Variablen nach der for Schleife:

a = 21, b = 34, c = 34

Return b = 34

Fibonacci-Zahlen: Dyn. Programmieren

Bottom-Up-Ansatz ohne zusätzliche Datenstruktur (Array)

```
static int fib(int n) {
  int a = 0, b = 1, c;
  if (n == 0)
    return a;
  for (int i = 2; i <= n; i++) {
    c = a + b;
    a = b;
    b = c;
  }
  return b;
}
```

Test für n = 9

Fib = 0, 1, 1, 2, 3, 5, 8, 13, 21, 34

Sich ergebende Werte der Variablen nach der for Schleife:

a = 21, b = 34, c = 34

Return b = 34

Laufzeit: O(n)

Speicherplatzbedarf: O(1)

Fibonacci-Zahlen: Dyn. Programmieren

Bottom-Up-Ansatz ohne zusätzliche Datenstruktur (Array)

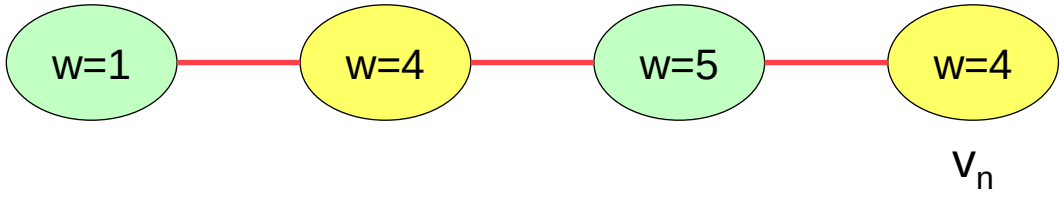
```
int fib(int n) {
  int a = 0, b = 1;
  if (n == 0)
    return 0;
  for (int i = n; i >= 2; i--) {
    if (a<b) {
      a = a + b;
    else {
      b = a + b;
    }
  }
  return a>b ? a:b;
}
```

Übungsblatt 10: Aufgabe 1, 3a,c,d, 7, 8, 10

Stabile Menge maximalen Gewichts in Pfadgraphen

Dynamische Programmierung:

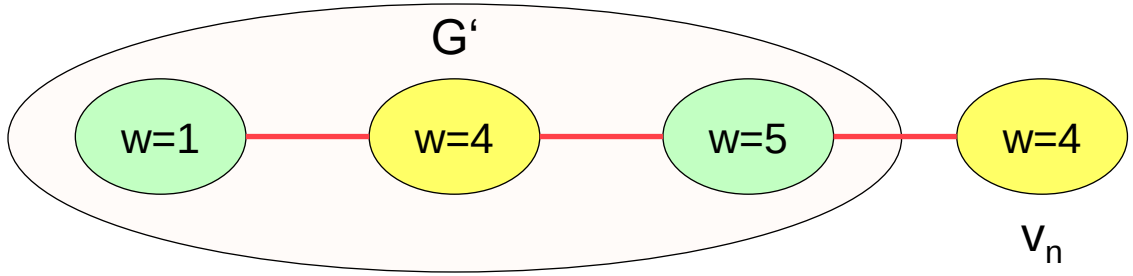
- Fall 1: Knoten v_n (der n-te Knoten) existiert nicht in SMMG von G
  -
  -
  -
  -
  -
  -
  -
  -



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Greedy-Algorithmen, Minimum Spanning Trees und dynamische Programmierung, Stanford University, Tim Roughgarden, Coursera

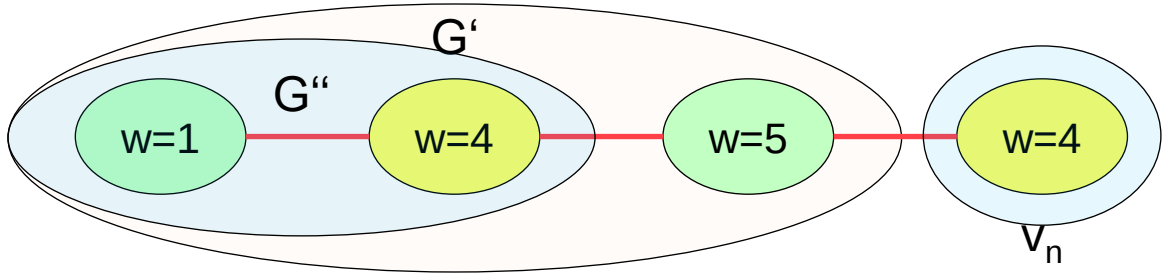
Stabile Menge maximalen Gewichts in Pfadgraphen



Dynamische Programmierung:

- Fall 1: Knoten v_n (der n-te Knoten) existiert nicht in SMMG von G
  - Dann ist SMMG von G identisch zu der SMMG von G‘ := G \ v_n
-
  -
-
-
-
-
-

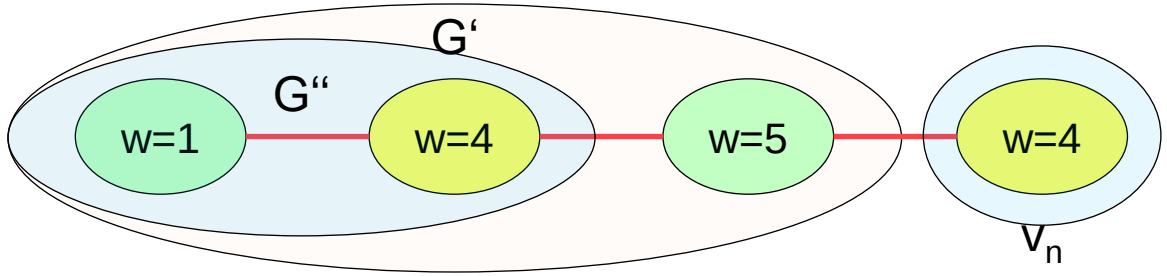
Stabile Menge maximalen Gewichts in Pfadgraphen



Dynamische Programmierung:

- Fall 1: Knoten v_n (der n-te Knoten) existiert nicht in SMMG von G
  - Dann ist SMMG von G identisch zu der SMMG von G‘ := G \ v_n
- Fall 2: Knoten v_n existiert in SMMG von G
  - Dann ist SMMG von G identisch zu der SMMG von G“ := G \ {v_n, v_{n-1}} zuzüglich v_n

Stabile Menge maximalen Gewichts in Pfadgraphen

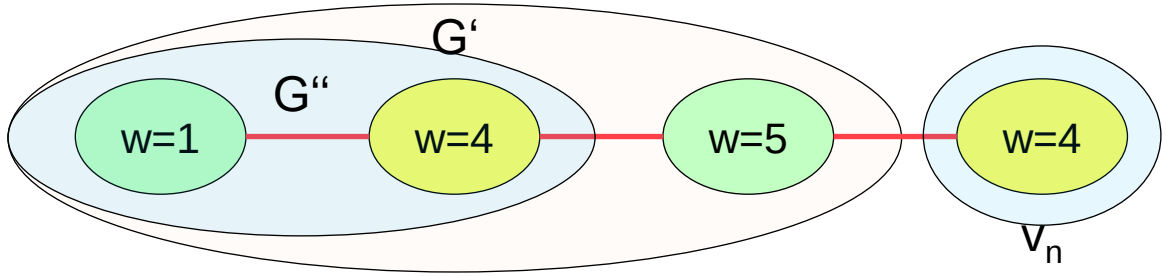


Dynamische Programmierung:

- Fall 1: Knoten v_n (der n-te Knoten) existiert nicht in SMMG von G
  - Dann ist SMMG von G identisch zu der SMMG von G‘ := G \ v_n
- Fall 2: Knoten v_n existiert in SMMG von G
  - Dann ist SMMG von G identisch zu der SMMG von G“ := G \ {v_n, v_{n-1}} zuzüglich v_n

- Probiere beide Fälle aus und liefere die SMMG mit größerem Gesamtgewicht zurück

Stabile Menge maximalen Gewichts in Pfadgraphen



Dynamische Programmierung:

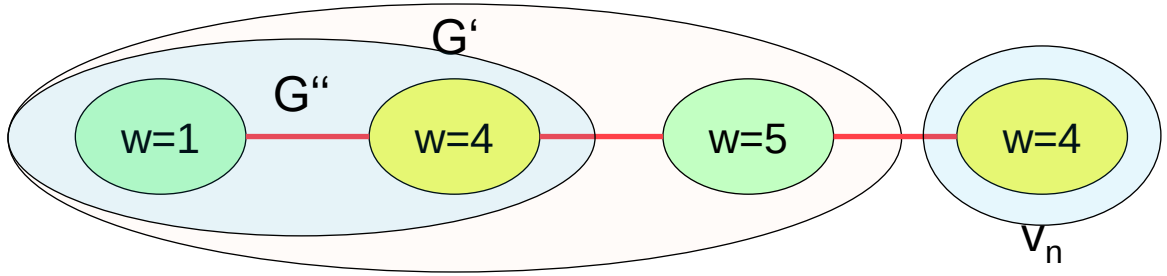
- Fall 1: Knoten v_n (der n-te Knoten) existiert nicht in SMMG von G
  - Dann ist SMMG von G identisch zu der SMMG von G‘ := G \ v_n
- Fall 2: Knoten v_n existiert in SMMG von G
  - Dann ist SMMG von G identisch zu der SMMG von G“ := G \ {v_n, v_{n-1}} zuzüglich v_n

- Probiere beide Fälle aus und liefere die SMMG mit größerem Gesamtgewicht zurück

Brute-Force Algorithmus mit Memoization:

- Fall 1 für G: Berechne rekursiv SMMG von G‘
- Fall 2 für G: Berechne rekursiv SMMG von G“
- Liefere die SMMG mit größerem Gesamtgewicht zurück
-

Stabile Menge maximalen Gewichts in Pfadgraphen



Dynamische Programmierung:

- Fall 1: Knoten v_n (der n-te Knoten) existiert nicht in SMMG von G
  - Dann ist SMMG von G identisch zu der SMMG von G‘ := G \ v_n
- Fall 2: Knoten v_n existiert in SMMG von G
  - Dann ist SMMG von G identisch zu der SMMG von G“ := G \ {v_n, v_{n-1}} zuzüglich v_n

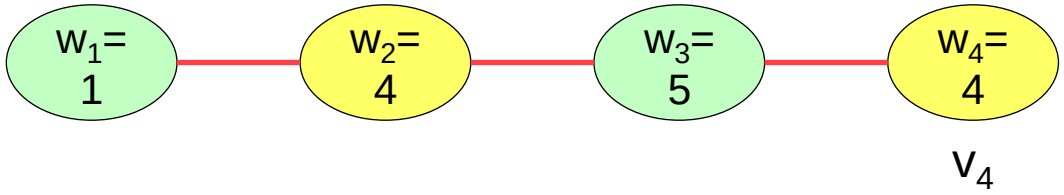
- Probiere beide Fälle aus und liefere die SMMG mit größerem Gesamtgewicht zurück

Brute-Force Algorithmus mit Memoization:

- Fall 1 für G: Berechne rekursiv SMMG von G‘
- Fall 2 für G: Berechne rekursiv SMMG von G“
- Liefere die SMMG mit größerem Gesamtgewicht zurück
- → Mit Memoization hat der Algorithmus Laufzeit von O(n)

Stabile Menge maximalen Gewichts in Pfadgraphen

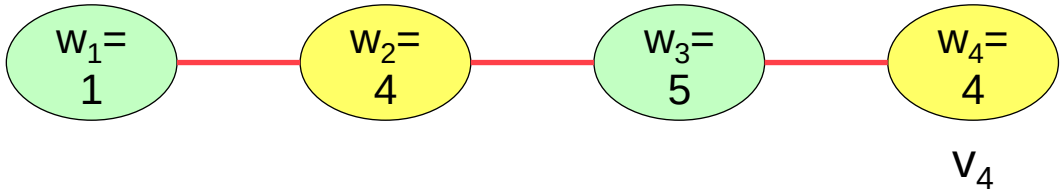
Algorithmus zur Berechnung von SMMG von G:



Stabile Menge maximalen Gewichts in Pfadgraphen

Algorithmus zur Berechnung von SMMG von G:

- Fall 1: v_4 nicht in SMMG von G
  -



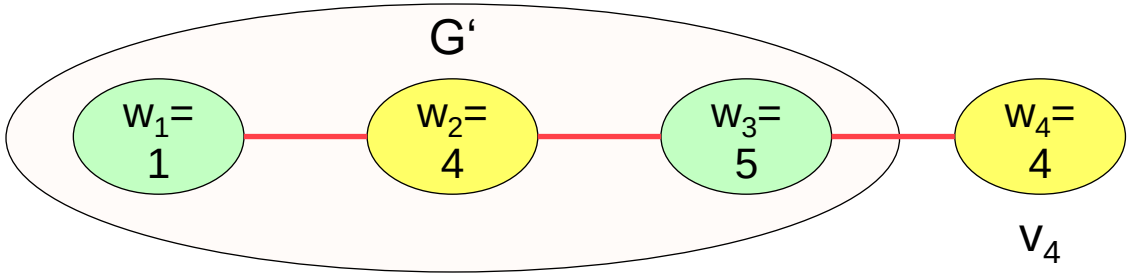
- Fall 2: v_4 ist in SMMG von G

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Greedy-Algorithmen, Minimum Spanning Trees und dynamische Programmierung, Stanford University, Tim Roughgarden, Coursera

Stabile Menge maximalen Gewichts in Pfadgraphen

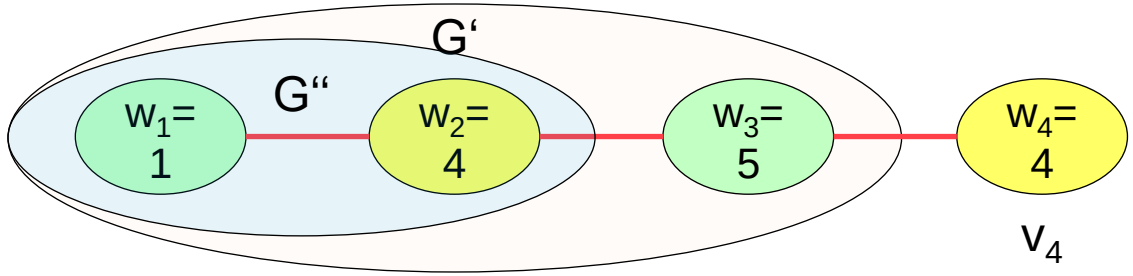
Algorithmus zur Berechnung von SMMG von G:

- Fall 1: v_4 nicht in SMMG von G
  - Berechne rekursiv SMMG von G‘



- Fall 2: v_4 ist in SMMG von G

Stabile Menge maximalen Gewichts in Pfadgraphen

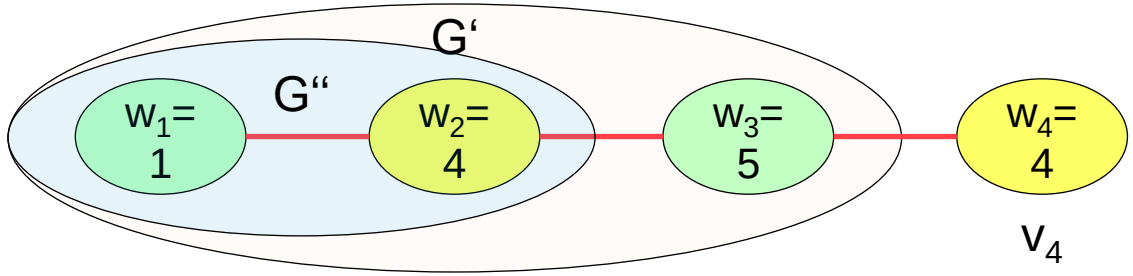


Algorithmus zur Berechnung von SMMG von G:

- Fall 1: v_4 nicht in SMMG von G
  - Berechne rekursiv SMMG von G‘
    - Fall 1: v_3 nicht in SMMG von G‘: Berechne rekursiv SMMG von G“

- Fall 2: v_4 ist in SMMG von G

Stabile Menge maximalen Gewichts in Pfadgraphen

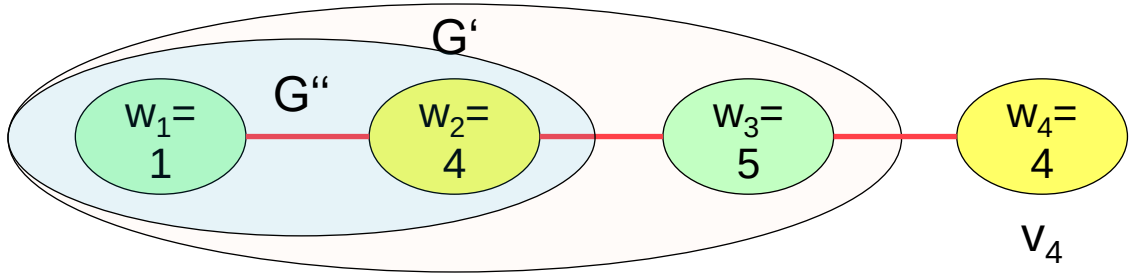


Algorithmus zur Berechnung von SMMG von G:

- Fall 1: v_4 nicht in SMMG von G
  - Berechne rekursiv SMMG von G‘
    - Fall 1: v_3 nicht in SMMG von G‘: Berechne rekursiv SMMG von G“
      - Fall 1: v_2 nicht in SMMG von G“: Gesamtgewicht von SMMG von G“: $w_1 = 1$

- Fall 2: v_4 ist in SMMG von G

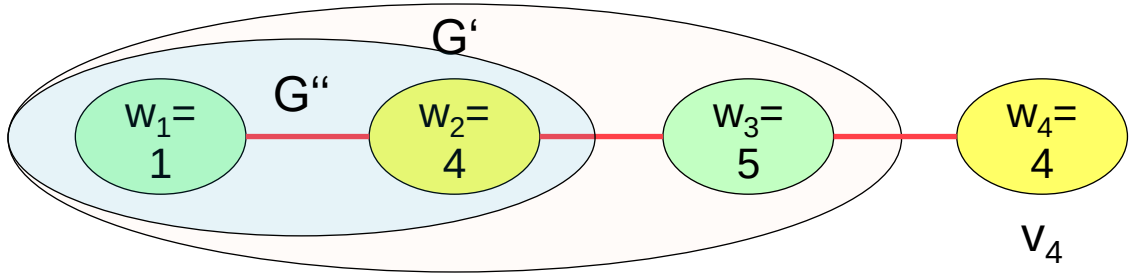
Stabile Menge maximalen Gewichts in Pfadgraphen



Algorithmus zur Berechnung von SMMG von G:

- Fall 1: v_4 nicht in SMMG von G
  - Berechne rekursiv SMMG von G‘
    - Fall 1: v_3 nicht in SMMG von G‘: Berechne rekursiv SMMG von G“
      - Fall 1: v_2 nicht in SMMG von G“: Gesamtgewicht von SMMG von G“: $w_1 = 1$
      - Fall 2: v_2 ist in SMMG von G“: Gesamtgewicht von SMMG von G“: $w_2 = 4$
- Fall 2: v_4 ist in SMMG von G

Stabile Menge maximalen Gewichts in Pfadgraphen

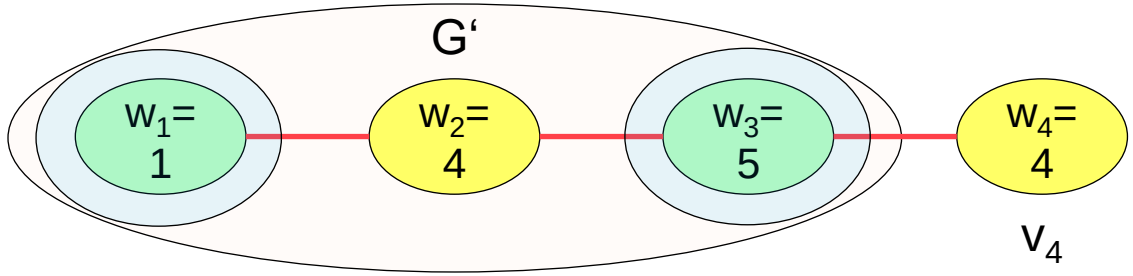


Algorithmus zur Berechnung von SMMG von G:

- Fall 1: v_4 nicht in SMMG von G
  - Berechne rekursiv SMMG von G‘
    - Fall 1: v_3 nicht in SMMG von G‘: Berechne rekursiv SMMG von G“
      - Fall 1: v_2 nicht in SMMG von G“: Gesamtgewicht von SMMG von G“: $w_1 = 1$
      - Fall 2: v_2 ist in SMMG von G“: Gesamtgewicht von SMMG von G“: $w_2 = 4$
      - → Gesamtgewicht von SMMG von G“: Max(1, 4) = **4**

- Fall 2: v_4 ist in SMMG von G

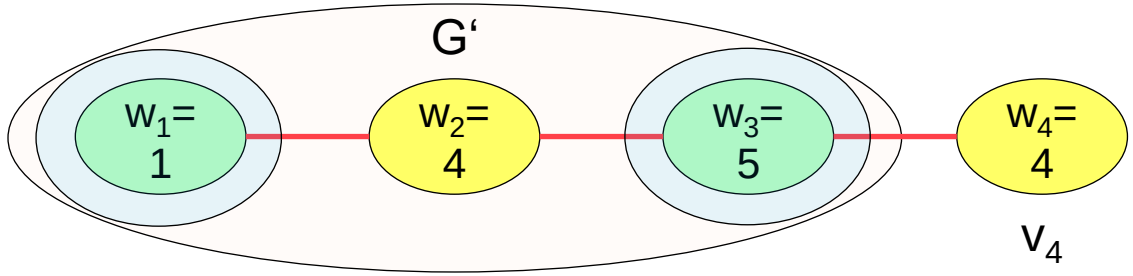
Stabile Menge maximalen Gewichts in Pfadgraphen



Algorithmus zur Berechnung von SMMG von G:

- Fall 1: v_4 nicht in SMMG von G
  - Berechne rekursiv SMMG von G‘
    - Fall 1: v_3 nicht in SMMG von G‘: Berechne rekursiv SMMG von G“
      - Fall 1: v_2 nicht in SMMG von G“: Gesamtgewicht von SMMG von G“: $w_1 = 1$
      - Fall 2: v_2 ist in SMMG von G“: Gesamtgewicht von SMMG von G“: $w_2 = 4$
      - → Gesamtgewicht von SMMG von G“: Max(1, 4) = **4**
    - Fall 2: v_3 in SMMG von G‘ → Gesamtgewicht von SMMG von G‘: $w_1 + w_3 = \mathbf{6}$
    -
- Fall 2: v_4 ist in SMMG von G

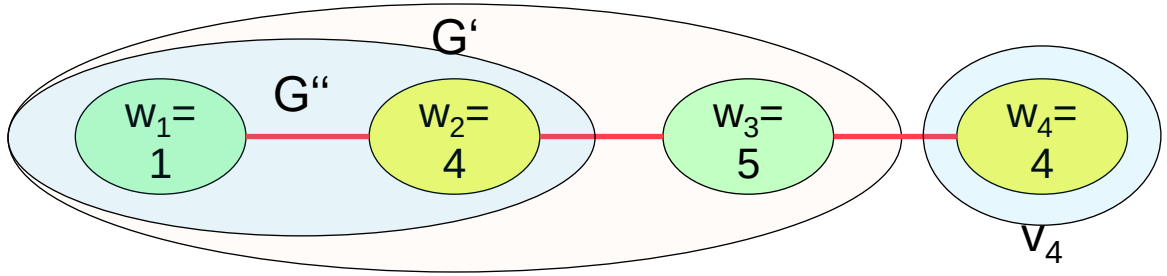
Stabile Menge maximalen Gewichts in Pfadgraphen



Algorithmus zur Berechnung von SMMG von G:

- Fall 1: v_4 nicht in SMMG von G
  - Berechne rekursiv SMMG von G‘
    - Fall 1: v_3 nicht in SMMG von G‘: Berechne rekursiv SMMG von G“
      - Fall 1: v_2 nicht in SMMG von G“: Gesamtgewicht von SMMG von G“: $w_1 = 1$
      - Fall 2: v_2 ist in SMMG von G“: Gesamtgewicht von SMMG von G“: $w_2 = 4$
      - → Gesamtgewicht von SMMG von G“: Max(1, 4) = **4**
    - Fall 2: v_3 in SMMG von G‘ → Gesamtgewicht von SMMG von G‘: $w_1 + w_3 = \mathbf{6}$
    - → Gesamtgewicht von SMMG von G‘: Max(**4**, **6**) = **6**
- Fall 2: v_4 ist in SMMG von G

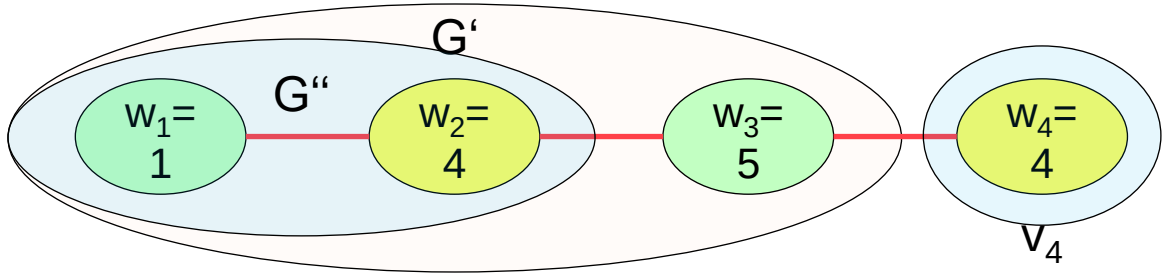
Stabile Menge maximalen Gewichts in Pfadgraphen



Algorithmus zur Berechnung von SMMG von G:

- **Fall 1**: v_4 nicht in SMMG von G
  - Berechne rekursiv SMMG von G‘
    - Fall 1: v_3 nicht in SMMG von G‘: Berechne rekursiv SMMG von G“
      - Fall 1: v_2 nicht in SMMG von G“: Gesamtgewicht von SMMG von G“: $w_1 = 1$
      - Fall 2: v_2 ist in SMMG von G“: Gesamtgewicht von SMMG von G“: $w_2 = 4$
      - → Gesamtgewicht von SMMG von G“: Max(1, 4) = **4**
    - Fall 2: v_3 in SMMG von G‘ → Gesamtgewicht von SMMG von G‘: $w_1 + w_3 = 6$
    - → Gesamtgewicht von SMMG von G‘: Max(**4**, **6**) = **6**
- **Fall 2**: v_4 ist in SMMG von G
  - Berechne rekursiv SMMG von G“: **4** (s.o.)
  -
-
  -

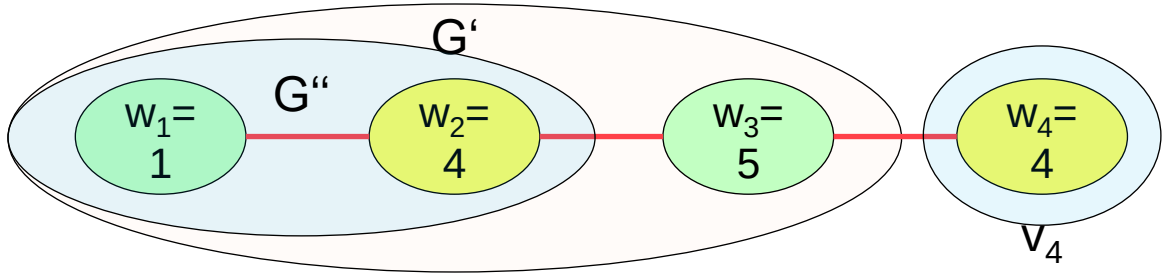
Stabile Menge maximalen Gewichts in Pfadgraphen



Algorithmus zur Berechnung von SMMG von G:

- **Fall 1**: v_4 nicht in SMMG von G
  - Berechne rekursiv SMMG von G‘
    - Fall 1: v_3 nicht in SMMG von G‘: Berechne rekursiv SMMG von G“
      - Fall 1: v_2 nicht in SMMG von G“: Gesamtgewicht von SMMG von G“: $w_1 = 1$
      - Fall 2: v_2 ist in SMMG von G“: Gesamtgewicht von SMMG von G“: $w_2 = 4$
      - → Gesamtgewicht von SMMG von G“: Max(1, 4) = **4**
    - Fall 2: v_3 in SMMG von G‘ → Gesamtgewicht von SMMG von G‘: $w_1 + w_3 = 6$
    - → Gesamtgewicht von SMMG von G‘: Max(**4**, **6**) = **6**
- **Fall 2**: v_4 ist in SMMG von G
  - Berechne rekursiv SMMG von G“: **4** (s.o.)
  - Gesamtgewicht von SMMG von G: **4** + w_4 = **4** + 4 = **8**
-
  -

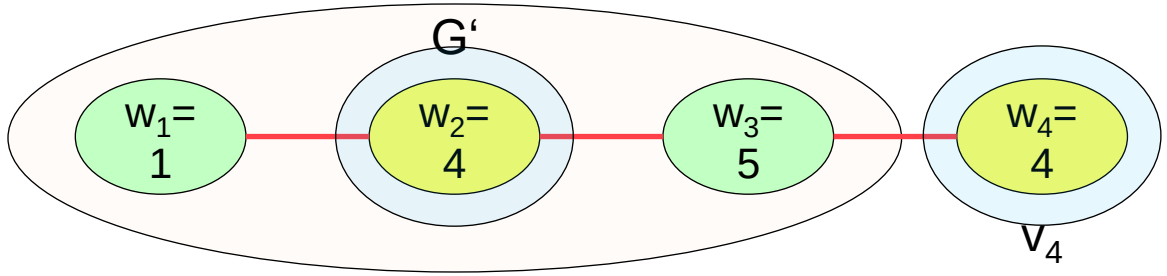
Stabile Menge maximalen Gewichts in Pfadgraphen



Algorithmus zur Berechnung von SMMG von G:

- **Fall 1**: v_4 nicht in SMMG von G
  - Berechne rekursiv SMMG von G‘
    - Fall 1: v_3 nicht in SMMG von G‘: Berechne rekursiv SMMG von G“
      - Fall 1: v_2 nicht in SMMG von G“: Gesamtgewicht von SMMG von G“: $w_1 = 1$
      - Fall 2: v_2 ist in SMMG von G“: Gesamtgewicht von SMMG von G“: $w_2 = 4$
      - → Gesamtgewicht von SMMG von G“: Max(1, 4) = **4**
    - Fall 2: v_3 in SMMG von G‘ → Gesamtgewicht von SMMG von G‘: $w_1 + w_3 = 6$
    - → Gesamtgewicht von SMMG von G‘: Max(**4**, **6**) = **6**
- **Fall 2**: v_4 ist in SMMG von G
  - Berechne rekursiv SMMG von G“: **4** (s.o.)
  - Gesamtgewicht von SMMG von G: **4** + w_4 = **4** + 4 = **8**
- Liefere die SMMG mit größerem Gesamtgewicht zurück
  - Gesamtgewicht von SMMG von G: Max(**Fall 1**, **Fall 2**) = Max(**6**, **8**) = 8

Stabile Menge maximalen Gewichts in Pfadgraphen



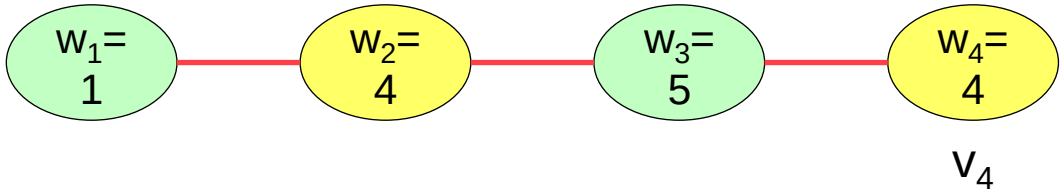
Algorithmus zur Berechnung von SMMG von G:

- **Fall 1**: v_4 nicht in SMMG von G
  - Berechne rekursiv SMMG von G‘
    - Fall 1: v_3 nicht in SMMG von G‘: Berechne rekursiv SMMG von G“
      - Fall 1: v_2 nicht in SMMG von G“: Gesamtgewicht von SMMG von G“: $w_1 = 1$
      - Fall 2: v_2 ist in SMMG von G“: Gesamtgewicht von SMMG von G“: $w_2 = 4$
      - → Gesamtgewicht von SMMG von G“: Max(1, 4) = **4**
    - Fall 2: v_3 in SMMG von G‘ → Gesamtgewicht von SMMG von G‘: $w_1 + w_3 = 6$
    - → Gesamtgewicht von SMMG von G‘: Max(**4**, **6**) = **6**
- **Fall 2**: v_4 ist in SMMG von G
  - Berechne rekursiv SMMG von G“: **4** (s.o.)
  - Gesamtgewicht von SMMG von G: **4** + w_4 = **4** + 4 = **8**
- Liefere die SMMG mit größerem Gesamtgewicht zurück
  - Gesamtgewicht von SMMG von G: Max(**Fall 1**, **Fall 2**) = Max(**6**, **8**) = 8

Stabile Menge maximalen Gewichts in Pfadgraphen

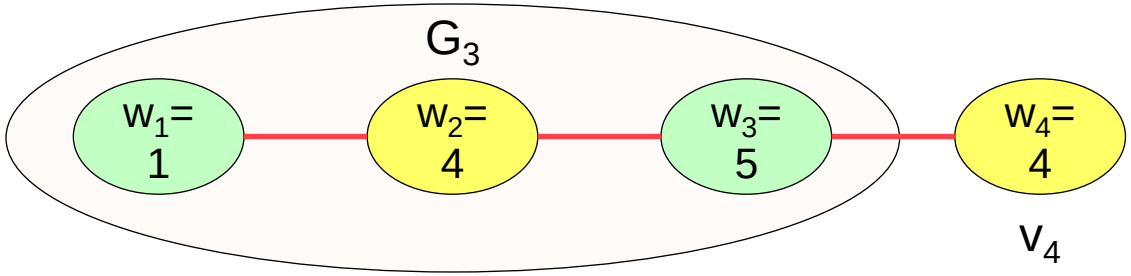
Dynamische Programmierung:

- Algorithmus 2: Bottom-Up-Ansatz:



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Greedy-Algorithmen, Minimum Spanning Trees und dynamische Programmierung, Stanford University, Tim Roughgarden, Coursera

Stabile Menge maximalen Gewichts in Pfadgraphen

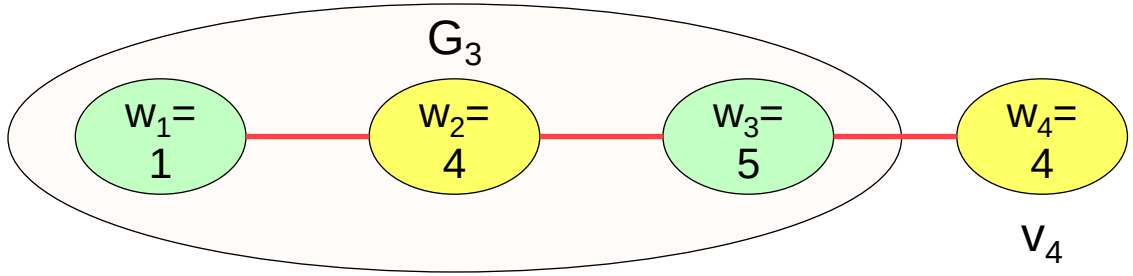


Dynamische Programmierung:

- Algorithmus 2: Bottom-Up-Ansatz:
- Definiere G_i als die ersten i Knoten von G.
-
-
-
  -
  -
  -

Quelle: Greedy-Algorithmen, Minimum Spanning Trees und dynamische Programmierung, Stanford University, Tim Roughgarden, Coursera

Stabile Menge maximalen Gewichts in Pfadgraphen

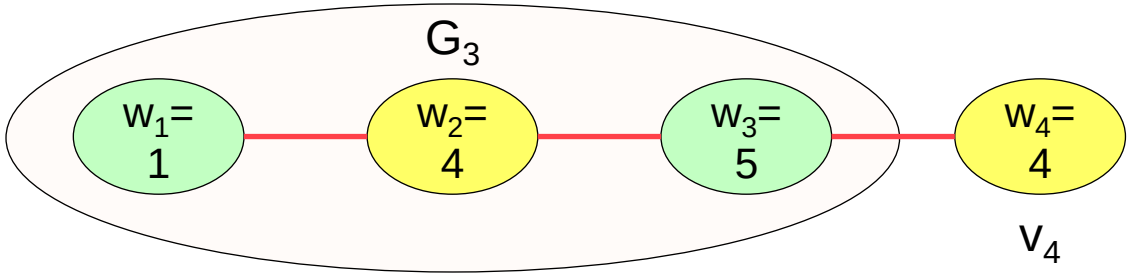


Dynamische Programmierung:

- Algorithmus 2: Bottom-Up-Ansatz:

- Definiere G_i als die ersten i Knoten von G.
- Idee: Speichere im iten Index eines Arrays A: A[i] = Gesamtgewicht der SMMG von G_i
- Initialisierung: A[0] = 0, A[1] = w_1

Stabile Menge maximalen Gewichts in Pfadgraphen



Dynamische Programmierung:

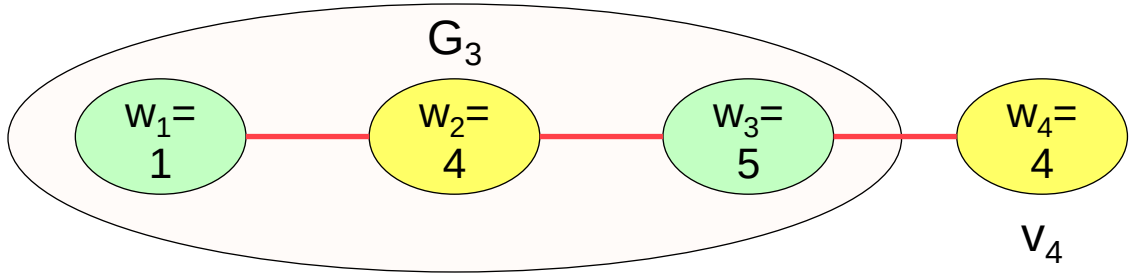
- Algorithmus 2: Bottom-Up-Ansatz:

- Definiere G_i als die ersten i Knoten von G.
- Idee: Speichere im iten Index eines Arrays A: A[i] = Gesamtgewicht der SMMG von G_i
- Initialisierung: A[0] = 0, A[1] = w_1

Main loop:

- For i = 2, 3, 4, …, n:
  - A[i] = max{A[i-1], A[i-2] + w_i}

Stabile Menge maximalen Gewichts in Pfadgraphen



Dynamische Programmierung:

- Algorithmus 2: Bottom-Up-Ansatz:

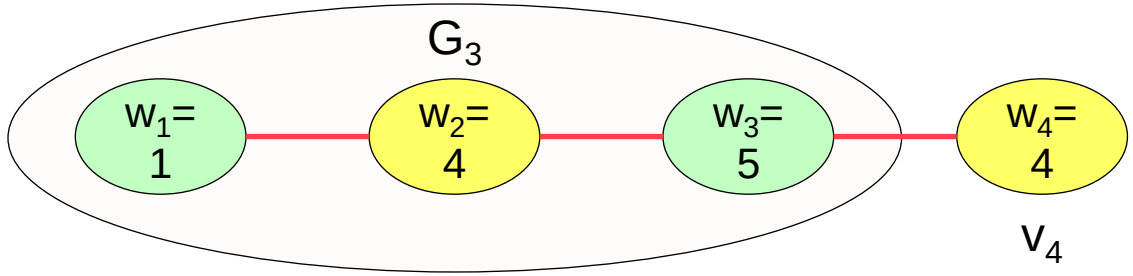
- Definiere G_i als die ersten i Knoten von G.
- Idee: Speichere im iten Index eines Arrays A: A[i] = Gesamtgewicht der SMMG von G_i
- Initialisierung: A[0] = 0, A[1] = w_1

Main loop:

- For i = 2, 3, 4, …, n:
  - A[i] = max{A[i-1], A[i-2] + w_i}

  - A[i-1] ist Fall 1: Gesamtgewicht der SMMG von G_{i-1}
  - A[i-2] + w_i ist Fall 2: Gesamtgewicht der SMMG von G_{i-2} + {v_i}

Stabile Menge maximalen Gewichts in Pfadgraphen



Dynamische Programmierung:

- Algorithmus 2: Bottom-Up-Ansatz:

- Definiere G_i als die ersten i Knoten von G.
- Idee: Speichere im iten Index eines Arrays A: A[i] = Gesamtgewicht der SMMG von G_i
- Initialisierung: A[0] = 0, A[1] = w_1

Main loop:

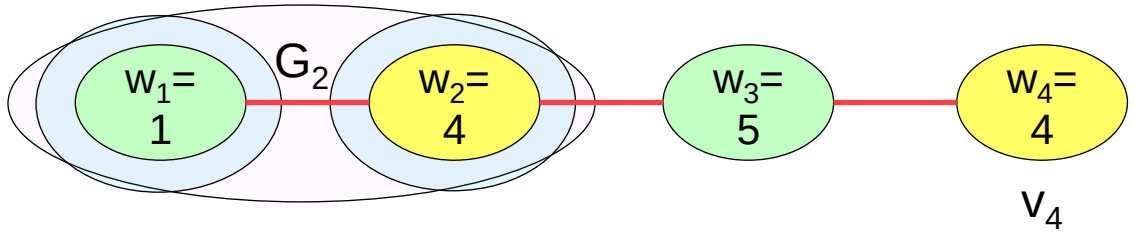
- For i = 2, 3, 4, …, n:
  - A[i] = max{A[i-1], A[i-2] + w_i}

  - A[i-1] ist Fall 1: Gesamtgewicht der SMMG von G_{i-1}
  - A[i-2] + w_i ist Fall 2: Gesamtgewicht der SMMG von G_{i-2} + {v_i}

Laufzeit: O(n)
Speicherplatzbedarf: O(n)

Stabile Menge maximalen Gewichts in Pfadgraphen

Dynamische Programmierung:



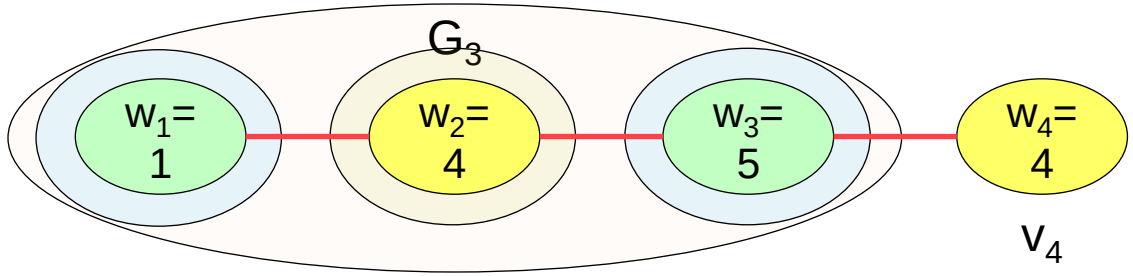
Main loop:

- For i = 2, 3, 4, …, n:
  - A[i] = max{A[i-1], A[i-2] + w_i}

- i = 2:
  - A[2] = max{w_1, 0 + w_2} = 4

Stabile Menge maximalen Gewichts in Pfadgraphen

Dynamische Programmierung:



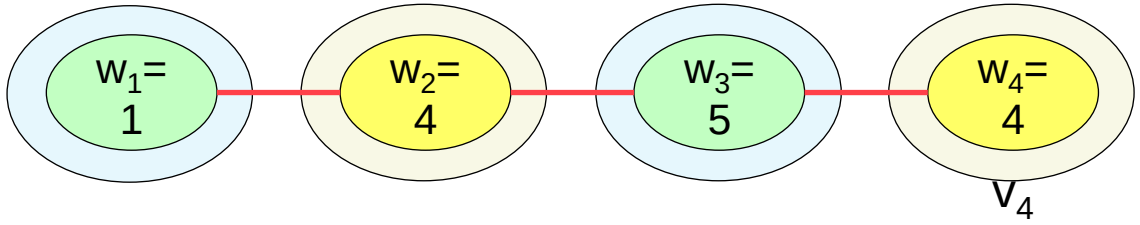
Main loop:

- For i = 2, 3, 4, …, n:
  - A[i] = max{A[i-1], A[i-2] + w_i}

- i = 2:
  - A[2] = max{w_1, 0 + w_2} = 4
- i = 3:
  - A[3] = max{A[2], w_1 + w_3} = max{4, 6} = 6

Stabile Menge maximalen Gewichts in Pfadgraphen

Dynamische Programmierung:



Main loop:

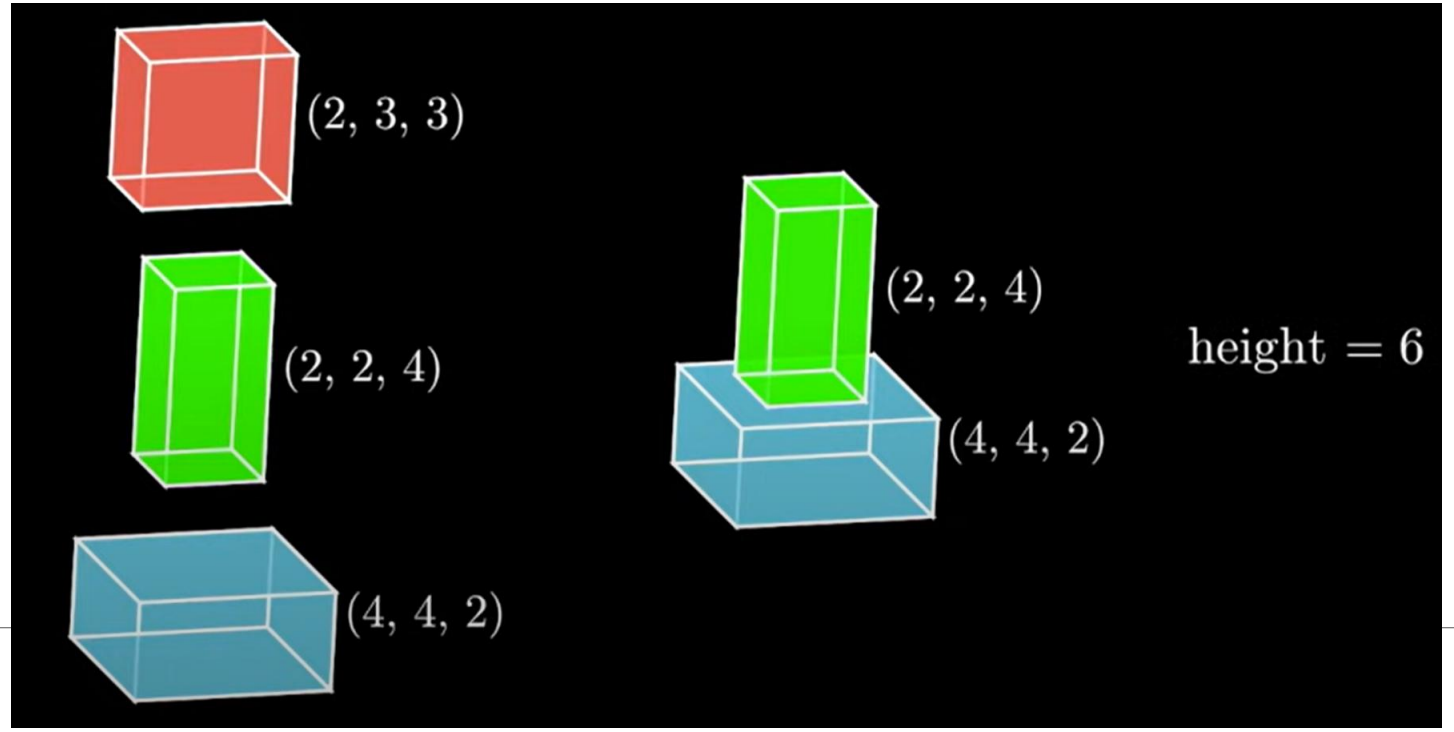
- For i = 2, 3, 4, …, n:
  - A[i] = max{A[i-1], A[i-2] + w_i}

- i = 2:
  - A[2] = max{w_1, 0 + w_2} = 4
- i = 3:
  - A[3] = max{A[2], w_1 + w_3} = max{4, 6} = 6
- i = 4:
  - A[4] = max{A[3], A[2] + w_4} = max{6, 4 + 4} = 8

Box Stacking Problem

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

- Gegeben n Kisten, wobei Kiste i die Länge L_i, die Breite W_i und die Höhe H_i hat
- Wie hoch kann man die Kisten maximal stapeln?
-
  -

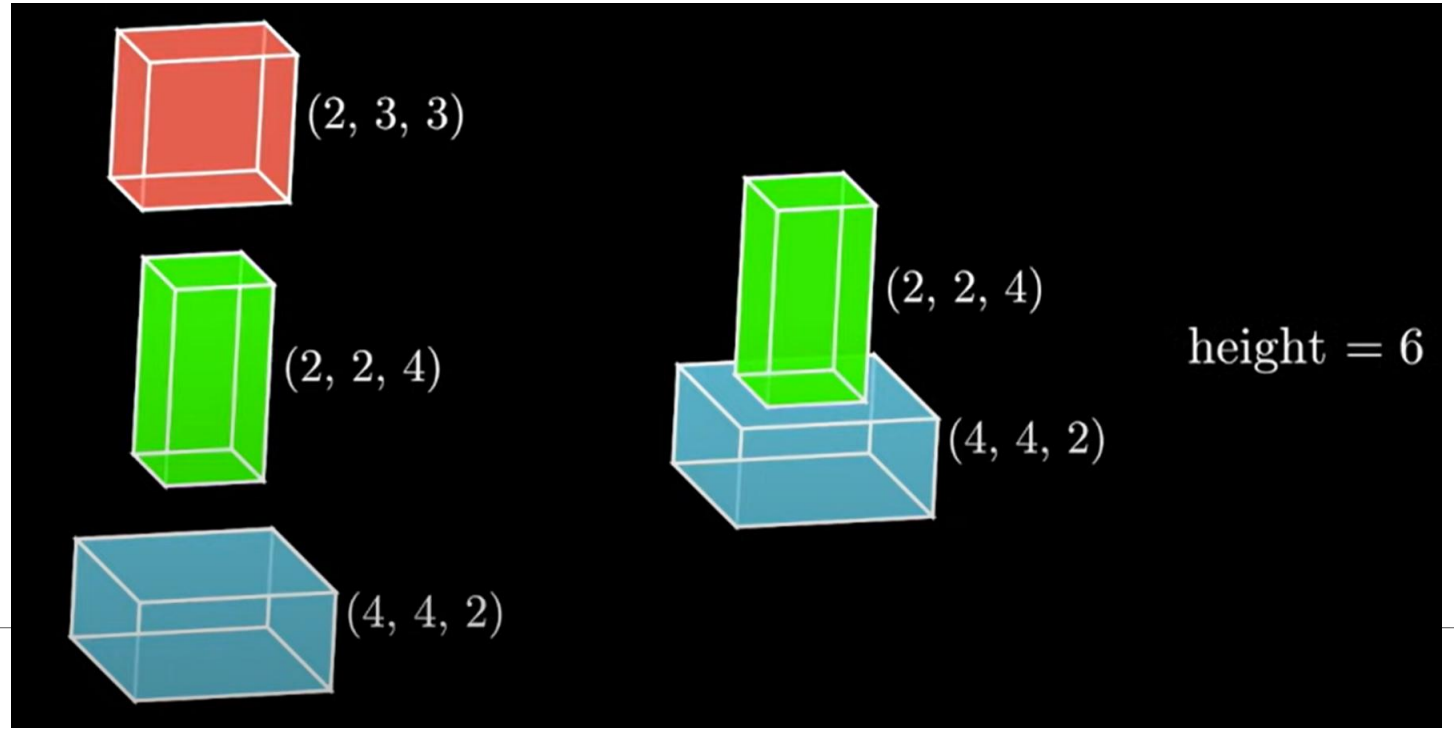


[5 Simple Steps for Solving Dynamic Programming Problems]

Box Stacking Problem

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

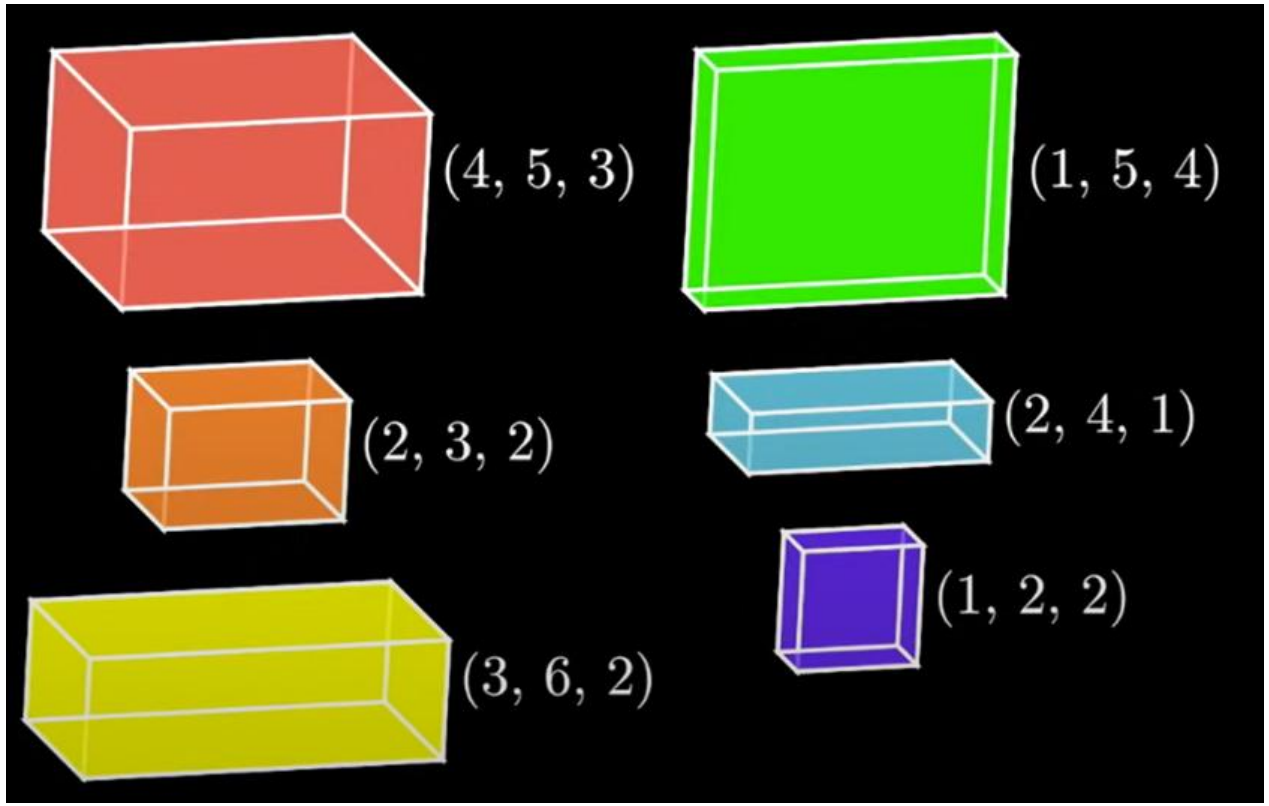
- Gegeben n Kisten, wobei Kiste i die Länge L_i, die Breite W_i und die Höhe H_i hat
- Wie hoch kann man die Kisten maximal stapeln?
- Ein gültiger Stapel ist, wenn gilt:
  - Kiste (L_i, W_i, H_i) darf auf Kiste (L_j, W_j, H_j) gestapelt werden, wenn $L_i < L_j$, $W_i < W_j$



[5 Simple Steps for Solving Dynamic Programming Problems]

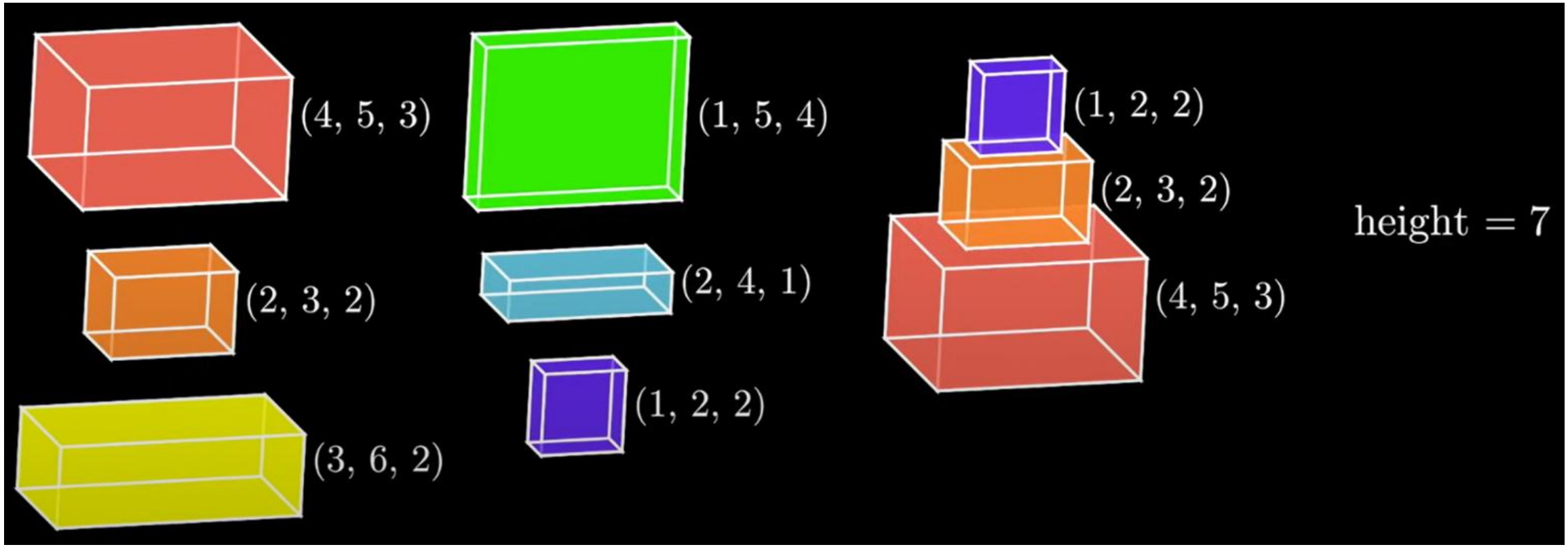
Box Stacking Problem: Beispiel

Wie sieht der höchstmögliche, gültige Stapel aus den folgenden Kisten aus?

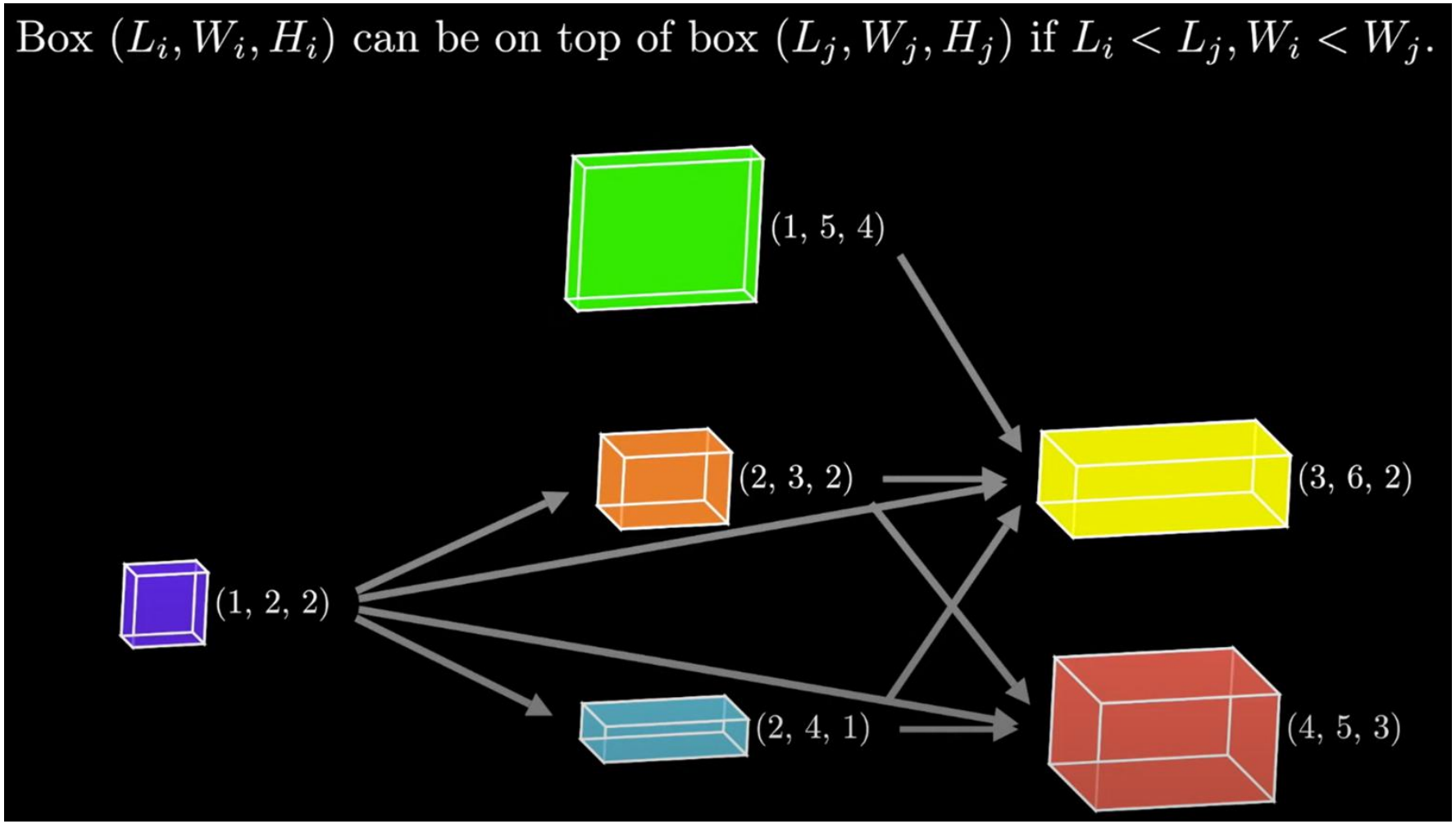


Box Stacking Problem: Beispiel

Wie sieht der höchstmögliche, gültige Stapel aus den folgenden Kisten aus?



DAG, um anzugeben, welche Box auf welche gestapelt werden kann



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

0/1 Knapsack Problem (0-1-Rucksackproblem) - Anwendungen

Online-Werbung optimal platzieren

- Marketingabteilung mit begrenztem Budget
- Verschiedene Kanäle mit unterschiedlichen Kosten und #Kunden:
  - Linkedin, Facebook, Instagram, …
- Welche Kanäle nutzen wir für Werbung für maximalen Ertrag?

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

0/1 Knapsack Problem (0-1-Rucksackproblem) - Anwendungen

Online-Werbung optimal platzieren

- Marketingabteilung mit begrenztem Budget
- Verschiedene Kanäle mit unterschiedlichen Kosten und #Kunden:
  - Linkedin, Facebook, Instagram, …
- Welche Kanäle nutzen wir für Werbung für maximalen Ertrag?

- Ähnlich:
  - Ausschreibungen platzieren.

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

0/1 Knapsack Problem (0-1-Rucksackproblem) - Anwendungen

Online-Werbung optimal platzieren

- Marketingabteilung mit begrenztem Budget
- Verschiedene Kanäle mit unterschiedlichen Kosten und #Kunden:
  - Linkedin, Facebook, Instagram, …
- Welche Kanäle nutzen wir für Werbung für maximalen Ertrag?

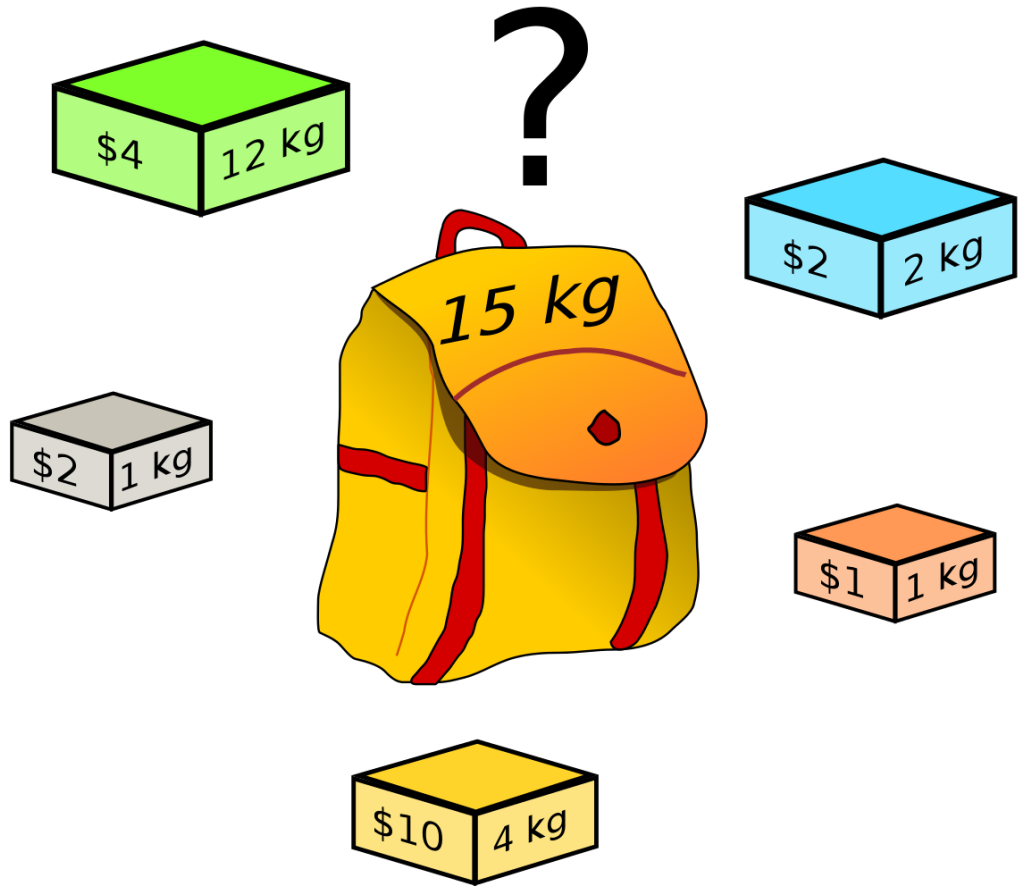
- Ähnlich:
  - Ausschreibungen platzieren.

Cloud-Computing

- Budget für virtuelle Maschinen
- Virtuelle Maschinen haben unterschiedliche Leistungskennzahlen und Kosten
- Welche virtuelle Maschinen starten wir, um maximale Leistung herauszuholen

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

0/1 Knapsack Problem (0-1-Rucksackproblem)



Prof. Dr. Daniel Gaida

Professor für Cyber-Physische Systeme

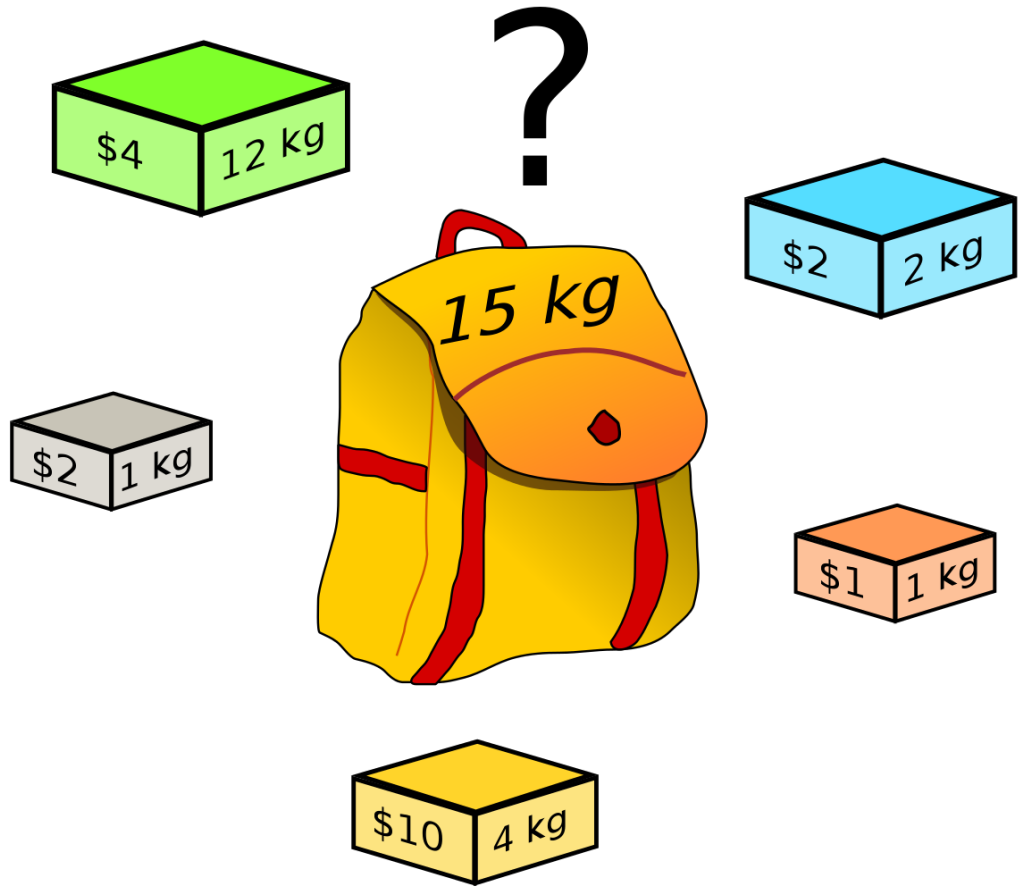
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Bildquelle: wikipedia.de

0/1 Knapsack Problem (0-1-Rucksackproblem)

Wie groß ist der maximale Wert, den man durch die Auswahl einer Teilmenge von Objekten mit einem Wert und einem Gewicht (v_i, w_i) erhalten kann, so dass die Summe der Gewichte die vorgegebene Kapazität des Rucksacks nicht überschreitet?



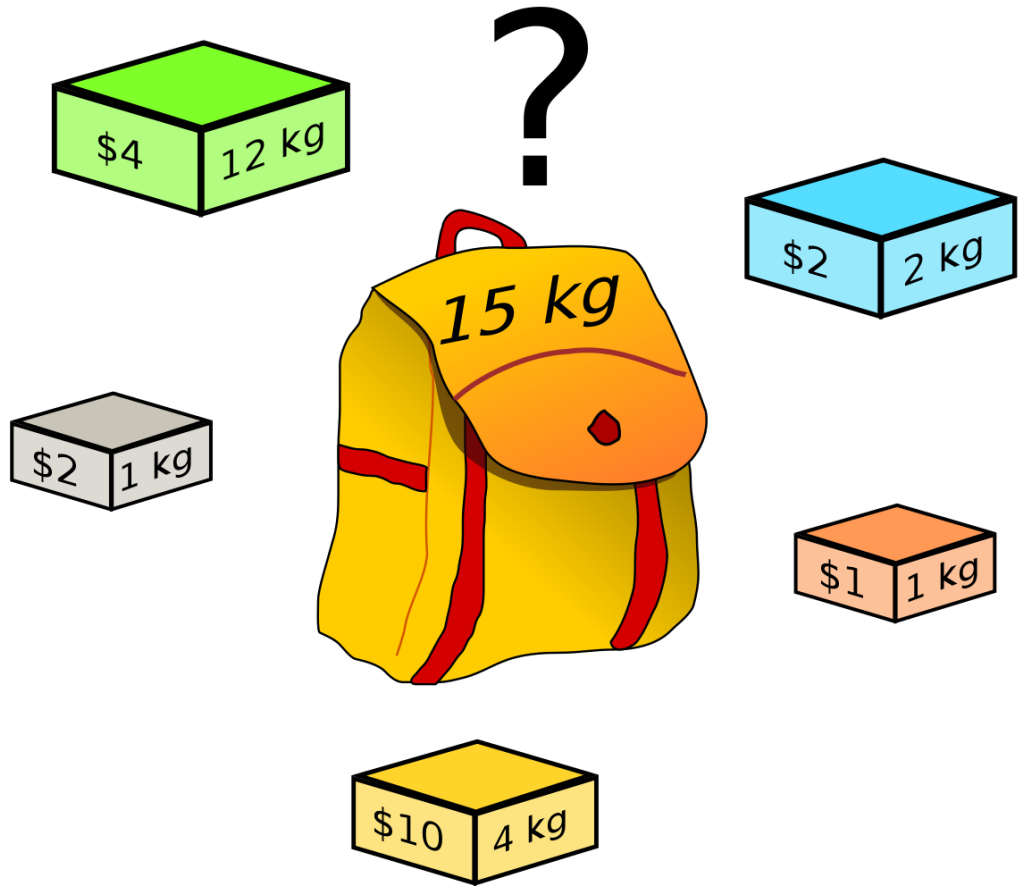
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022
Bildquelle: wikipedia.de

0/1 Knapsack Problem (0-1-Rucksackproblem)

Wie groß ist der maximale Wert, den man durch die Auswahl einer Teilmenge von Objekten mit einem Wert und einem Gewicht (v_i, w_i) erhalten kann, so dass die Summe der Gewichte die vorgegebene Kapazität des Rucksacks nicht überschreitet?

→ Dieb, der maximal hohen Wert mitnehmen möchte, aber nur begrenzte Transportmöglichkeit

0/1, da für jedes Objekt eine binäre Entscheidung zu treffen ist



[Problem Walk Through Video]

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Bildquelle: wikipedia.de

0-1-Rucksackproblem: Rekursiver Brute-Force Ansatz (Beispiel)

i	0	1	2	3	4
Gewichte w_i	1	2	4	2	5
Werte v_i	5	3	5	3	2

Kapazität: 10
Anzahl Objekte (n): 5

In jeder Iteration:
Wert im Rucksack: VR
Restkapazität des Rucksacks: CR

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

0-1-Rucksackproblem: Rekursiver Brute-Force Ansatz (Beispiel)

i	0	1	2	3	4
Gewichte w_i	1	2	4	2	5
Werte v_i	5	3	5	3	2

CR: 10
VR: 0

Kapazität: 10
Anzahl Objekte (n): 5

In jeder Iteration:
Wert im Rucksack: VR
Restkapazität des Rucksacks: CR

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

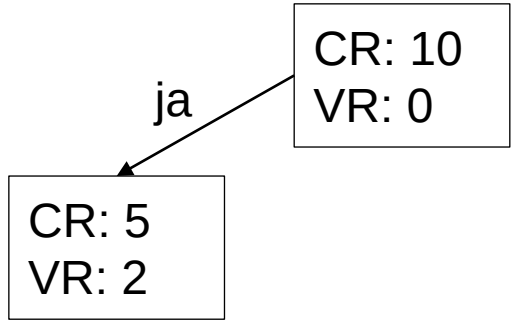
0-1-Rucksackproblem: Rekursiver Brute-Force Ansatz (Beispiel)

i	0	1	2	3	4
Gewichte w_i	1	2	4	2	5
Werte v_i	5	3	5	3	2

Kapazität: 10
Anzahl Objekte (n): 5

In jeder Iteration:
Wert im Rucksack: VR
Restkapazität des Rucksacks: CR

Objekt i=4 einfügen?



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

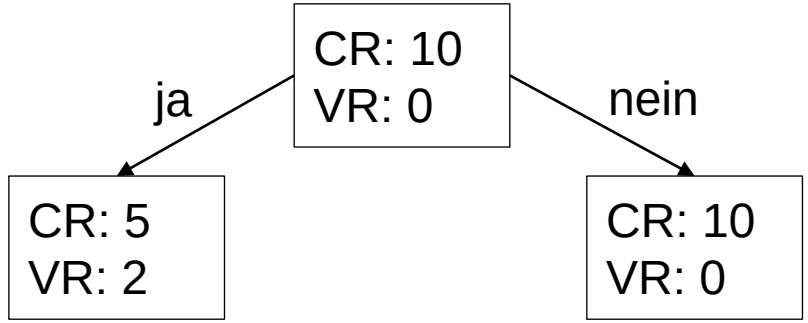
0-1-Rucksackproblem: Rekursiver Brute-Force Ansatz (Beispiel)

i	0	1	2	3	4
Gewichte w_i	1	2	4	2	5
Werte v_i	5	3	5	3	2

Kapazität: 10
Anzahl Objekte (n): 5

In jeder Iteration:
Wert im Rucksack: VR
Restkapazität des Rucksacks: CR

Objekt i=4 einfügen?



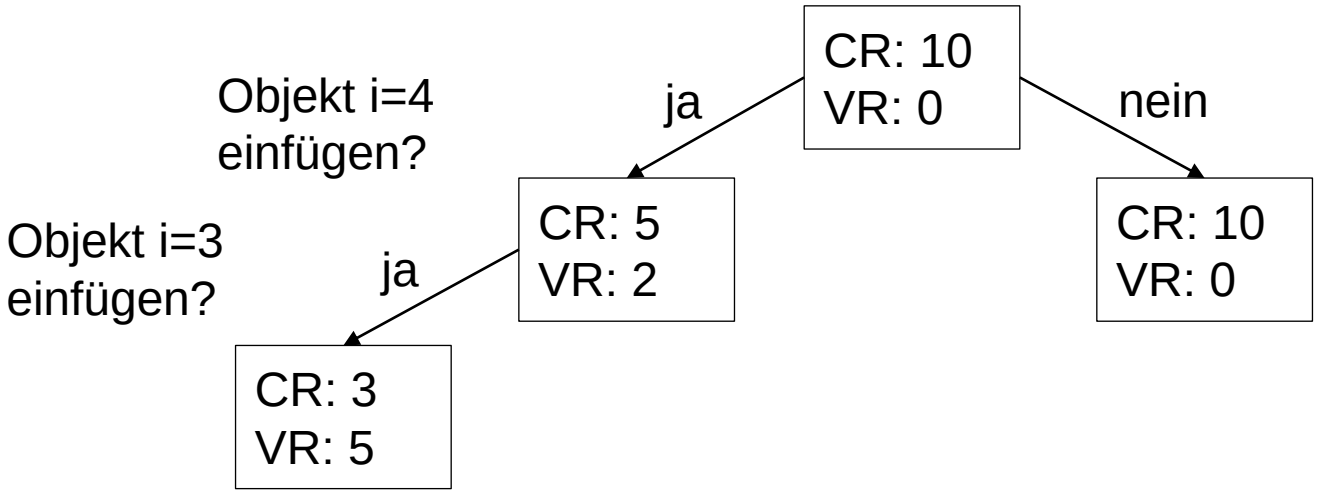
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

0-1-Rucksackproblem: Rekursiver Brute-Force Ansatz (Beispiel)

i	0	1	2	3	4
Gewichte w_i	1	2	4	2	5
Werte v_i	5	3	5	3	2

Kapazität: 10
Anzahl Objekte (n): 5

In jeder Iteration:
Wert im Rucksack: VR
Restkapazität des Rucksacks: CR



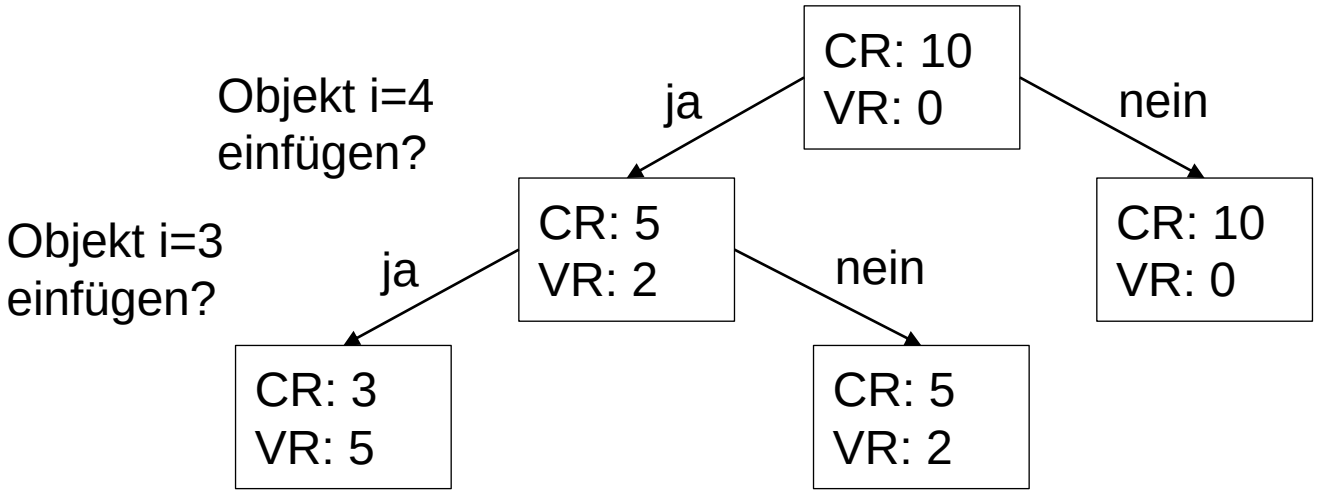
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

0-1-Rucksackproblem: Rekursiver Brute-Force Ansatz (Beispiel)

i	0	1	2	3	4
Gewichte w_i	1	2	4	2	5
Werte v_i	5	3	5	3	2

Kapazität: 10
Anzahl Objekte (n): 5

In jeder Iteration:
Wert im Rucksack: VR
Restkapazität des Rucksacks: CR



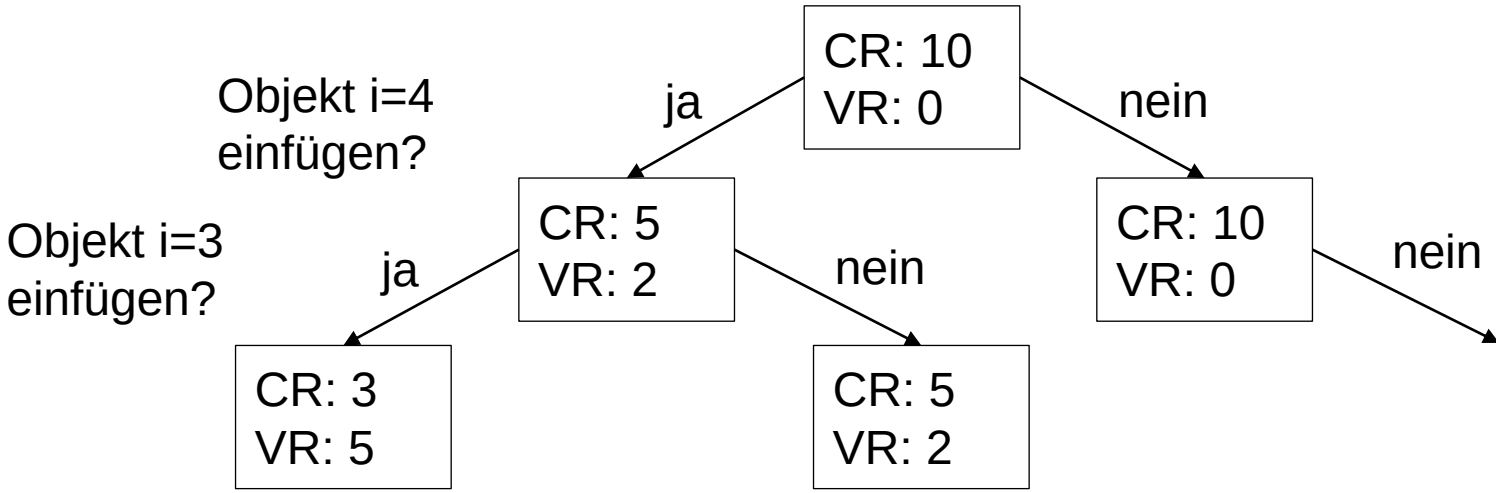
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

0-1-Rucksackproblem: Rekursiver Brute-Force Ansatz (Beispiel)

i	0	1	2	3	4
Gewichte w_i	1	2	4	2	5
Werte v_i	5	3	5	3	2

Kapazität: 10
Anzahl Objekte (n): 5

In jeder Iteration:
Wert im Rucksack: VR
Restkapazität des Rucksacks: CR



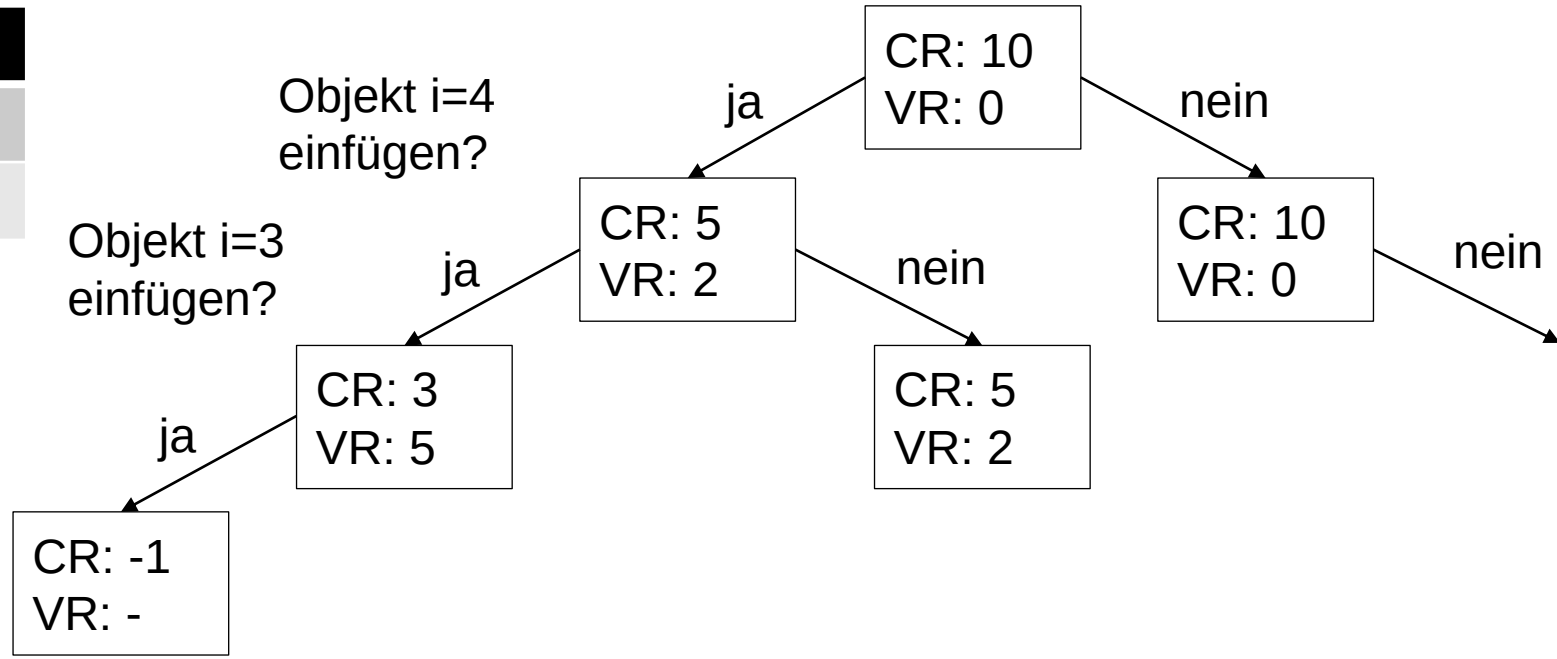
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

0-1-Rucksackproblem: Rekursiver Brute-Force Ansatz (Beispiel)

i	0	1	2	3	4
Gewichte w_i	1	2	4	2	5
Werte v_i	5	3	5	3	2

Kapazität: 10
Anzahl Objekte (n): 5

In jeder Iteration:
Wert im Rucksack: VR
Restkapazität des Rucksacks: CR



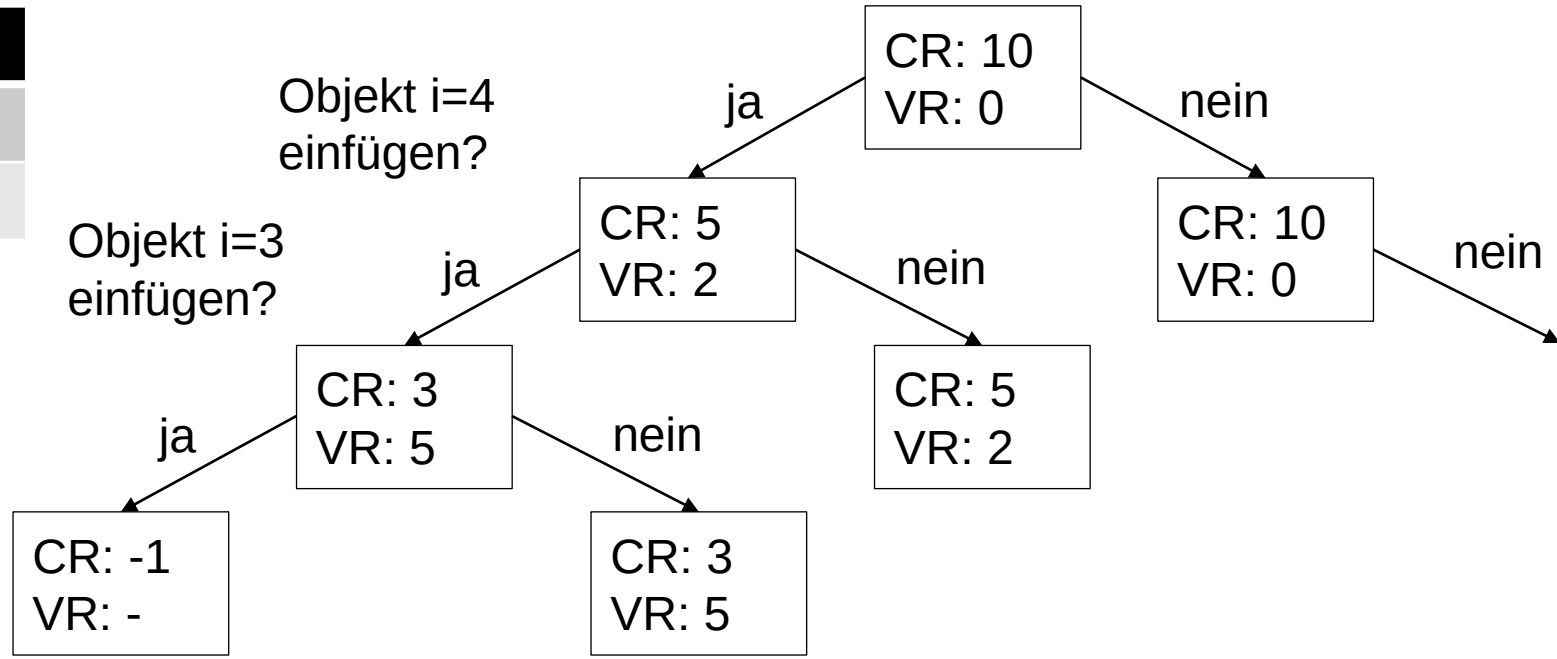
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

0-1-Rucksackproblem: Rekursiver Brute-Force Ansatz (Beispiel)

i	0	1	2	3	4
Gewichte w_i	1	2	4	2	5
Werte v_i	5	3	5	3	2

Kapazität: 10
Anzahl Objekte (n): 5

In jeder Iteration:
Wert im Rucksack: VR
Restkapazität des Rucksacks: CR



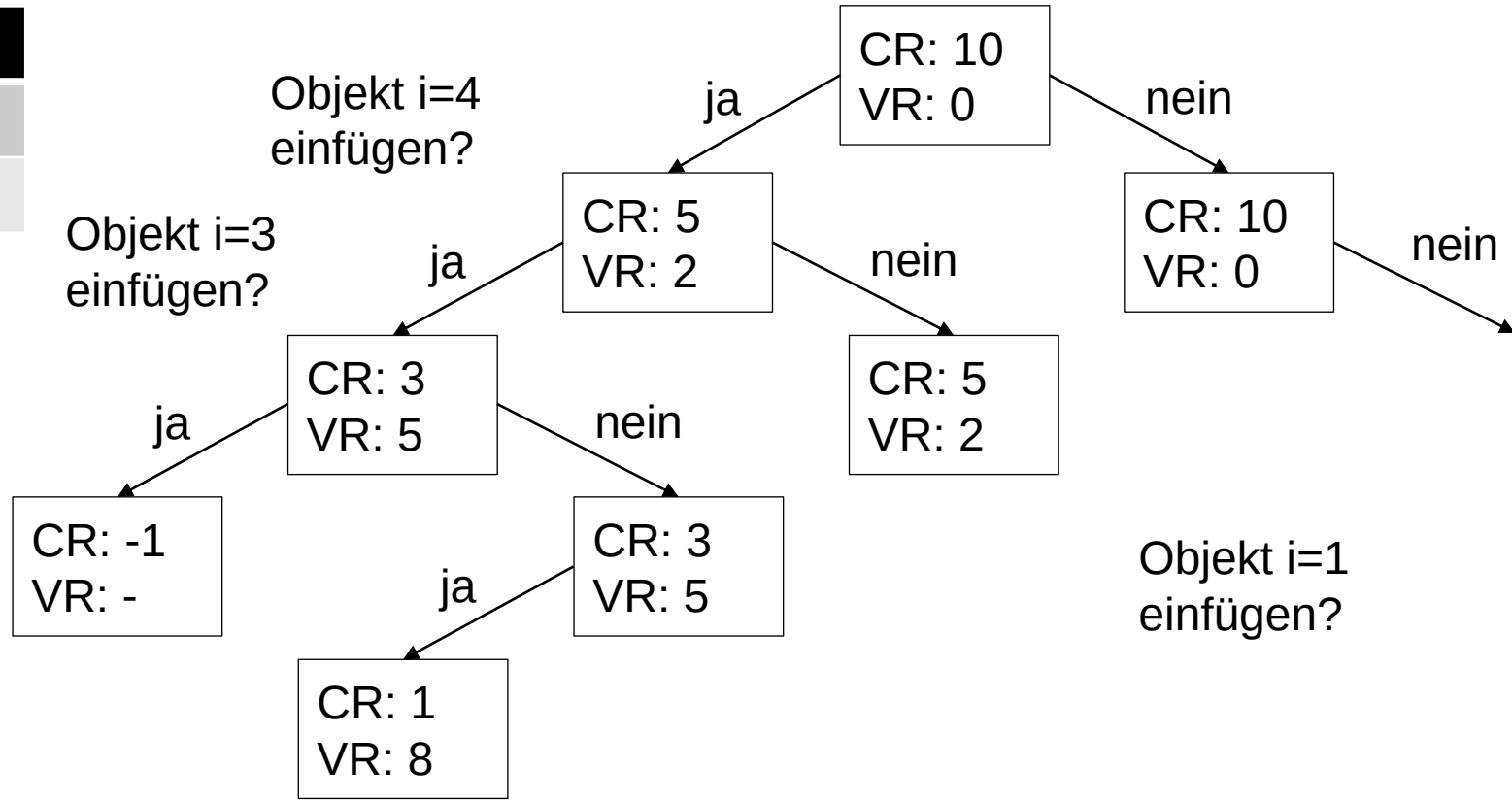
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

0-1-Rucksackproblem: Rekursiver Brute-Force Ansatz (Beispiel)

i	0	1	2	3	4
Gewichte w_i	1	2	4	2	5
Werte v_i	5	3	5	3	2

Kapazität: 10
Anzahl Objekte (n): 5

In jeder Iteration:
Wert im Rucksack: VR
Restkapazität des Rucksacks: CR



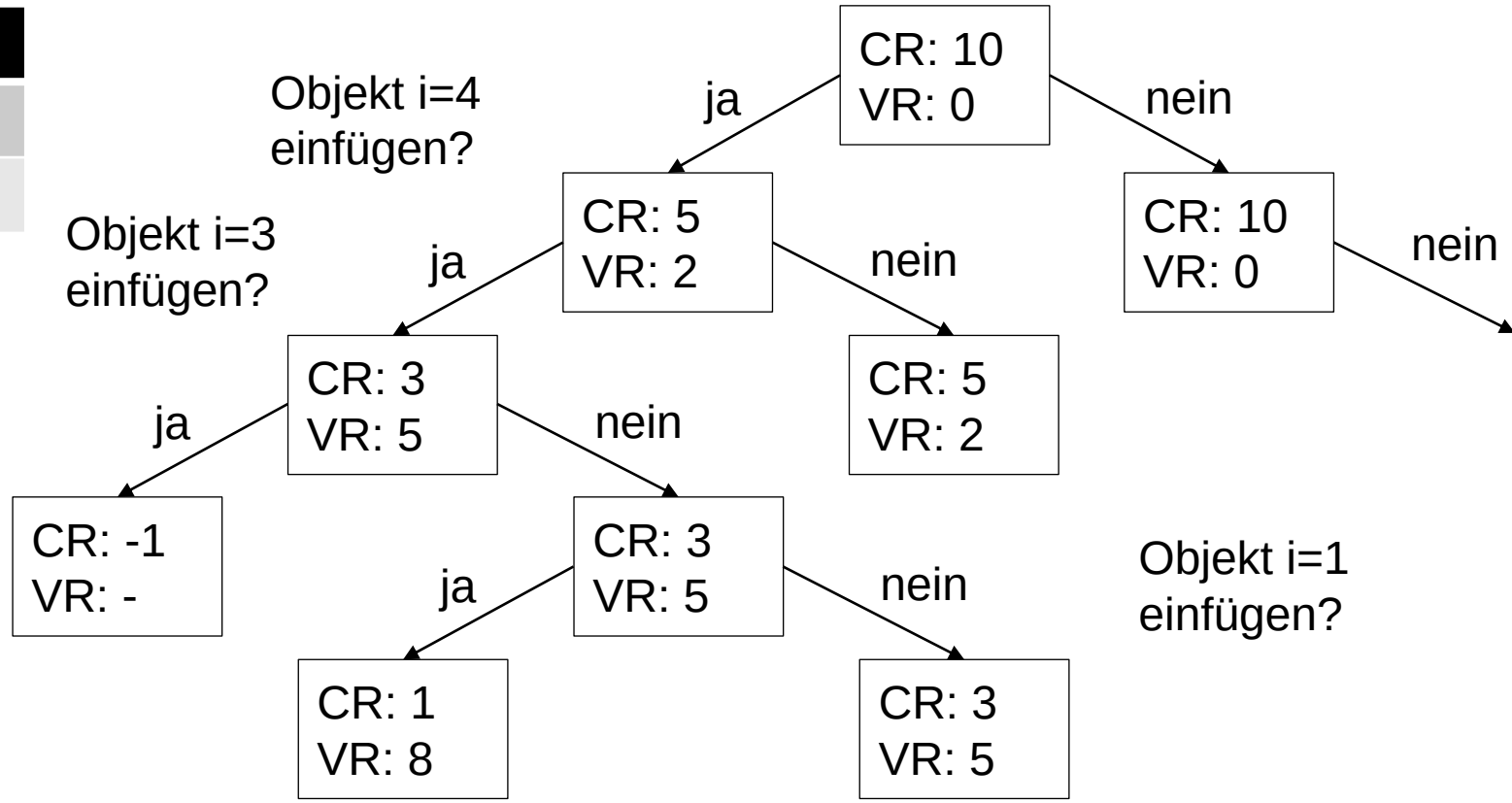
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

0-1-Rucksackproblem: Rekursiver Brute-Force Ansatz (Beispiel)

i	0	1	2	3	4
Gewichte w_i	1	2	4	2	5
Werte v_i	5	3	5	3	2

Kapazität: 10
Anzahl Objekte (n): 5

In jeder Iteration:
Wert im Rucksack: VR
Restkapazität des Rucksacks: CR



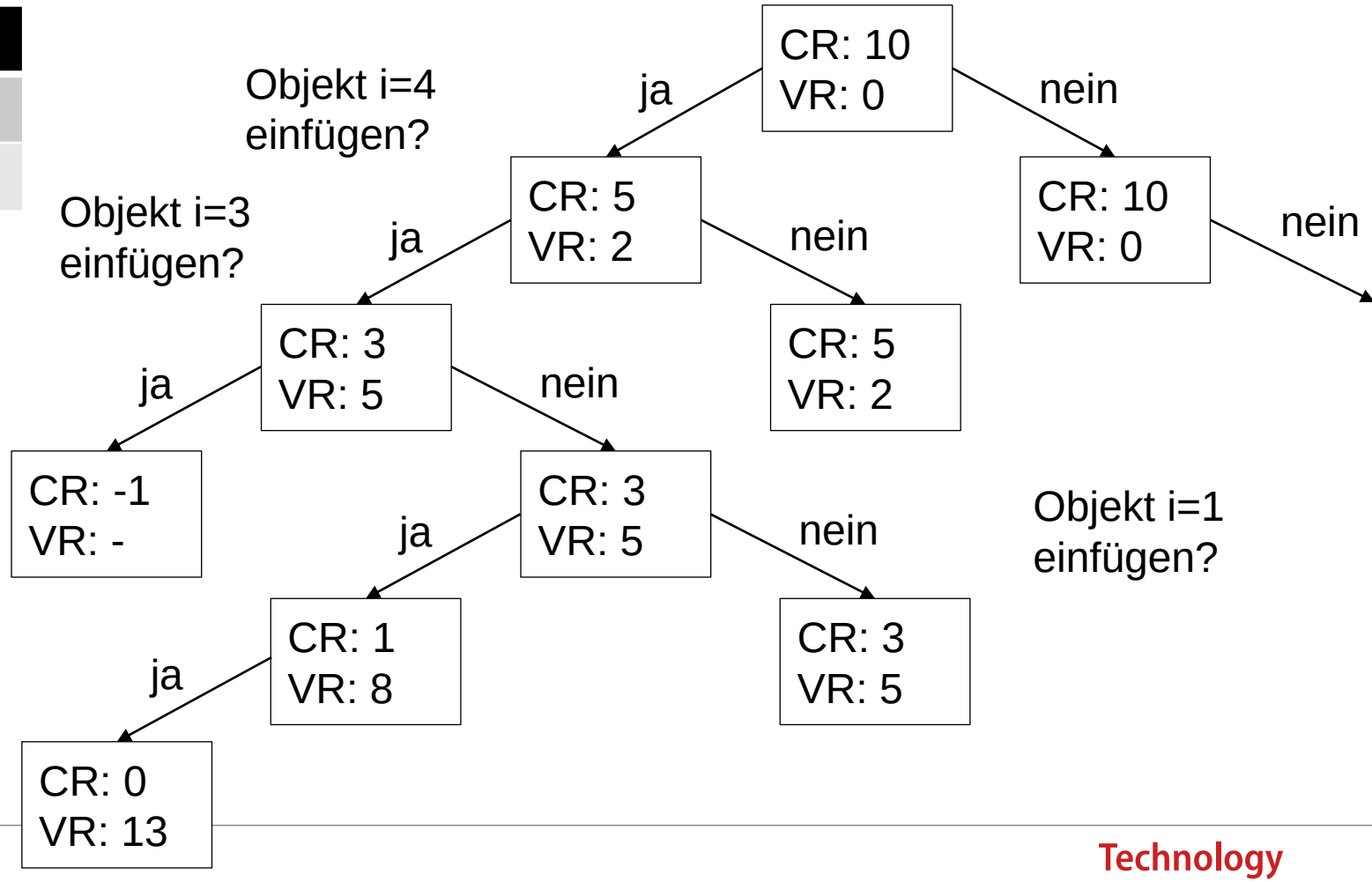
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

0-1-Rucksackproblem: Rekursiver Brute-Force Ansatz (Beispiel)

i	0	1	2	3	4
Gewichte w_i	1	2	4	2	5
Werte v_i	5	3	5	3	2

Kapazität: 10
Anzahl Objekte (n): 5

In jeder Iteration:
Wert im Rucksack: VR
Restkapazität des Rucksacks: CR



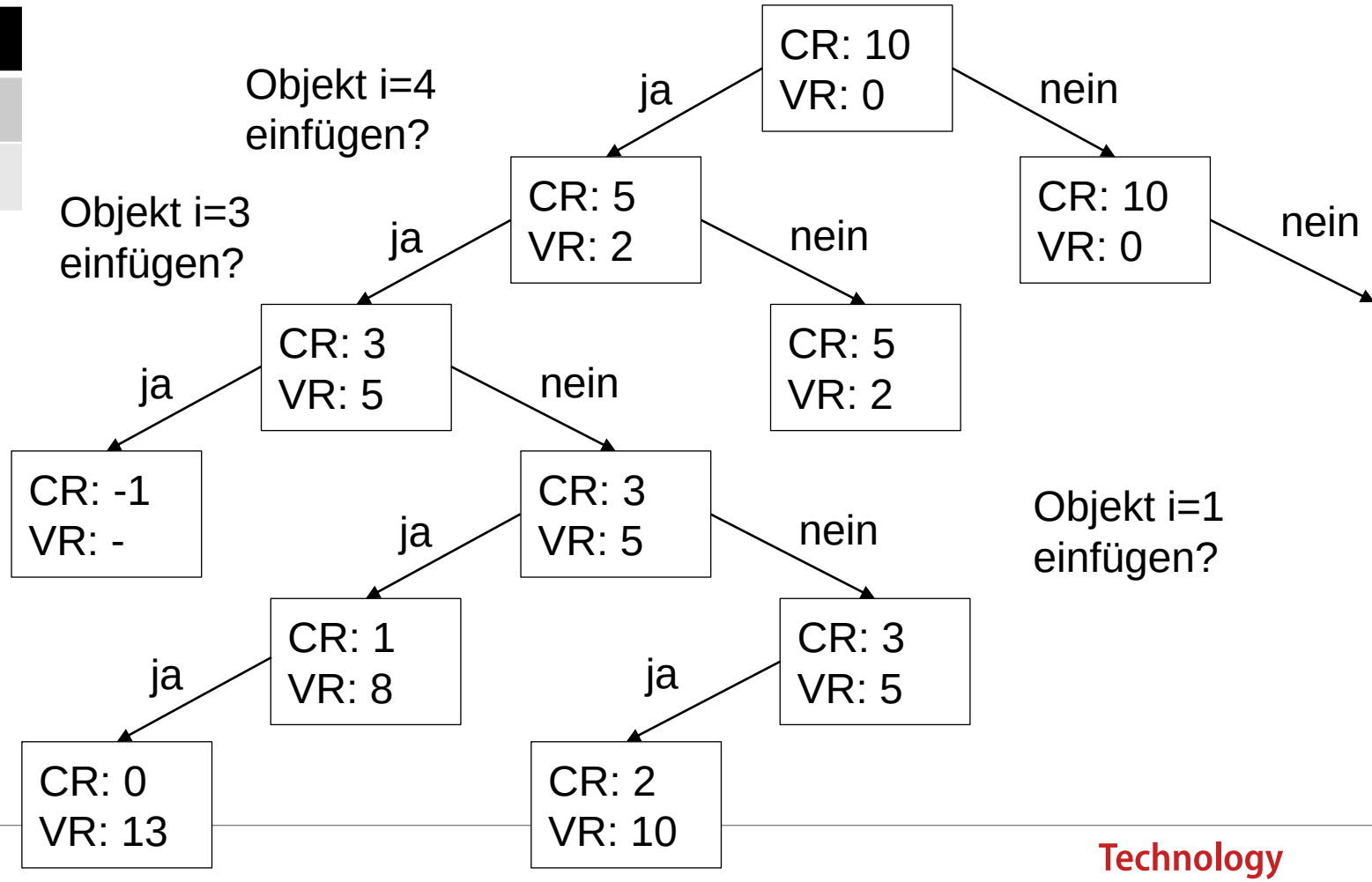
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

0-1-Rucksackproblem: Rekursiver Brute-Force Ansatz (Beispiel)

i	0	1	2	3	4
Gewichte w_i	1	2	4	2	5
Werte v_i	5	3	5	3	2

Kapazität: 10
Anzahl Objekte (n): 5

In jeder Iteration:
Wert im Rucksack: VR
Restkapazität des Rucksacks: CR



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

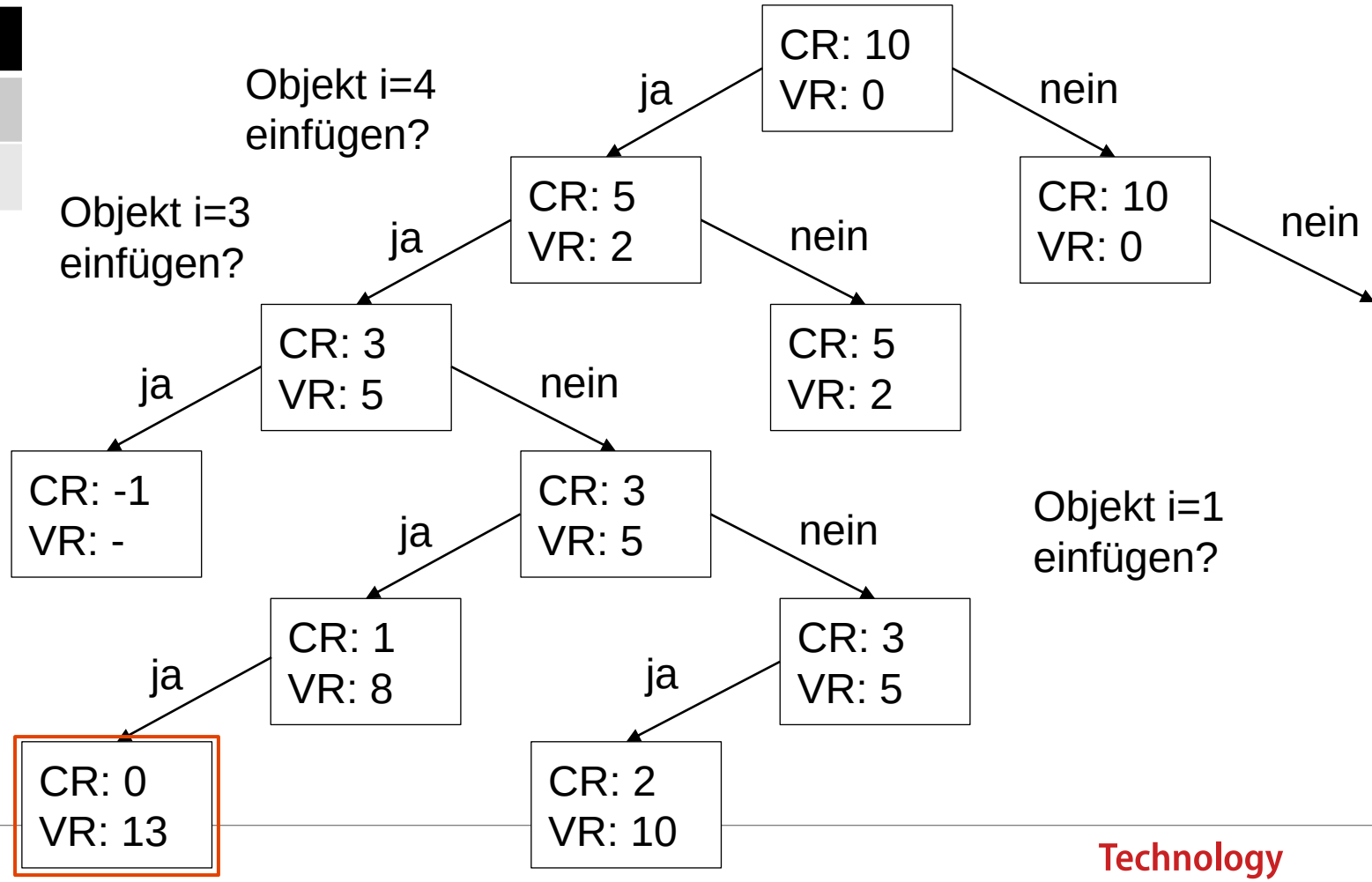
0-1-Rucksackproblem: Rekursiver Brute-Force Ansatz (Beispiel)

i	0	1	2	3	4
Gewichte w_i	1	2	4	2	5
Werte v_i	5	3	5	3	2

Kapazität: 10
Anzahl Objekte (n): 5

In jeder Iteration:
Wert im Rucksack: VR
Restkapazität des Rucksacks: CR

Laufzeit: $O(2^n)$



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

0-1-Rucksackproblem: Rekursiver Brute-Force Ansatz

```
public int knapsack(int i, int cap, int[] w, int[] v) {
   if (i == -1 || cap == 0) {              // kein Platz mehr im Rucksack
      value = 0;
   } else if (w[i] > cap) {                // ites Objekt zu schwer für Rucksack
      value = knapsack(i-1, cap, w, v);
   } else {                              // ites Objekt einfügen oder nicht? Was ist besser?
      int vno = knapsack(i-1, cap, w, v);
      int vyes = v[i] + knapsack(i-1, cap - w[i], w, v);
      value = Math.max(vno, vyes);
   }
   return value;
}
```

Erster Aufruf: knapsack(n-1, …)

0-1-Rucksackproblem: Rekursiver Brute-Force Ansatz

```
public int knapsack(int i, int cap, int[] w, int[] v) {
   if (i == -1 || cap == 0) {              // kein Platz mehr im Rucksack
      value = 0;
   } else if (w[i] > cap) {                // ites Objekt zu schwer für Rucksack
      value = knapsack(i-1, cap, w, v);
   } else {                              // ites Objekt einfügen oder nicht? Was ist besser?
      int vno = knapsack(i-1, cap, w, v);
      int vyes = v[i] + knapsack(i-1, cap - w[i], w, v);
      value = Math.max(vno, vyes);
   }
   return value;
}
```

Erster Aufruf: knapsack(n-1, …)

Laufzeit: $O(2^n)$

0-1-Rucksackproblem: Beispiel

i	0	1	2
w_i	1	4	4
v_i	5	3	5

Kapazität: 10
Anzahl Objekte (n): 3

In jeder Iteration:
Restkapazität des Rucksacks: CR

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

0-1-Rucksackproblem: Beispiel

i	0	1	2
w_i	1	4	4
v_i	5	3	5

CR: 10

Kapazität: 10
Anzahl Objekte (n): 3

In jeder Iteration:
Restkapazität des Rucksacks: CR

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

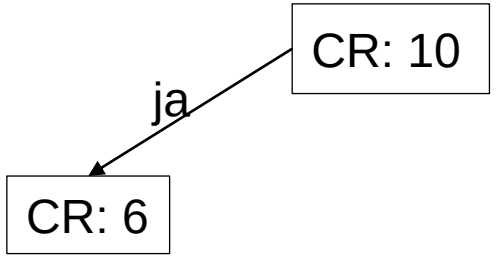
0-1-Rucksackproblem: Beispiel

i	0	1	2
w_i	1	4	4
v_i	5	3	5

Kapazität: 10
Anzahl Objekte (n): 3

In jeder Iteration:
Restkapazität des Rucksacks: CR

Objekt i=2 einfügen?



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

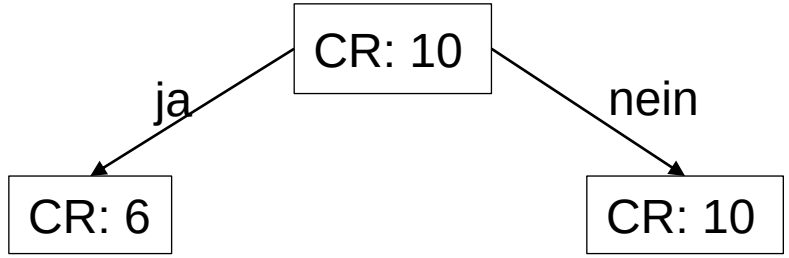
0-1-Rucksackproblem: Beispiel

i	0	1	2
w_i	1	4	4
v_i	5	3	5

Kapazität: 10
Anzahl Objekte (n): 3

In jeder Iteration:
Restkapazität des Rucksacks: CR

Objekt i=2 einfügen?



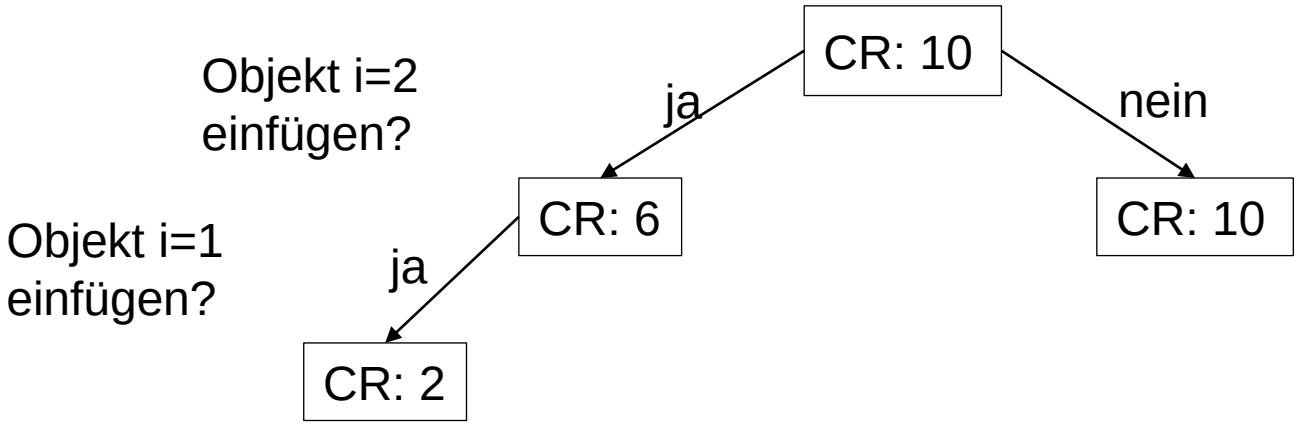
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

0-1-Rucksackproblem: Beispiel

i	0	1	2
w_i	1	4	4
v_i	5	3	5

Kapazität: 10
Anzahl Objekte (n): 3

In jeder Iteration:
Restkapazität des Rucksacks: CR



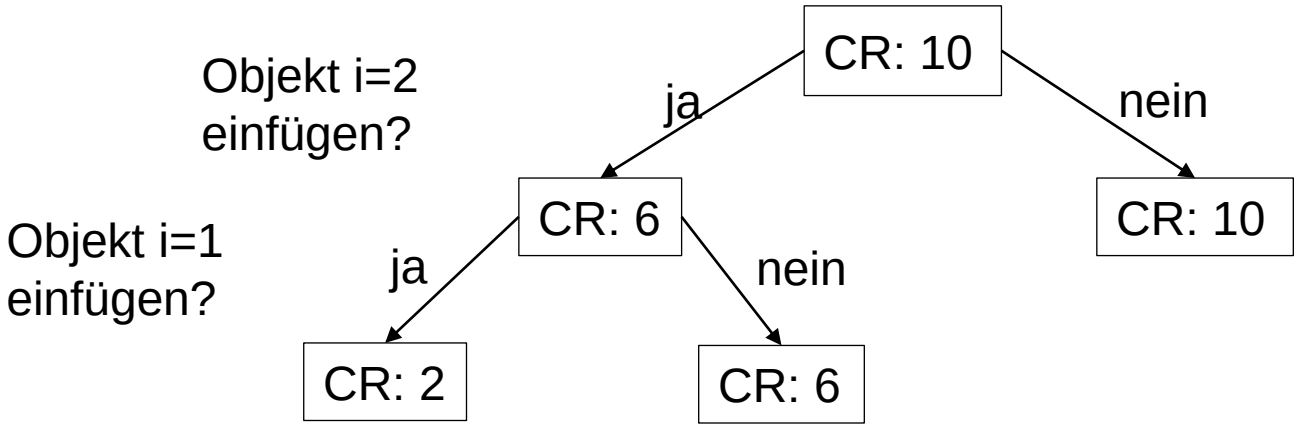
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

0-1-Rucksackproblem: Beispiel

i	0	1	2
w_i	1	4	4
v_i	5	3	5

Kapazität: 10
Anzahl Objekte (n): 3

In jeder Iteration:
Restkapazität des Rucksacks: CR



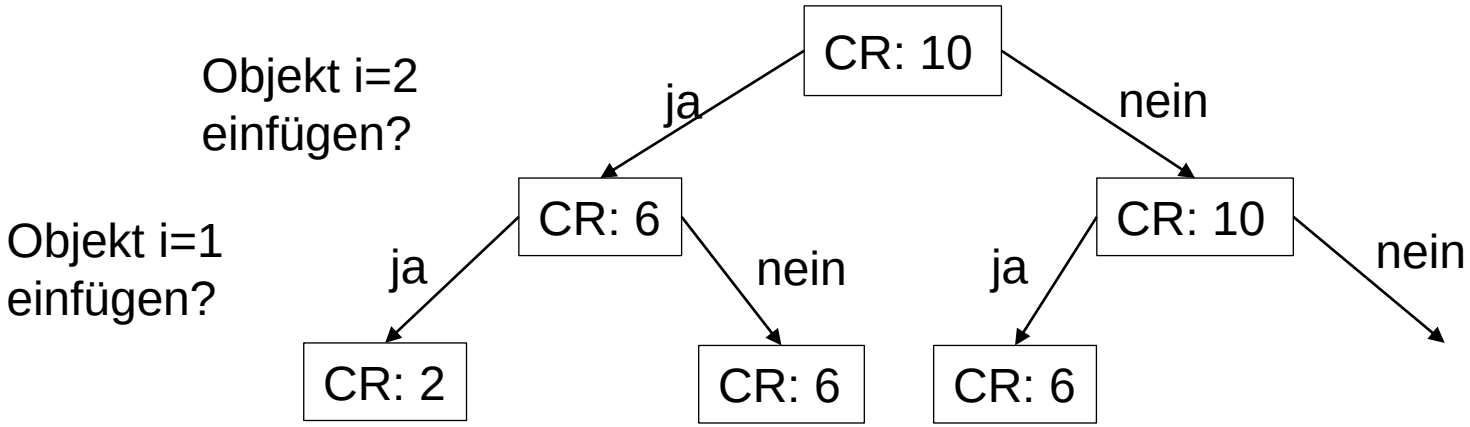
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

0-1-Rucksackproblem: Beispiel

i	0	1	2
w_i	1	4	4
v_i	5	3	5

Kapazität: 10
Anzahl Objekte (n): 3

In jeder Iteration:
Restkapazität des Rucksacks: CR



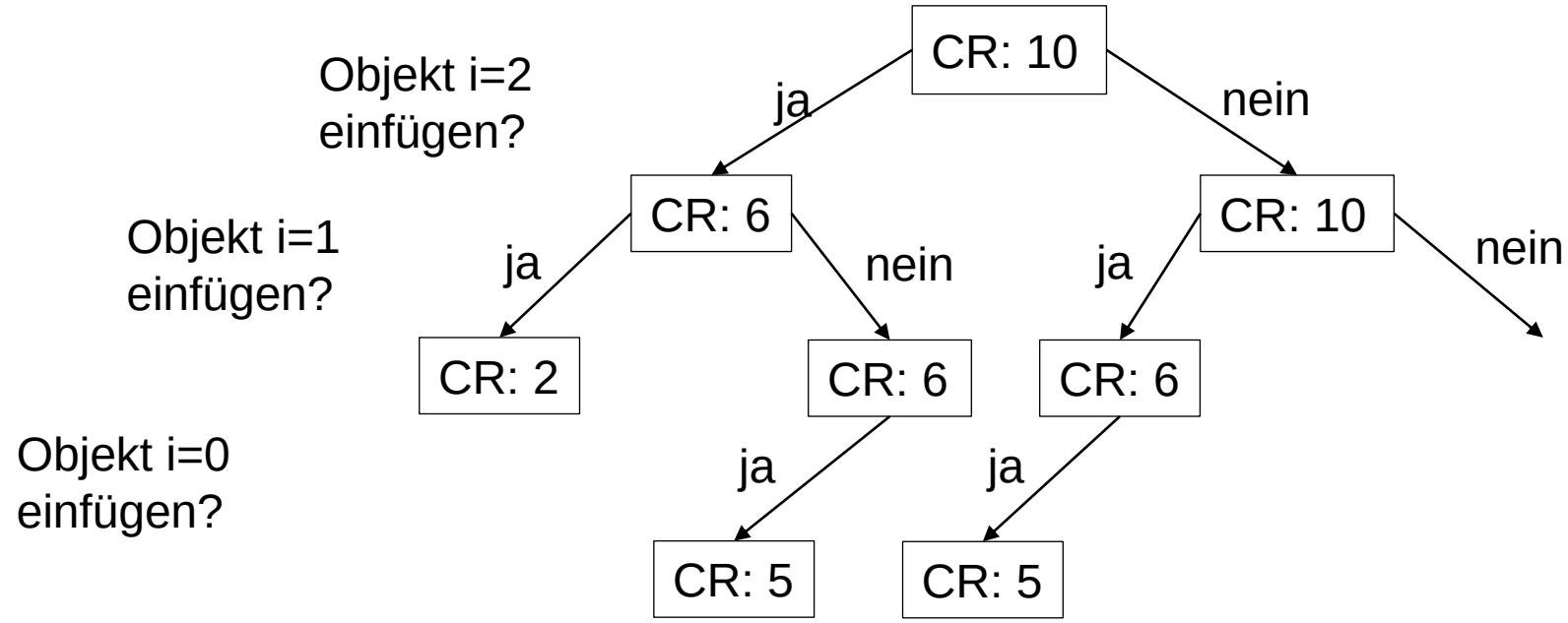
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

0-1-Rucksackproblem: Beispiel

i	0	1	2
w_i	1	4	4
v_i	5	3	5

Kapazität: 10
Anzahl Objekte (n): 3

In jeder Iteration:
Restkapazität des Rucksacks: CR



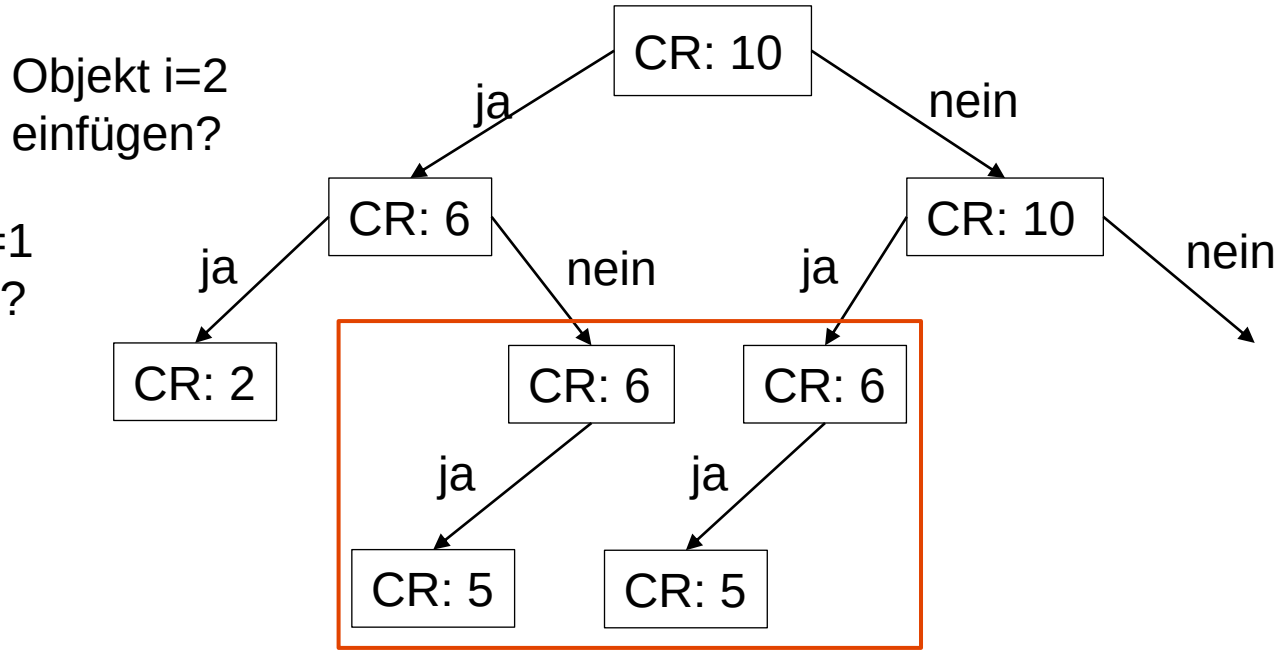
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

0-1-Rucksackproblem: Beispiel

i	0	1	2
w_i	1	4	4
v_i	5	3	5

Kapazität: 10
Anzahl Objekte (n): 3

In jeder Iteration:
Restkapazität des Rucksacks: CR



Identische Kapazität und wir möchten das gleiche Objekt einfügen

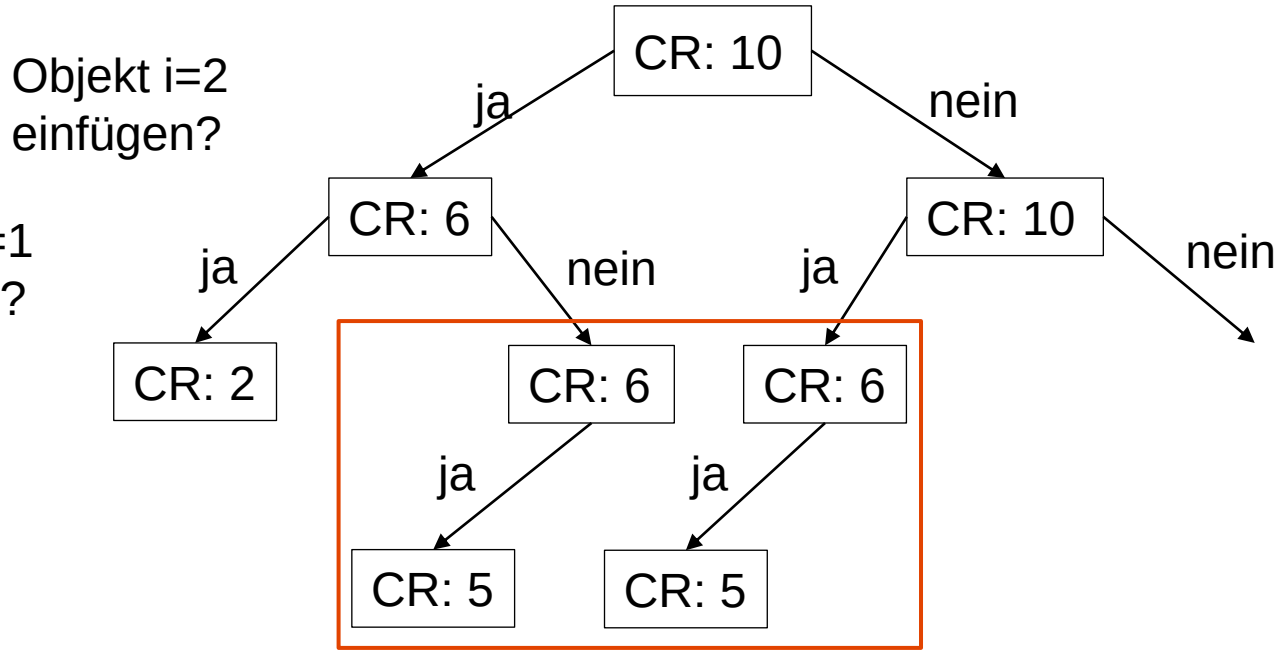
Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

0-1-Rucksackproblem: Beispiel

i	0	1	2
w_i	1	4	4
v_i	5	3	5

Kapazität: 10
Anzahl Objekte (n): 3

In jeder Iteration: Restkapazität des Rucksacks: CR



Teilproblem: Bei gegebener Restkapazität: welche Objekte fügen wir noch hinzu?

Identische Kapazität und wir möchten das gleiche Objekt einfügen

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

0-1-Rucksackproblem: Dyn. Programmieren mit Memoization

```
public int knapsack(int i, int cap, int[] w, int[] v, int[][] memo) {
   if (memo[i][cap] == NULL) {
      if (i == -1 || cap == 0) {                  // kein Platz mehr im Rucksack
         return 0;
      } else if (w[i] > cap) {                    // ites Objekt zu schwer für Rucksack
         memo[i][cap] = knapsack(i-1, cap, w, v, memo);
      } else {                          // ites Objekt einfügen oder nicht? Was ist besser?
         int vno = knapsack(i-1, cap, w, v, memo);
         int vyes = v[i] + knapsack(i-1, cap - w[i], w, v, memo);
         memo[i][cap] = Math.max(vno, vyes);
      }
   }
   return memo[i][cap];
}
```

0-1-Rucksackproblem: Dyn. Programmieren mit Memoization

```
public int knapsack(int i, int cap, int[] w, int[] v, int[][] memo) {
   if (memo[i][cap] == NULL) {
      if (i == -1 || cap == 0) {                  // kein Platz mehr im Rucksack
         return 0;
      } else if (w[i] > cap) {                    // ites Objekt zu schwer für Rucksack
         memo[i][cap] = knapsack(i-1, cap, w, v, memo);
      } else {                              // ites Objekt einfügen oder nicht? Was ist besser?
         int vno = knapsack(i-1, cap, w, v, memo);
         int vyes = v[i] + knapsack(i-1, cap - w[i], w, v, memo);
         memo[i][cap] = Math.max(vno, vyes);
      }
   }
   return memo[i][cap];
}
```

memo[i-1][cap] stellt den maximalen Wert dar, der mit einem Rucksack mit einer Restkapazität „cap“ unter Berücksichtigung der ersten i Elemente erreicht werden kann.

0-1-Rucksackproblem: Dyn. Programmieren mit Memoization

```
public int knapsack(int i, int cap, int[] w, int[] v, int[][] memo) {
   if (memo[i][cap] == NULL) {
      if (i == -1 || cap == 0) {                // kein Platz mehr im Rucksack
         return 0;
      } else if (w[i] > cap) {                  // ites Objekt zu schwer für Rucksack
         memo[i][cap] = knapsack(i-1, cap, w, v, memo);
      } else {                              // ites Objekt einfügen oder nicht? Was ist besser?
         int vno = knapsack(i-1, cap, w, v, memo);
         int vyes = v[i] + knapsack(i-1, cap - w[i], w, v, memo);
         memo[i][cap] = Math.max(vno, vyes);
      }
   }
   return memo[i][cap];
}
```

Laufzeit: O(n*Kapazität)
Speicherplatzbedarf: O(n*Kapazität)

0-1-Rucksackproblem: Dyn. Programmieren mit Memoization

```
public int knapsack(int i, int cap, int[] w, int[] v, int[][] memo) {
   if (memo[i][cap] == NULL) {
      if (i == -1 || cap == 0) {                  // kein Platz mehr im Rucksack
         return 0;
      } else if (w[i] > cap) {                    // ites Objekt zu schwer für Rucksack
         memo[i][cap] = knapsack(i-1, cap, w, v, memo);
      } else {                              // ites Objekt einfügen oder nicht? Was ist besser?
         int vno = knapsack(i-1, cap, w, v, memo);
         int vyes = v[i] + knapsack(i-1, cap - w[i], w, v, memo);
         memo[i][cap] = Math.max(vno, vyes);
      }
   }
   return memo[i][cap];
}
```

Laufzeit: O(n*Kapazität)
Speicherplatzbedarf: O(n*Kapazität)

Kapazität, Gewichte und n dürfen nur Ganzzahlen sein

0-1-Rucksackproblem: Bottom-Up-Ansatz

- Definiere 2-D Array A: (Anzahl Objekte n+1, **Kapazität+1** des Rucksacks)
-
  -
-
  -
    -
-

0-1-Rucksackproblem: Bottom-Up-Ansatz

- Definiere 2-D Array A: (Anzahl Objekte n+1, **Kapazität+1** des Rucksacks)
- Initialisierung: (Egal wie Restkapazität, wenn man kein Objekt mitnimmt, ist Wert des Rucksacks 0)
  - A[0, x] = 0 für x = 0, 1, 2, …, Kapazität
-
  -
    -
-

A[i, x]: maximal möglicher Wert des Rucksacks mit den Objekten 0, …, i - 1 und der Restkapazität von x

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Greedy-Algorithmen, Minimum Spanning Trees und dynamische Programmierung, Stanford University, Tim Roughgarden, Coursera

0-1-Rucksackproblem: Bottom-Up-Ansatz

- Definiere 2-D Array A: (Anzahl Objekte n+1, **Kapazität+1** des Rucksacks)
- Initialisierung: (Egal wie Restkapazität, wenn man kein Objekt mitnimmt, ist Wert des Rucksacks 0)
  - A[0, x] = 0 für x = 0, 1, 2, …, Kapazität

- For i= 0, 1, 2, 3, …, n-1:
  - For x = 0, 1, 2, 3, …, Kapazität:
    - A[i + 1, x] := max{A[i, x], A[i, x - w_i] + v_i}

- Return A[n, Kapazität]

A[i, x]: maximal möglicher Wert des Rucksacks mit den Objekten 0, …, i - 1 und der Restkapazität von x

0-1-Rucksackproblem: Bottom-Up-Ansatz

- Definiere 2-D Array A: (Anzahl Objekte n+1, **Kapazität+1** des Rucksacks)
- Initialisierung: (Egal wie Restkapazität, wenn man kein Objekt mitnimmt, ist Wert des Rucksacks 0)
  - A[0, x] = 0 für x = 0, 1, 2, …, Kapazität

- For i= 0, 1, 2, 3, …, n-1:
  - For x = 0, 1, 2, 3, …, Kapazität:
    - $A[i + 1, x] := \max\{A[i, x], A[i, x - w_i] + v_i\}$

- Return A[n, Kapazität]



2. Fall ignorieren, wenn $w_i > x$

A[i, x]: maximal möglicher Wert des Rucksacks mit den Objekten 0, …, i - 1 und der Restkapazität von x

0-1-Rucksackproblem: Bottom-Up-Ansatz

- Definiere 2-D Array A: (Anzahl Objekte n+1, **Kapazität+1** des Rucksacks)
- Initialisierung: (Egal wie Restkapazität, wenn man kein Objekt mitnimmt, ist Wert des Rucksacks 0)
  - A[0, x] = 0 für x = 0, 1, 2, …, Kapazität

- For i= 0, 1, 2, 3, …, n-1:
  - For x = 0, 1, 2, 3, …, Kapazität:
    - A[i + 1, x] := max{A[i, x], A[i, x - w_i] + v_i}

- Return A[n, Kapazität]



2. Fall ignorieren, wenn w_i > x

A[i, x]: maximal möglicher Wert des Rucksacks mit den Objekten 0, …, i - 1 und der Restkapazität von x

Laufzeit: O(n*Kapazität)
Speicherplatzbedarf: O(n*Kapazität)

Kapazität, Gewichte und n dürfen nur Ganzzahlen sein

0-1-Rucksackproblem: Bottom-Up-Ansatz

Übungsblatt 10
Aufgabe 2, 3b, 5, 4

- Definiere 2-D Array A: (Anzahl Objekte n+1, **Kapazität+1** des Rucksacks)
- Initialisierung: (Egal wie Restkapazität, wenn man kein Objekt mitnimmt, ist Wert des Rucksacks 0)
  - A[0, x] = 0 für x = 0, 1, 2, …, Kapazität

- For i= 0, 1, 2, 3, …, n-1:
  - For x = 0, 1, 2, 3, …, Kapazität:
    - A[i + 1, x] := max{A[i, x], A[i, x - w_i] + v_i}

- Return A[n, Kapazität]



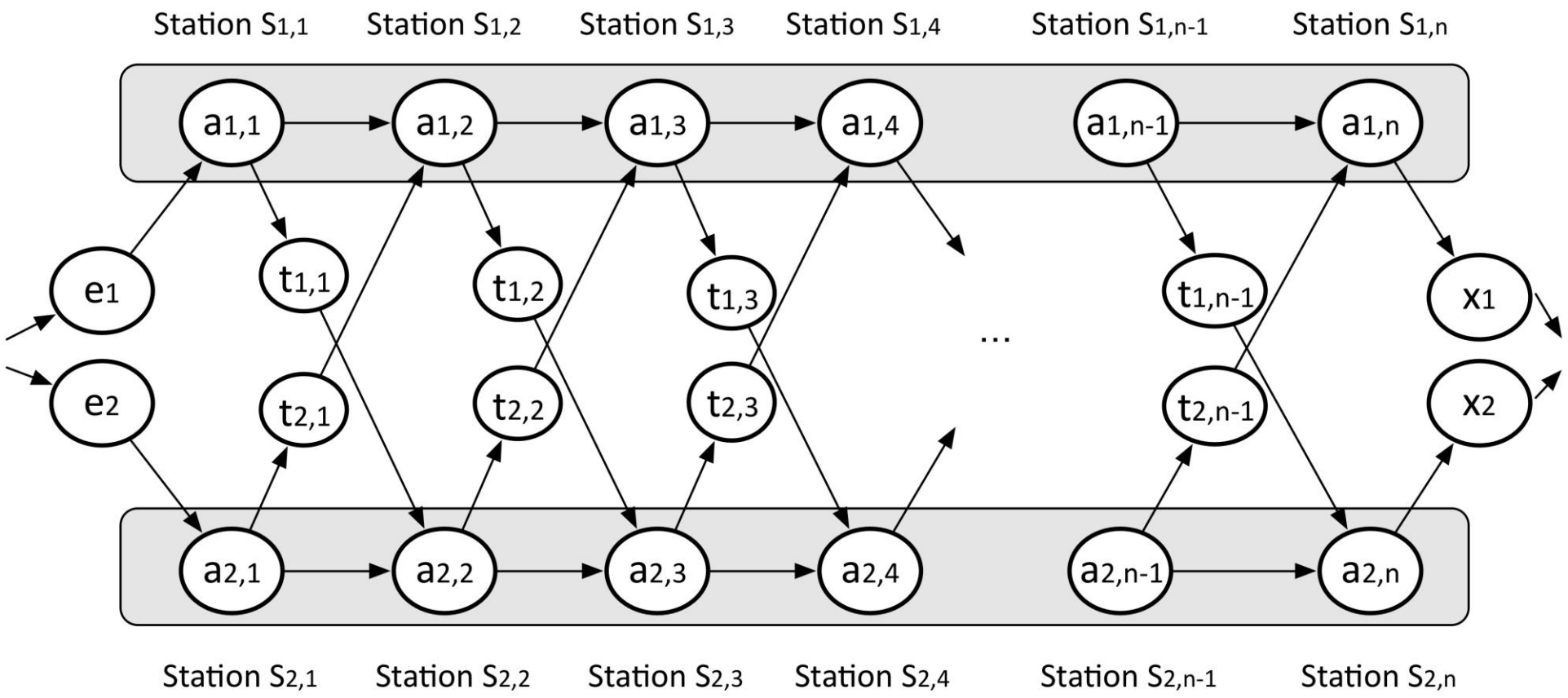
2. Fall ignorieren, wenn w_i > x

A[i, x]: maximal möglicher Wert des Rucksacks mit den Objekten 0, …, i - 1 und der Restkapazität von x

Laufzeit: O(n*Kapazität)
Speicherplatzbedarf: O(n*Kapazität)

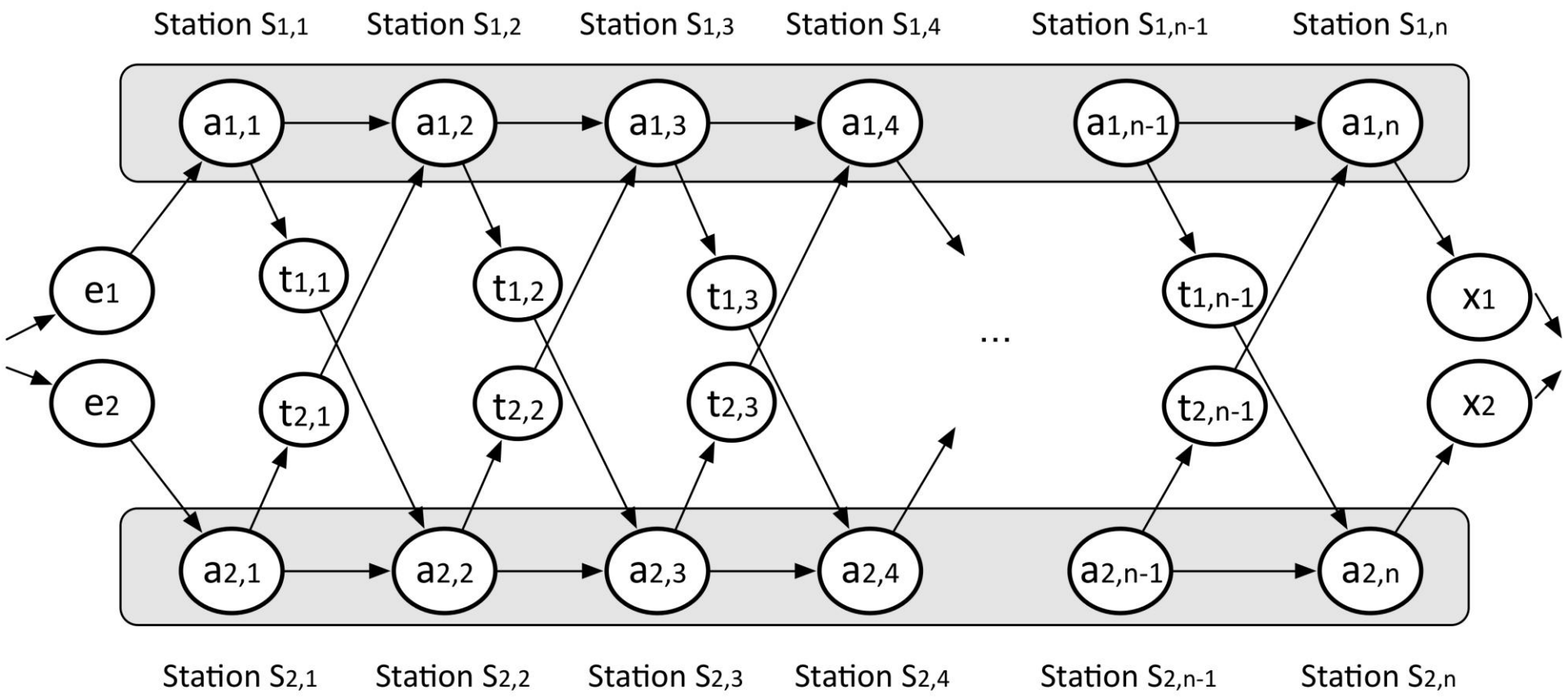
Kapazität, Gewichte und n dürfen nur Ganzzahlen sein

Line Scheduling



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Prof. Dr. Heiner Klocke, Algorithmik, TH Köln, WS 21/22

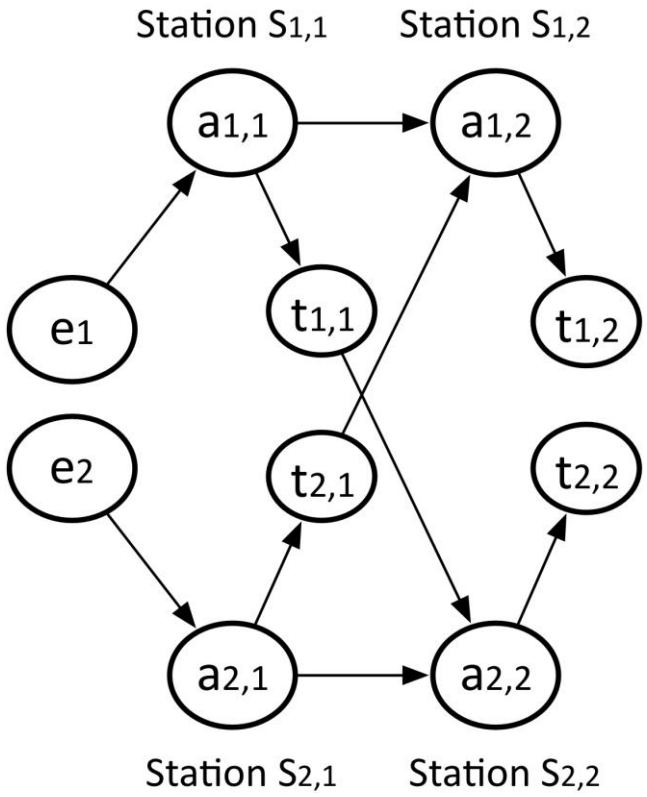
Line Scheduling



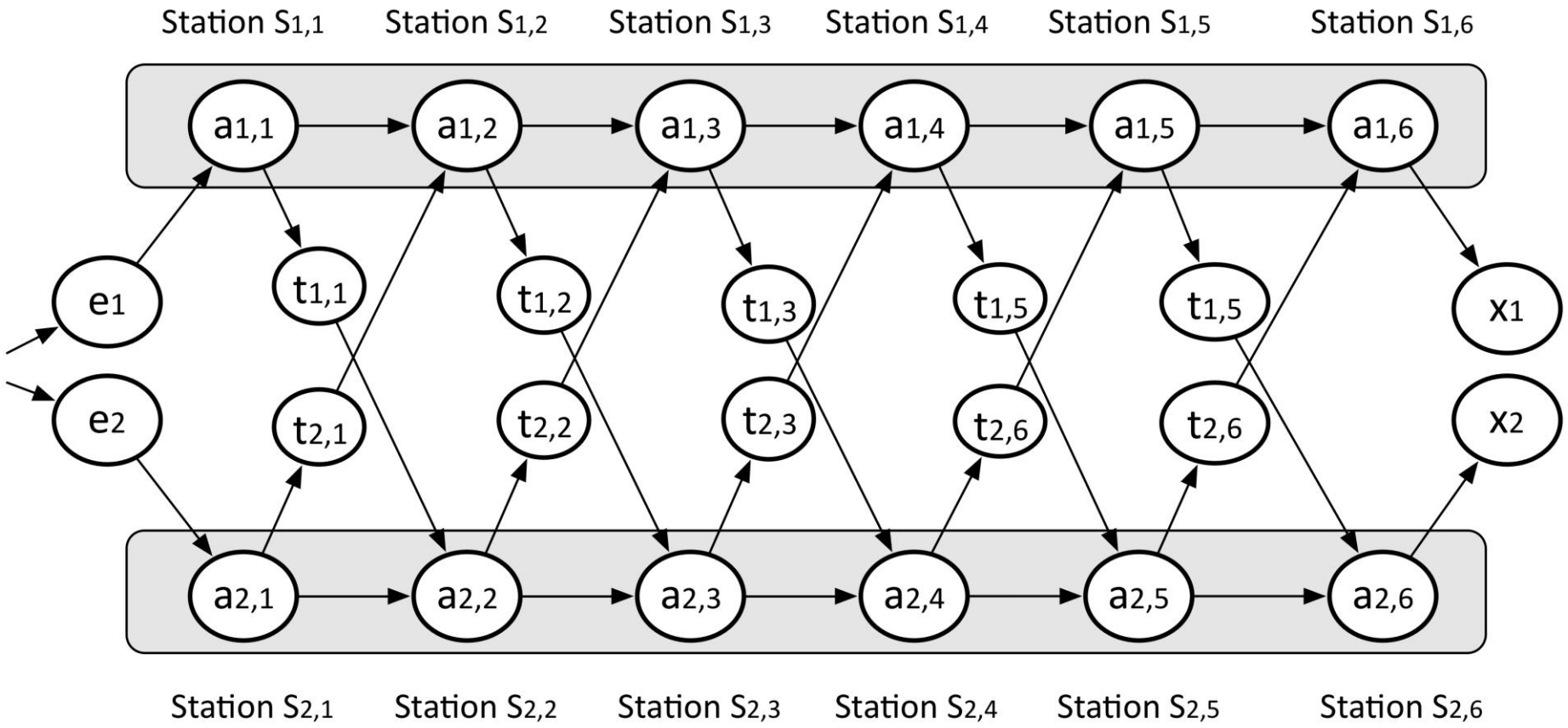
Optimierungsaufgabe: Minimierung der Fertigungszeit

Line Scheduling - Bezeichnungen

- $S_{i,j}$: j-te Station auf Band i = 1, 2
- $S_{1,j}$ gleiche Funktion wie $S_{2,j}$
- Montagezeiten $a_{1,j} \neq a_{2,j}$
- e_i = Eintrittszeit in Band i
- $t_{i,j}$ = Transferkosten
- x_i = Austrittszeit von Band i
- n Stationen, j = 1, 2, …, n

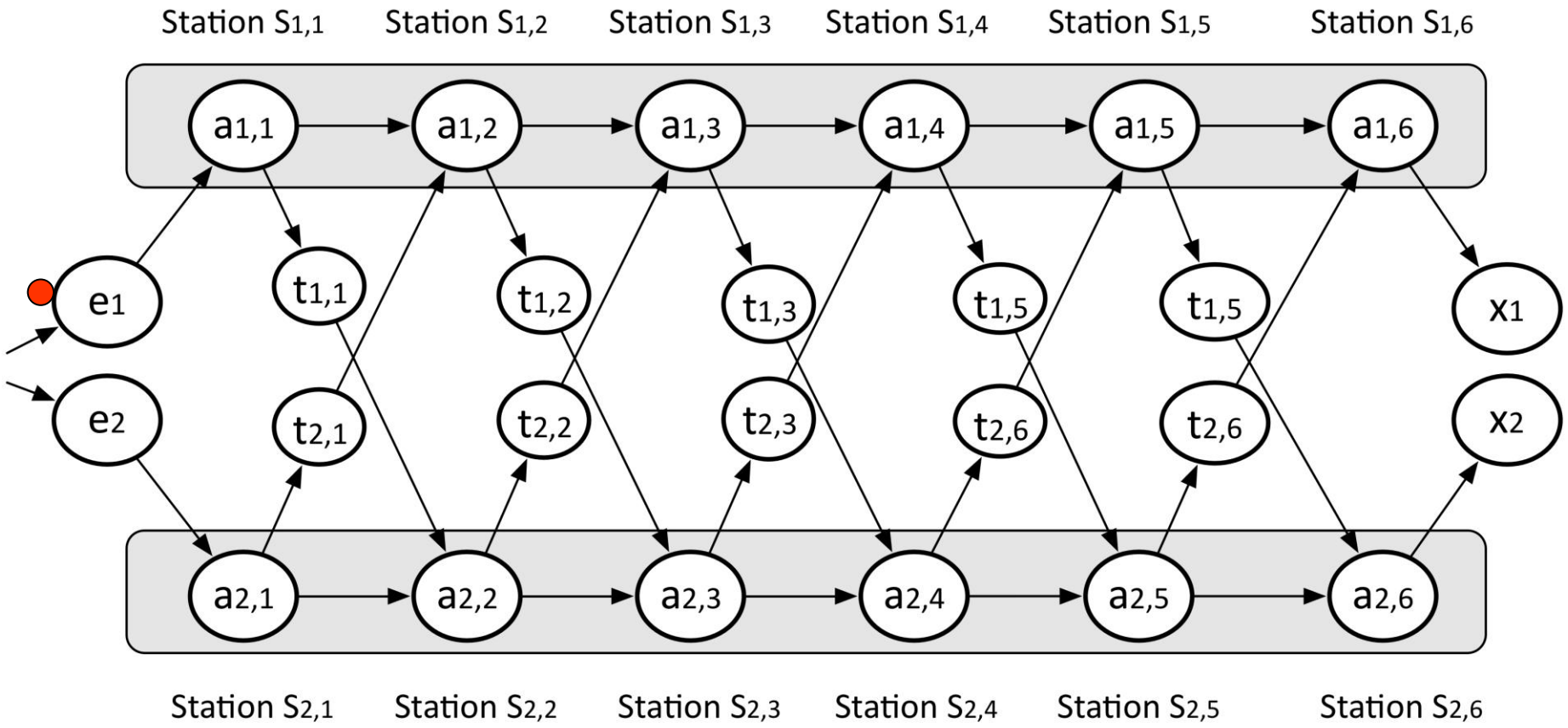


Line Scheduling - Beispiel



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Prof. Dr. Heiner Klocke, Algorithmik, TH Köln, WS 21/22

Line Scheduling - Beispiel



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Prof. Dr. Heiner Klocke, Algorithmik, TH Köln, WS 21/22

Sequence Alignment von DNA

Um zwei genetische Merkmale auf **Ähnlichkeit** zu prüfen, müssen die entsprechenden Aminosäuresequenzen verglichen und zueinander ausgerichtet werden – **Sequenzen Alignment**.

```
AGGCTATCACCTGACCTCCAGGCCGA TGCCC
TAGCTATCACGACCGCGGTCGATTTGCCCGAC
```


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Prof. Dr. Heiner Klocke, Algorithmik, TH Köln, WS 21/22

Sequence Alignment von DNA

Um zwei genetische Merkmale auf **Ähnlichkeit** zu prüfen, müssen die entsprechenden Aminosäuresequenzen verglichen und zueinander ausgerichtet werden – **Sequenzen Alignment**.

Mit Hilfe einer Kostenfunktion (**alignment score**) werden zwei Aminosäuresequenzen so ausgerichtet, dass ihr Alignmentwert maximal ist.

Durch einen Alignment-Algorithmus werden die Sequenzen so übereinander geschrieben, dass jedes Element der ersten Sequenz einem der zweiten Sequenz oder einer Lücke (**Gap**) zugeordnet ist. Die Reihenfolge der Aminosäuren wird **nicht** verändert.

```
AGGCTATCACCTGACCTCCAGGCCGA TGCCC
TAGCTATCACGACCGCGGTCGATTTGCCCGAC
```

```
-AGGCTATCACCTGACCTCCAGGCCGA--TGCCC---
TAG-CTATCAC--GACCGC--GGTCGATTTGCCCGAC
```


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Prof. Dr. Heiner Klocke, Algorithmik, TH Köln, WS 21/22

Weitere Beispiele für dynamische Programmierung

- https://www.geeksforgeeks.org/fundamentals-of-algorithms/

- https://www.geeksforgeeks.org/find-the-minimum-cost-to-reach-a-destination-where-every-station-is-connected-in-one-direction/
- https://www.geeksforgeeks.org/shortest-path-for-directed-acyclic-graphs/
- https://www.geeksforgeeks.org/find-minimum-number-of-coins-that-make-a-change/

- Bellman-Ford-Algorithmus (kürzester Pfad mit negativen Kantengewichten)
- Floyd-Warshall-Algorithmus (kürzeste Pfade zwischen allen Knoten (nicht nur einem Startknoten))

Helpful walk through video:

5 Simple Steps for Solving Dynamic Programming Problems

Dynamische Programmierung ist sinnvoll,

wenn

- das Problem rekursiv formuliert werden kann.
- das Problem in Teilprobleme zerlegbar ist, die sich wiederholen.
- genügend Speicherplatz zur Zwischenspeicherung (Memoization) oder Vorberechnung (Bottom-Up) zur Verfügung steht.
- die Funktionsparameter ganzzahlig sind.
- Sie keine bessere Idee haben!

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: Prof. Dr. Heiner Klocke, Algorithmik,
TH Köln, WS 21/22

Dynamische Programmierung

Wie kam es zu dem Begriff?

Bellman explains the reasoning behind the term *dynamic programming* in his autobiography, *Eye of the Hurricane: An Autobiography*:

„The 1950s were not good years for mathematical research. […] Hence, I felt I had to do something to shield […] the Air Force from the fact that I was really doing mathematics inside the RAND Corporation. […] Thus, I thought dynamic programming was a good name. It was something not even a Congressman could object to. So I used it as an umbrella for my activities."

Lernziele von heute und Fragen zur Überprüfung der Lernziele

Die Studierenden können bestimmte rekursive Problemstellungen durch Dynamische Programmierung sehr effizient lösen, indem sie sich überlappende Teilprobleme identifizieren, einen rekursiven Algorithmus entwickeln und mit der Technik Memoization Zwischenergebnisse abspeichern, um später bestimmte rekursive Problemstellungen effizient lösen zu können.

Hinweis: Dieses Lernziel ist nicht erreicht

Potenzielle Klausuraufgaben s. Übungsblatt 10

Lernziele von heute und Fragen zur Überprüfung der Lernziele

Die Studierenden können gegebene rekursive Brute-Force-Lösungen für Optimierungs- und Zählprobleme (z. B. Fibonacci, Treppensteigen, 0-1-Rucksack) mithilfe von **Dynamischer Programmierung** effizienter machen, indem sie

(1) anhand des **Rekursionsbaums** mehrfach auftretende **Teilprobleme** (gleiche Parameterkombinationen) erkennen,

(2) diese durch **Memoization** zwischenspeichern und

(3) die Berechnungsreihenfolge der Teilprobleme in einen **iterativen Bottom-Up-Algorithmus** überführen,

um Laufzeit und Speicherbedarf der Lösungen zu bewerten und bei späteren Problemen mit **überlappenden Teilproblemen** von exponentieller auf polynomielle Laufzeit zu kommen.

Übungsblatt 10, Aufgabe 9

Zukunft der Algorithmik

- KI zur Generierung von effizienten Algorithmen:
  - Matrixmultiplikation
    - Fawzi, A., Balog, M., Huang, A. et al. Discovering faster matrix multiplication algorithms with reinforcement learning. Nature 610, 47–53 (2022). https://doi.org/10.1038/s41586-022-05172-4
  - Sortieralgorithmen
    - Daniel J Mankowitz, Andrea Michi, Anton Zhernov, Marco Gelmi, Marco Selvi, Cosmin Paduraru, Edouard, Leurent, Shariq Iqbal, Jean-Baptiste Lespiau, Alex Ahern, et al. Faster sorting algorithms discovered using deep reinforcement learning. Nature, 618(7964):257–263, 2023.
  - Online Bin Packing
    - Romera-Paredes, B., Barekatain, M., Novikov, A. et al. Mathematical discoveries from program search with large language models. Nature (2023). https://doi.org/10.1038/s41586-023-06924-6

Zukunft der Algorithmik

- KI zur Generierung von effizienten Algorithmen:
  - https://github.com/openai/math/blob/main/preprints/The-Exponential-Time-Hypothesis-is-False-October-25-2026/paper.pdf

Was Sie jetzt können und was Sie mit dem Gelernten anfangen können

Ihre erworbenen Kompetenzen:

- Sie können praktische Problemstellungen analysieren und formal beschreiben.
-
-
-

Was Sie jetzt können und was Sie mit dem Gelernten anfangen können

Ihre erworbenen Kompetenzen:

- Sie können praktische Problemstellungen analysieren und formal beschreiben.
- Sie wählen für Probleme passende Datenstrukturen aus und nutzen diese effektiv.
-
-

Was Sie jetzt können und was Sie mit dem Gelernten anfangen können

Ihre erworbenen Kompetenzen:

- Sie können praktische Problemstellungen analysieren und formal beschreiben.
- Sie wählen für Probleme passende Datenstrukturen aus und nutzen diese effektiv.
- Sie verstehen die Funktionsweise von einigen Algorithmen, können diese anwenden, implementieren und an spezifische Problemstellungen anpassen.
-

Was Sie jetzt können und was Sie mit dem Gelernten anfangen können

Ihre erworbenen Kompetenzen:

- Sie können praktische Problemstellungen analysieren und formal beschreiben.
- Sie wählen für Probleme passende Datenstrukturen aus und nutzen diese effektiv.
- Sie verstehen die Funktionsweise von einigen Algorithmen, können diese anwenden, implementieren und an spezifische Problemstellungen anpassen.
- Sie sind in der Lage, die Laufzeit und den Speicherbedarf eines Algorithmus systematisch zu bewerten und zu optimieren.

Was Sie jetzt können und was Sie mit dem Gelernten anfangen können

Ihr Nutzen in Studium und Beruf:

- Sie können reale Probleme in Informatikprojekten effizient lösen.
-
-
-

Was Sie jetzt können und was Sie mit dem Gelernten anfangen können

Ihr Nutzen in Studium und Beruf:

- Sie können reale Probleme in Informatikprojekten effizient lösen.
- Sie haben ein solides Fundament für fortgeschrittene Themen wie Netzwerksicherheit oder Künstliche Intelligenz.
-
-

Was Sie jetzt können und was Sie mit dem Gelernten anfangen können

Ihr Nutzen in Studium und Beruf:

- Sie können reale Probleme in Informatikprojekten effizient lösen.
- Sie haben ein solides Fundament für fortgeschrittene Themen wie Netzwerksicherheit oder Künstliche Intelligenz.
- **Kompetitive Vorteile**: Ihr Verständnis von Algorithmen und Datenstrukturen wird Ihnen helfen, leistungsfähigere Software zu entwickeln und sich im Beruf zu behaupten.
-

Was Sie jetzt können und was Sie mit dem Gelernten anfangen können

Ihr Nutzen in Studium und Beruf:

- Sie können reale Probleme in Informatikprojekten effizient lösen.
- Sie haben ein solides Fundament für fortgeschrittene Themen wie Netzwerksicherheit oder Künstliche Intelligenz.
- **Kompetitive Vorteile**: Ihr Verständnis von Algorithmen und Datenstrukturen wird Ihnen helfen, leistungsfähigere Software zu entwickeln und sich im Beruf zu behaupten.
- **Lebenslanges Lernen**: Sie haben Werkzeuge und Denkweisen erworben, die Ihnen helfen, neue Technologien und Konzepte eigenständig zu erlernen.

Ende der Vorlesung und Fragen

- Fragen?

Glückwunsch, dass Sie es bis hierhin geschafft haben!

Viel Erfolg bei der Klausur!

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln